

Amber Protocol

Parish Lab

Table of Contents

Introduction.....	4
Amber file organization	4
User manuals.....	4
Online tutorials.....	4
Remotely logging in from a personal laptop.....	5
Windows 10	5
Mac OS X	8
Programs for your personal laptop.....	9
UCSF Chimera.....	9
UCSF ChimeraX.....	17
Other Programs	20
Preparing an Amber job	21
Basic ligand preparation	22
Advanced ligand preparation	24
Charged and Hypervalent Ligands.....	24
Using Gabedit	25
Using a script	27
Modified Residues and Covalent Ligands	32
Preparing an Input Structure for RESP Fitting	33
PREP and PREPi File Format.....	35
Editing the PREPi File	39
Editing the PDB for MD	55
Addressing Missing Parameters.....	57
Creating initial Amber topology and coordinate files.....	57
Other <i>tleap</i> Example Files.....	66
A Note on Histidine Residues.....	71
Running an Amber Job	72
CPU vs GPU	73
Available GPUs	73

Selecting GPUs for use	74
On a local Parish lab computer	74
Thais and AMBER18.....	84
On URSPDR	85
On Vadur.....	87
On Urarana.....	89
On MERCURY (Marcy).....	91
On NERSC.....	92
Invoking Amber (<i>pmemd.cuda</i>)	99
In serial.....	99
In parallel	100
Example mdin files	101
Minimization.....	102
Heating.....	105
Equilibration	107
Unrestrained MD	110
Analysis of Unrestrained MD Simulations	112
<i>cpptraj</i>	112
Concatenating and imaging trajectories.....	112
Obtaining RMSD, RMSF, Distances and angles	115
Calculating RMSD from a Specific Reference Structure	117
Sorting Frames of a Trajectory by Specific Criteria	119
Calculating the Surface Accessible Surface Area (SASA) of a Complex	120
Hydrogen Bonding Analysis.....	121
Other Analyses.....	125
Plotting data using gnuplot	125
MMPBSA.py & MMPBSA.py.MPI	131
On Unusual Solvent and Ions.....	135
Umbrella MD Simulations.....	137
Example mdin files for Umbrella MD	137
Minimization.....	137
Heating.....	138
Equilibration	139

Restrained MD	140
Restraint File	141
Analysis of Umbrella MD Simulations.....	142
Using gnuplot to generate histograms.....	142
Weighted Histogram Analysis Method (WHAM)	145
Downloading and Compiling Wham	145
<i>Compiling WHAM</i>	145
Executing WHAM	146
Analysis of Wham Calculations.....	147
Amber Troubleshooting	148
My simulation keeps dying at minimization and heating	148
I can't get MMGBSA/MMGBSA to run properly	150

Introduction

This protocol serves as a guide for running the Amber suite of programs to setup, run, and analyze molecular dynamics (MD) simulations.

Amber file organization

As mentioned above, Amber is a *suite* of programs. It isn't just one program, but a host of programs all meant to serve a different purpose. If you would like to see all the programs available with Amber, simply type 'ls' in the \$AMBERHOME/bin/ folder. There are a host of programs, most of which are not discussed in this manual. But when this protocol refers to a particular Amber program (or executable), that program will be in that bin folder.

Every year Amber releases what is called 'AmberTools.' AmberTools refers to a bunch of programs that Amber releases free-of-charge that can do a lot of things. In general, though, AmberTools means all Amber programs except for those that run MD. So all programs used to build the system and run analysis are released with AmberTools, while the actual programs that can be used to run MD are released every two years as part of 'Amber'.

User manuals

More detailed instructions and definitions of terms (that are not found in this protocol) can be found in the Amber 2018 manual. This manual covers the Amber18 and AmberTools 18 software suites. It can be found in the following directory on all workstations.

Amber 2018 manual: \$AMBERHOME/doc/Amber18.pdf

I would not advise reading through these manuals from cover-to-cover. They should be used as a reference. If you want to know how to do a particular type of simulation, read that section. If you need to know what a variable in the input file means, look it up. That's the real purpose of these manuals.

Online tutorials

Great Amber tutorials can be found online here:

<http://ambermd.org/tutorials/>

The best tutorial for beginner Amber users can be found here:

<http://ambermd.org/tutorials/basic/tutorial1/>

We will actually be covering a lot of the information from Tutorial 1 in this protocol, so there is a decent amount of overlap between the two.

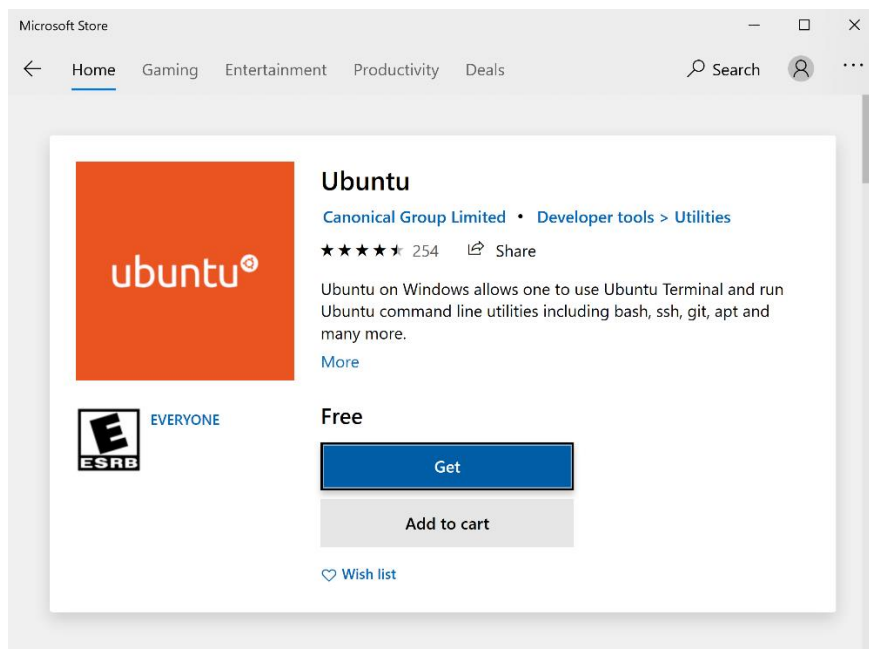
Note that some of these tutorials can be somewhat old because they refer to outdated versions of Amber. So if you read through the tutorials and they mention using Amber 14 or Amber 16, just know that the tutorials can still be applied to our current version of Amber (Amber 18).

Remotely logging in from a personal laptop

In many instances, it is more convenient and time efficient to log into the local Parish lab workstations remotely rather than physically going into the lab and logging into the consoles. For example, if all you need to do is check on a simulation, it is more convenient to do so from your personal laptop rather than heading into the lab. This can be done from any personal laptop running Windows 10 or Mac OS X. The following sections will detail how to do this for both operating systems.

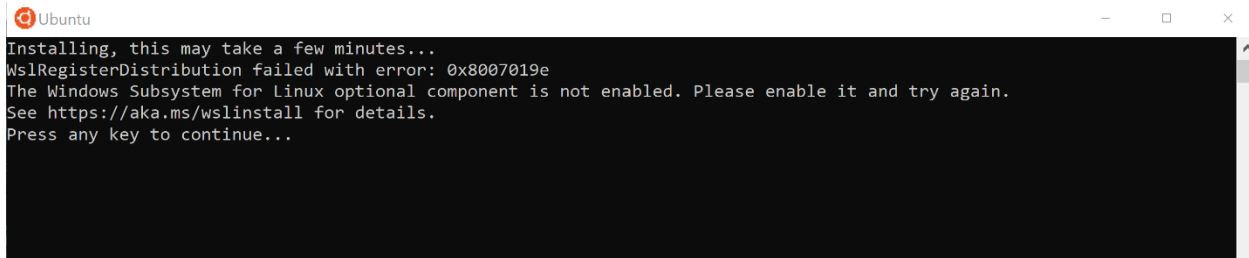
Windows 10

Unlike on a Mac, in order to remotely log into a local workstation or GPU cluster from a Windows 10 PC, you will need to install a program to set things up. First, in the Windows search bar, search for the Microsoft Store and launch the app. Once in the store, enter the term 'Ubuntu' into the search bar. This should bring up the app pictured below.



This app will install an Ubuntu-based bash terminal on your Windows 10 PC.

Click on 'get'. It will prompt you for your password or login pin. Enter this, and wait for the program to install. Once the install finishes, in the Windows search bar search for the Ubuntu app and launch it. This will launch a terminal that looks like this.



```

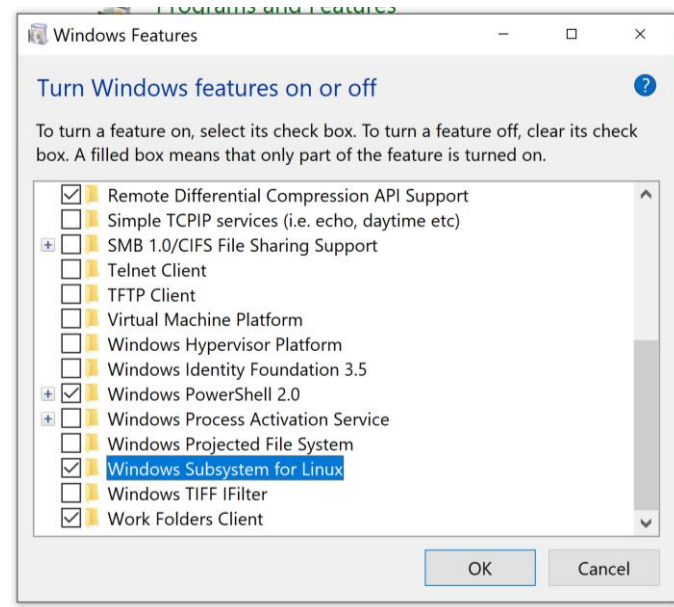
Installing, this may take a few minutes...
WslRegisterDistribution failed with error: 0x8007019e
The Windows Subsystem for Linux optional component is not enabled. Please enable it and try again.
See https://aka.ms/wslinstall for details.
Press any key to continue...

```

Before you can successfully run the bash terminal, you'll need to enable the Windows Subsystem for Linux. In the Windows search bar, search for the Control Panel. Launch this app, and click on 'Programs'. Within programs, click on the underlined link in the image below.

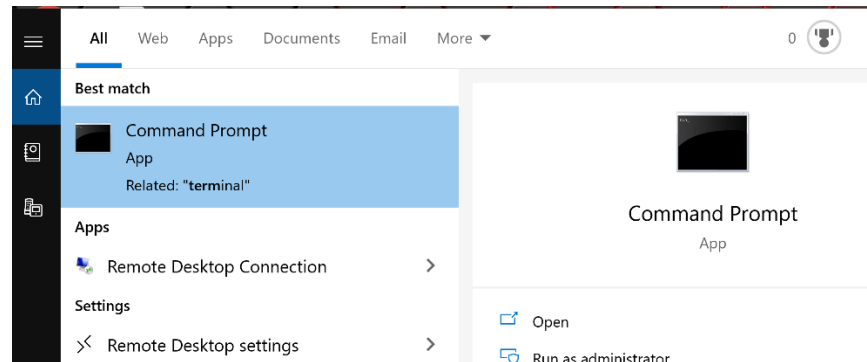


This will then open a long list of features that can be enabled. Scroll down near the bottom until you see Windows Subsystem for Linux. Check the box next to this option and hit ok.



It'll take a minute to install the necessary files. Once it's finished, you'll be prompted to restart your computer. After restarting, open the Ubuntu app again. It will then install more files. Once finished, it will ask you to enter a Unix username. Enter your netid here. Next, it will ask you to enter a password. This can be anything. It doesn't need to be your actual Richmond netid password. This password is only for running root commands on your own laptop, and unless you do this frequently you will not need to enter this password again.

Once you've finished this, close out of the Ubuntu app. From here, we'll do everything from the command prompt instead. This is because the command prompt is far more customizable than the Ubuntu app. Search in the Windows search bar for 'Command Prompt' and open the app.

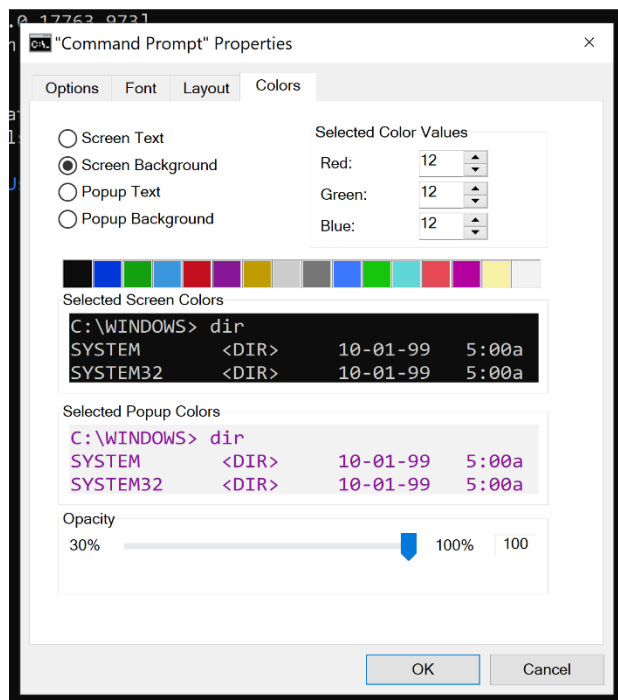


This will open a terminal on your laptop. Enter the command `bash` and this will run the Linux subshell on your laptop. All Unix commands (including `ls`, `cd`, `rsync`, etc.) will work within this terminal. You can navigate within the directories, list files, make folders, etc. on your laptop the same way as on the Parish lab workstations, using the same commands. Note that every time you open a new command prompt, you will need to first type the command `bash`.

To remotely log into any of the Parish lab workstations or GPU clusters, simply type `ssh workstation` as you would on any of the local workstations. For example, if I were to log into Thais from my laptop within this terminal, I would enter `ssh thais`.

If you are trying to `rsync` files from a local workstation to your laptop, you cannot use `rsync -azvp -progress filename workstation:/pathway` to send files from a workstation to your laptop. This is because your laptop doesn't have the necessary permissions for this to occur. You instead need to use the terminal on your laptop to navigate to the directory you want your files in on your laptop, and enter the command `rsync -avz workstation:/pathway/'filename' .` (Note that the period is part of the command). This will pull the files from a local workstation into the folder on your laptop where you entered this command.

You can also customize your terminal to change the font style, size, background colors, etc. To do this, just right-click on the top bar of the terminal and click on 'properties'. This will open a window that looks like the following.

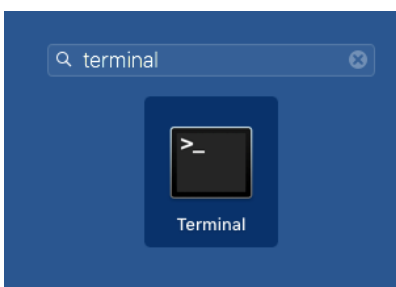


From here, you can customize things to your heart's content.

The last thing to note is that to copy and paste a filename, pathway, etc. within this terminal, you cannot highlight and click with the scroll wheel like on the local workstations. You'll instead need to highlight what you want to copy, and type CTRL + C. To paste, just right-click within the terminal.

Mac OS X

Unlike Windows 10 PCs, remotely logging into the Parish lab workstations and GPU clusters on a Mac requires no additional work to set things up. Simply search in your apps for 'Terminal' and open the app.



This will open a terminal on your laptop. All Unix commands (including `ls`, `cd`, `rsync`, etc.) will work within this terminal. You can navigate within the directories, list files, make folders, etc. on your laptop the same way as on the Parish lab workstations, using the same commands.

To remotely log into any of the Parish lab workstations, simply type the command `ssh netid@workstation`. If I were to use this command to remotely log into Thais, I would type `ssh ja9ia@thais`. Note that unlike on the local workstations, you will be required to type your netid before the name of the workstation you are trying to `ssh` into. This is because your username on your personal laptop is likely not the same as your netid, whereas on the local workstations your username is your netid. To successfully log into your username on the local workstations, you need to include your netid in your `ssh` command. Once you have logged into a local workstation however, you can `ssh` from that workstation to another using the normal `ssh workstation` command without your netid.

If you are trying to `rsync` files from a local workstation to your laptop, you cannot use `rsync -azvp -progress filename workstation:/pathway` to send files from a workstation to your laptop. This is because your laptop doesn't have the necessary permissions for this to occur. You instead need to use the terminal on your laptop to navigate to the directory you want your files in on your laptop, and enter the command `rsync -avz workstation:/pathway/'filename' .` (Note that the period is part of the command). This will pull the files from a local workstation into the folder on your laptop where you entered this command.

The last thing to note is that to copy and paste a filename, pathway, etc. within this terminal, you cannot highlight and click with the scroll wheel like on the local workstations. You'll instead need to highlight what you want to copy, and type command + C. To paste, just type command + V within the terminal.

Programs for your personal laptop

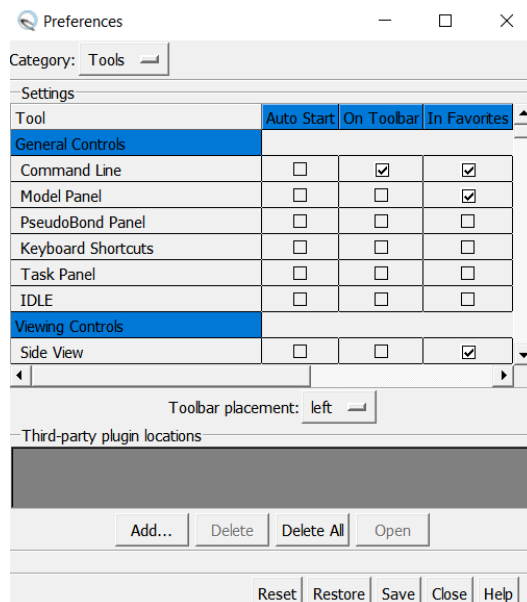
While most data analysis is conducted using the lab workstations, sometimes it is more convenient to conduct data analysis from your personal laptop. The following sections detail some of the programs most frequently used for data analysis in the Parish lab.

UCSF Chimera

UCSF Chimera is the primary program we use to visualize PDBs, generate images from simulations, create MD movies, etc on the local workstations. To do this from your personal laptop rather than on the workstations, you should install the latest version of the program UCSF Chimera (version 1.14 or newer). You can find both the Windows 10 and Mac OS X installs at the following link.

<https://www.cgl.ucsf.edu/chimera/download.html>

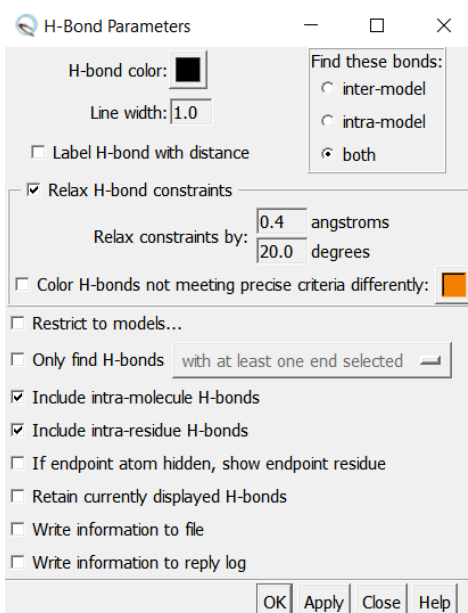
Chimera allows you to place shortcuts for frequently used tools on the left-hand side of the program window. This section is the toolbar. To add tools to this toolbar, search on the top-left menu for Favorites. Click on the Favorites menu, and then select Add to Favorites/Toolbar... This will open the following window.



You'll want to add the following programs to your toolbar: Command Line, FindHBond, Distances, Angles/Torsions, MatchMaker, Sequence, MD Movie, and Reply Log. These will generally be the most frequently used tools. Let's briefly discuss them.

The **Command Line** tool (rather shockingly) opens a command line at the bottom of the Chimera program window. This allows you to write commands in the command line rather than using the GUI. Generally, this won't be used all too commonly, however it's nice to have this in the toolbar just in case.

The **FindHBond** tool, like the Command Line, is self-explanatory. This program will draw lines between atoms participating in hydrogen bonds. Clicking on this tool will open the following window.

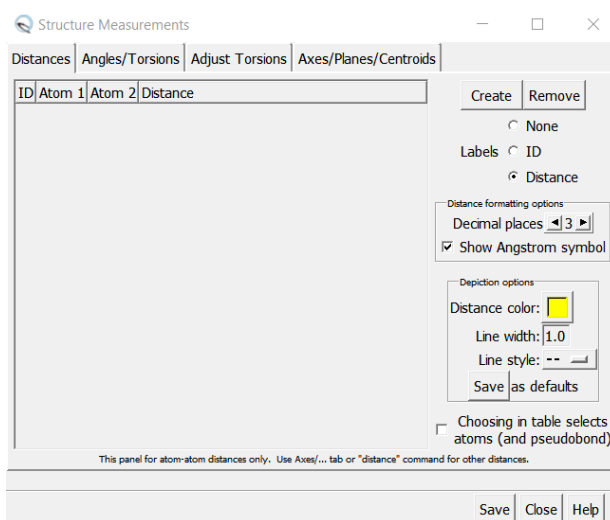


As you can see, this tool gives us quite a few options. Generally, I recommend keeping the options defining the hydrogen bonds themselves at their default values. However, depending on what you are looking for some of the other options might be useful.

The inter-model and intra-model option are useful for limited the hydrogen bonds that are displayed. The inter-model option is commonly used to show only the hydrogen bonds occurring between a protein and a ligand, whereas the intra-model option will only show hydrogen bonds occurring between two atoms in the protein or ligand.

Other useful options include the 'Label H-bond with distance', 'If endpoint atom hidden, show endpoint residue', and 'Write information to file options'. These are pretty self-explanatory as well.

The **Distances** tool will measure distances between two selected atoms and draw a line between them. The following window will open upon selecting this tool.



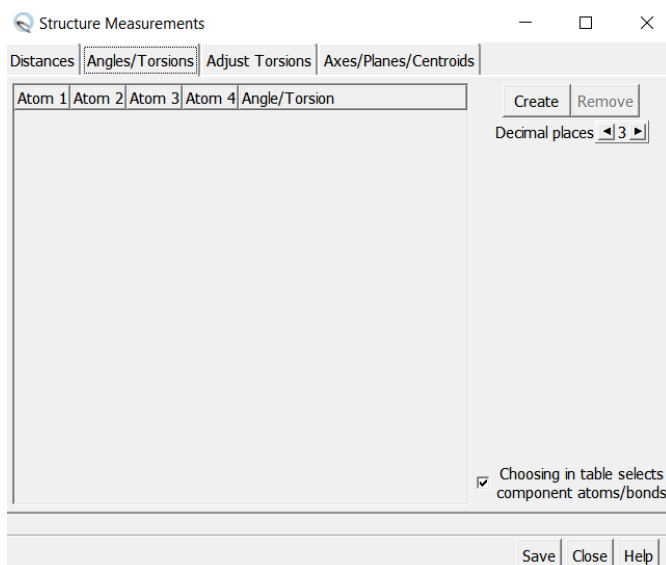
As you can see, you have some options here as well. On the right-hand side, you have several label options. However, given that this is the distance tool, I recommend keeping the Distance option selected. Other notable options are the line color (yellow by default, I recommend changing that) and the line style.

If you are using this tool, you'll want to select the two atoms of interest. This can be done by control-clicking the first atom and then shift-control-clicking the second atom. Then, simply click on Create on the right-hand side, and you'll see something like the following appear in your window.

ID	Atom 1	Atom 2	Distance
1	ARG 559.A NH2	ARG 559.A NH1	2.293Å

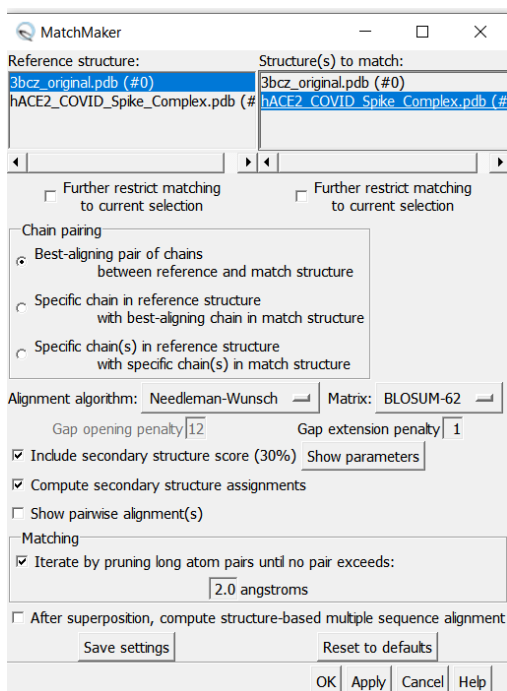
The ID is simply a number assigned to this measured distance by Chimera. Atom 1 is the first selected atom, and Atom 2 is the second. The distance is (again, quite shockingly) the distance in units of angstroms between the two atoms.

The **Angles/Torsions** tool will measure the angle between three atoms or the torsional angle between four atoms. The interface for this tool is very similar to that of the distances tool.



Like with the Distances tool, we'll select Atom 1 using control-click, and then every other atom with shift-control-click. Next, simply click on Create, and the angles and torsional information will be printed to the same window in a similar fashion to the distances.

Next up, we have the **MatchMaker** tool. This tool is used to perform an RMSD-based alignment of one structure to another.

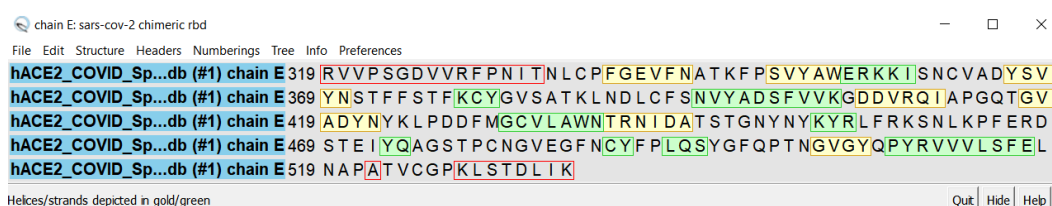


The most important part of this tool is selecting the Reference structure and the Structure(s) to match. The reference structure will be held in place, and all structures selected in the ‘Structure(s) to match’ section will be aligned to this reference structure.

In some cases, you might want to align structures based on the RMSD of specific regions of the two proteins. This can be done by selecting the region / amino acids of interest and clicking the ‘Further restrict matching to current selection’ option.

There are some instances where you might want to align structures differently. Different alignment methods can be selected in the Chain pairing section. By default, Chimera will align structures by reducing the distances between all atoms. However, you also have the option to align a structure to the best matching chain as well as the option to align a structure to multiple chains.

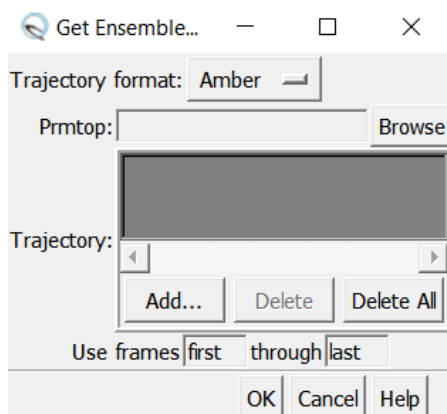
The **Sequence** tool will display the amino acid sequence of your protein. If your protein has multiple chains, it will first ask you to select the chain(s) you want to see the sequence of. The sequence will be displayed like the following example.



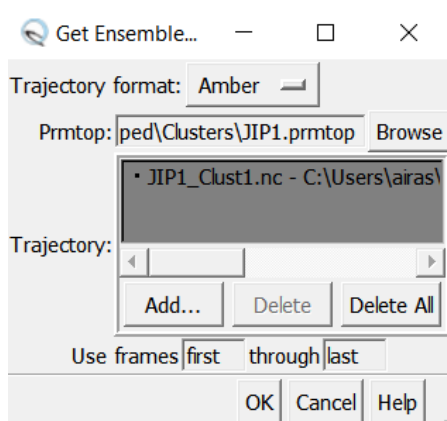
The bolded and highlighted text on the left indicates which model and chain this amino acid sequence belongs to. The numbering directly to the right of this indicates the amino acid number. The amino acid 1-letter code for all residues is shown. Residues highlighted in yellow form an alpha helix, whereas residues highlighted in green form a beta-sheet. The red box surrounding some of the residues indicates that while the PDB has these amino acids listed, their coordinates are missing from the structure itself. This isn't uncommon for a PDB downloaded from the Protein Data Bank, but ideally there would be no missing residues.

Within this window, you can select amino acids of interest just by clicking and dragging across the 1-letter codes. If you want to select multiple sections at once, hold the shift and control keys and drag across your new section. All selected amino acids will be highlighted in bright green. Previously selected amino acids will be highlighted in white.

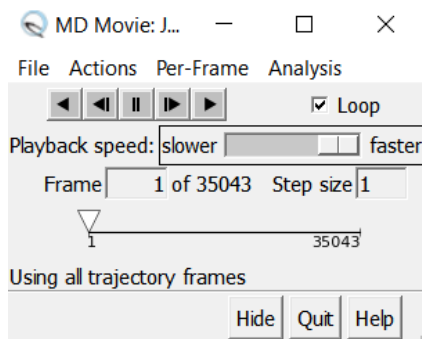
The **MD Movie** tool is perhaps the most important of this list. As you may have gathered, this tool will create a movie of your MD simulation. When you click on this tool, you'll see a window like the following.



First and foremost, the selected trajectory format should be Amber. This is of course because we use the program Amber to run our MD simulations. In the section labeled Prmtop, you'll want to click the Browse button. Navigate through your files until you get to the prmtop file of interest. The file name and pathway will be displayed in the light grey box between Prmtop and the Browse button. Next, you'll want to load in your trajectory. Click on the Add... button and navigate through your files until you find the NETCDF trajectory of interest (.nc file). Once you've selected your .nc file, you see the file name and pathway appear in the dark grey box. Your MD Movie window should look like the following example.



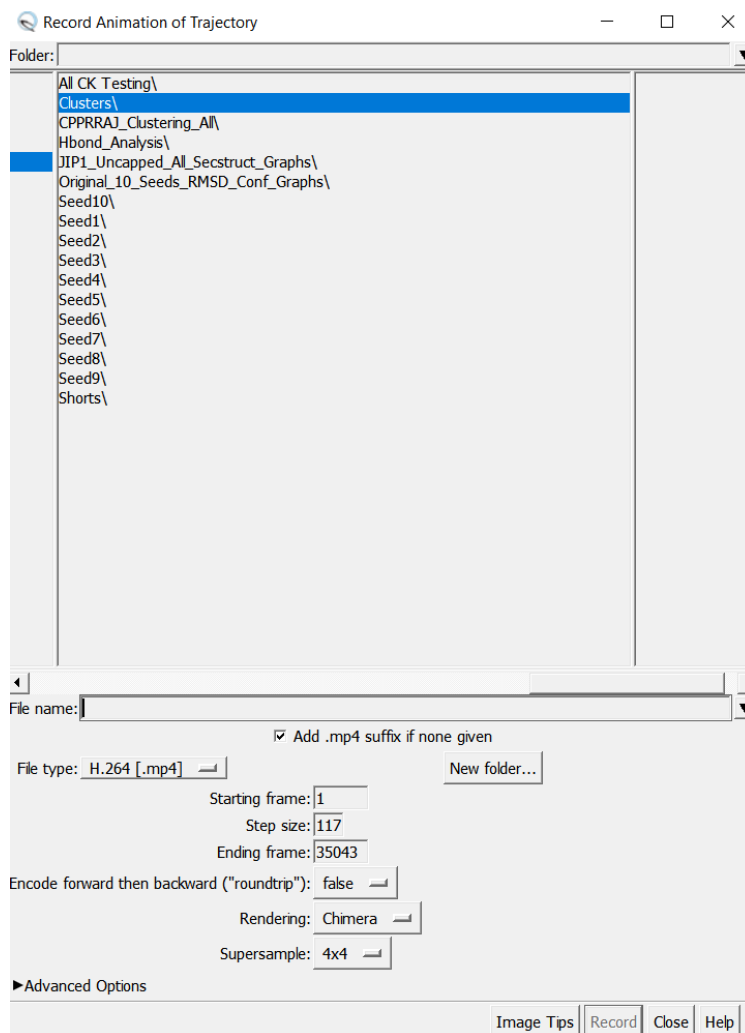
If everything looks good, click on the OK button. From here, you'll notice a brief pause as Chimera opens the trajectory. Once it finishes, you'll see the trajectory in the main Chimera window, and a new MD Movie window will pop up.



The main feature of this window is the frame slider. When we run an MD simulation, typically we set 1 frame as equal to 0.1 ns. So, a 100ns simulation will contain a total of 1000 frames. You can navigate through to different points in your simulation using the frame slider. You'll also see buttons that look like typical pause, play, and fast-forward buttons. The pause and play buttons work exactly as you think they do. The fast-forward looking button actually will increment the simulation a single time by the number of frames in the Step size option (by default 1 frame). The playback speed slider will change how fast the simulation movie plays when you click the play button, and the Loop option will loop the MD movie. This will be your main interface for viewing your MD simulation.

You'll note that there are top menu options. Most of these simply will perform certain analyses on your simulation. However, Amber has a built-in tool called *cpptraj* that does this, and as such we typically will just use that tool instead.

If you want to save the movie, click on File and then Record Movie... The following menu will pop up.



The top part of this window is simply your file directories. You can navigate through here to save the movie to whatever location you wish. By default, it will look to save movies in the same folder that contains the .nc file you loaded in. Give your file a name in the File name section. Next, for the File type option, make sure you select H.264 [.mp4] files. This simply refers to the way your movie file is encoded. This format is preferred because it offers compatibility across a wide range of operating systems.

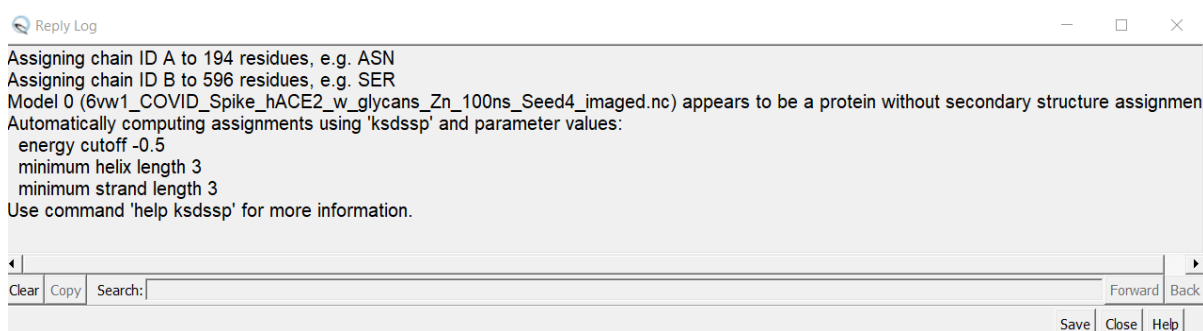
You'll see the Starting frame, Step size, and Ending frame boxes next. You can change the start and end frames to be whatever you want. By default, the start frame will be the first frame and the end frame will be the last frame of your simulation. The step size is set automatically to ensure that your movie will be about 10 seconds long at 25 frames per second.

The remaining options are related to the video quality. The Encode forward then backward option will be set to false by default. Setting it to true significantly increases video encoding time without producing a noticeable improvement in video quality, and as such I recommend keeping it set to false. The Rendering method option is set to Chimera by default, but the POV-Ray method is also available. POV-Ray is highly computationally expensive, and as such I would also recommend keeping this set to the default Chimera rendering method.

Lastly, we see the Supersample option. Supersampling is a graphical technique that renders a video out at a resolution higher than your screen can display. A typical full HD laptop screen resolution is 1920x1080 pixels. Setting the Supersample option to 4x4 as in the above example will render the video at a resolution of 7680x4320 pixels (also referred to as 8K resolution). This process reduces jagged lines (a common issue with all types of graphical rendering) and increases the smoothness of your final video. Depending on what you set this to however, it can be computationally expensive. The default is 3x3, and generally I would recommend keeping this setting. However, if your personal computer has a dedicated Nvidia or AMD GPU, you can set this to 4x4 like in my example.

You'll also note the Advanced Options dropdown menu. Don't worry about this unless your laptop contains a dedicated GPU. If you do have a dedicated GPU however, you can click on it and change the video quality. By default it is set to good quality, but a dedicated GPU will allow you to select the highest quality option. You'll also see the Image Format option. Chimera renders out a video by creating an image file of every frame. This just allows you to change that image file format. By default it is set to PPM, but you could change it to JPEG or PNG if you wanted to. This doesn't have a major impact on the video quality however, so I recommend just keeping it as the default format. Lastly, you'll see the Additional recording and Additional encoding options. As the names of these options imply, they allow you to enter additional recording and encoding commands.

The last tool we'll discuss is the **Reply Log**. This is nothing more than a text log of everything that has occurred in your current Chimera session. In most cases this can be ignored. However, any data from calculations you may run in Chimera will appear here (RMSD of models aligned using the MatchMaker tool, distances between atoms, etc.). Error messages will also appear in this log. If you open the reply log, it will look like the following example.



In this particular example, the reply log is simply giving us information on how the listed .nc file was loaded in and rendered using the MD Movie tool. Every tool you run will list information on how that tool was executed within the reply log.

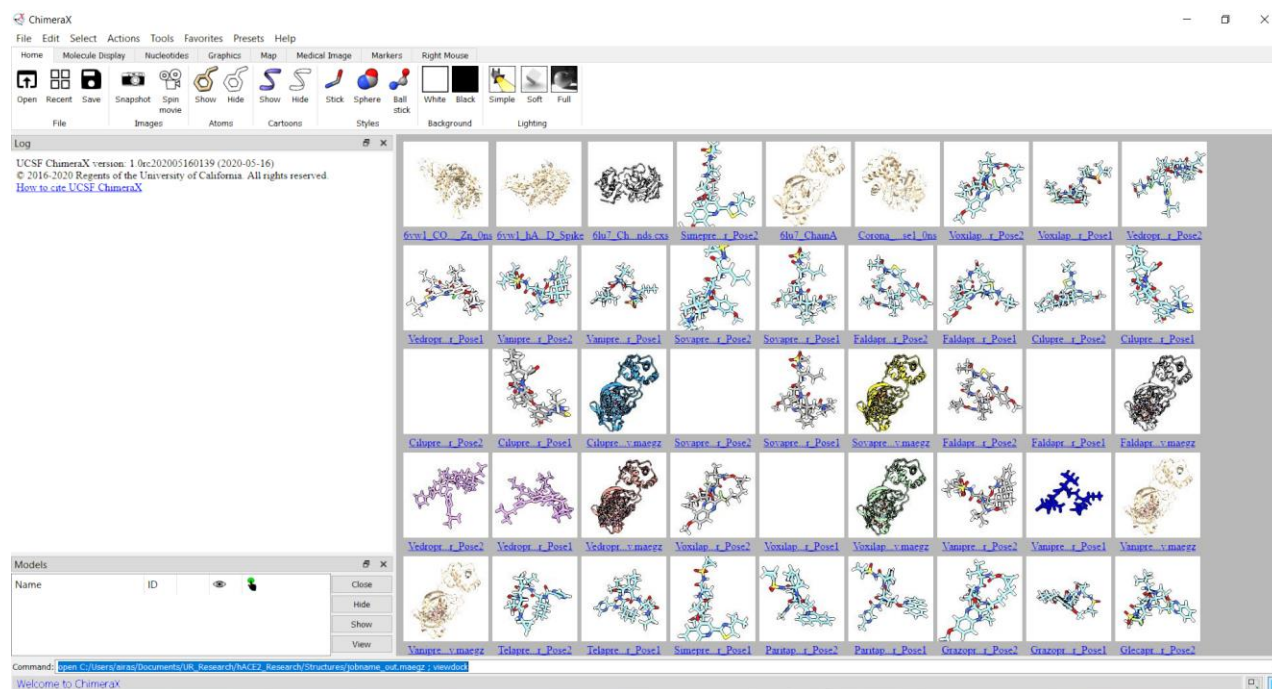
With that, you should have enough information to begin using UCSF Chimera. Additional detailed tutorials and command information can be found in the [UCSF Chimera User's Guide](#).

UCSF ChimeraX

A newer version of this program called UCSF ChimeraX also exists. The program has just recently completed the early development stage and is still missing some features. Unlike UCSF Chimera, it is dependent on commands rather than GUI tools. As such, it has a steeper learning curve. While it is lacking some features, it can render significantly higher-quality images than Chimera. You can download it for both Windows 10 and Mac OS X at the following link.

<http://www.cgl.ucsf.edu/chimerax/download.html>

Upon opening ChimeraX, you'll see a window like the following.



As ChimeraX is a much more modern program than Chimera, you'll see that the interface is significantly more intuitive in many ways. On the left-hand side the reply log and model list will be open by default. On the right-hand side you'll see images of recently opened files. You can click these to quickly reopen them. The command line is also open by default at the bottom of the screen, and it will save all of your most recently used commands.

While I won't provide a comprehensive walkthrough of how to use this program yet as its feature list is still volatile, the following list of commands should assist you in getting to know how to use ChimeraX.

One additional thing to note is that when saving files, you will need to specify the directory to save that file in. In these examples, I'm saving all of my files into the Windows 10 directory `C:\Users\airas\Documents\`. While the directory pathway format will be similar on all Windows 10 machines, the directory you save your files in will of course be different. Furthermore, a Mac OS X directory will be formatted differently. For example, my

documents folder on my Mac is in the directory `/Users/Justin/Documents`. You may note that this is similar to a Linux formatted directory.

Show / Hide Atoms/Bonds/Cartoons/Models/Surfaces

- `show #1 atoms`
 - Shows all atoms for model #1
- `hide #1 atoms`
 - Hides all atoms for model #1
- `show #1.2.1 models`
 - Shows model #1.2.1
- `hide #1 models`
 - Hides model #1.2.1

Command for saving a high-resolution image:

- `save "C:\\Users\\airas\\Documents\\image3.png" width 3840 height 2160 supersample 20`
 - This saves a 1080p image with supersample 20 in the above directory

Series of Commands for Creating a Rotation Movie:

- `movie record supersample 4 size 1920,1080`
 - This records the following actions in 1080p resolution with supersample 4
- `turn y 2 180`
 - This rotates the model on the y axis in two increments of 180 degrees
- `movie encode "C:\\Users\\airas\\Documents\\movie1.mp4" quality highest`
 - This saves the movie in the highest quality in the directory specified

Color and Selection Commands:

- `color byatom`
 - This colors all displayed atoms by the element according to the Jmol color scheme
- `color /A:156,128 orange`
 - This is a more advanced example command. It will color residues 156 and 128 in chain A orange. This can be done with other named colors, like lime, magenta, purple, blue, green, etc.
- `select /A:100`
 - This command is an example that specifically selects residue 100 in chain A
- `select #2`
 - This command selects only Molecule ID # 2
- `select @H1,H2`
 - This command selects only atoms named H1 and H2
- `select #1/B:100@CA`
 - This command selects molecule 1, chain B, residue 100 atom named CA
 - This is just many of the previous commands combined
- `del #1`
 - This command will delete all atoms in molecule 1

Distance Command:

- `distance #1@H1,H5`

- This command will measure the distance between atoms H1 and H5 in molecule 1

Series of Commands for Creating a Morph Movie:

- `morph #2,1 frames 180`
 - This instructs ChimeraX to calculate morphing coordinates of model 2 to model 1, using 180 intermediate structures
- `movie record supersample 4 size 3840,2160`
 - This records the following actions in 4K resolution with supersample 4
- `coordset #3`
 - This instructs ChimeraX to record the coordinates loaded in model 3
- `movie encode "C:\\Users\\airas\\Documents\\movie1.mp4" quality highest`
 - This saves the movie in the highest quality in the directory specified

Save PDB from a Multi-Structure File:

- `save "C:\\Users\\airas\\Documents\\Structure.pdb" format pdb models #1.2.2`
 - This command will save model #1.2.2 as a PDB in the directory indicated above

Open Maestro MAE Files:

- Click on tools -> More tools... -> select the input/output category on the left -> Install the Maestro tool
- `open "C:\\Users\\airas\\Documents\\SP_Dock_pv.maegz" ; viewdock`
 - This command opens your MAE file containing your receptor file and docked ligands

If you're looking for more detailed command information and tutorials, you can find them in the [UCSF ChimeraX User Guide](#).

Other Programs

As an alternative to Chimera or ChimeraX, one can download the visualization software known as VMD. This program performs all of the same functions as Chimera. However, I do not recommend it, as it is far less user-friendly and generally produces lower-quality images. However, if you want it, the download link for Windows 10 and Mac OS X is below.

<https://www.ks.uiuc.edu/Development/Download/download.cgi?PackageName=VMD>

An optional program you could download the text editor Vim. This is the text editor all of the local workstations use. It makes editing files like PDBs and scripts easier from your laptop. Unfortunately, it is not available for Mac OS X, but this isn't too big of a deal. Remotely logging in to the local workstations negates the need for this on both Windows 10 and Mac OS X. The download link for Windows 10 is below. Within this link, click on the top link for a modern version of Vim, and be sure to install the 64-bit version.

<https://www.vim.org/download.php#pc>

Another optional program that I find very useful is called Avogadro. This program uses a GUI to allow you to easily build small molecules and optimize their geometry using the GAFF force field. This is nice for projects that deal with ligand construction. You can find the download link for Windows 10 and Mac OS X here.

<https://avogadro.cc/>

Lastly, the program OpenBabel has niche uses when converting between file formats. For example, if an advanced ligand requires you to run Gaussian calculations for partial charges, you can convert the file format from a PDB and/or mol2 to a Gaussian COM file using this program. While this program is available for Mac OS X, it is significantly more difficult to install. The download link is as follows.

<http://openbabel.org/wiki/Category:Installation>

Preparing an Amber job

Amber MD simulations require a minimum of three input files for the simulation: a topology file, a coordinate file, and an MD input file. In this section, we will discuss how you prepare the initial topology file and coordinate file. Topology and coordinate file preparation can be done using one of two available programs in Amber: *xleap* or *tLeap*. They both have nearly identical functionality, but the difference is that *xleap* uses a GUI (graphical user interface) while *tLeap* can be run directly from your Terminal command line. In this protocol, we will use *tLeap* because it is much more stable.

Amber Equation, Force Field Parameters & Partial Charges

MD simulations are governed by the Amber equation. This equation calculates the potential energy on pairs of atoms according to the following equation.

$$E_{pair} = \sum_{bonds} K_r (r - r_{eq})^2 + \sum_{angles} K_\theta (\theta - \theta_{eq})^2 + \sum_{dihedrals} \frac{V_n}{2} [1 + \cos(n\phi - \gamma)] + \sum_{i < j} \left[\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6} + \frac{q_i q_j}{\epsilon R_{ij}} \right]$$

The first term in this equation is the sum of all bond stretching potentials. This is modeled from the spring equation. K_r is the harmonic force constant, which is an interpretation of the ease or difficulty of stretching or compressing a particular bond from its equilibrium bond distance. r is the bond length at a particular frame and r_{eq} is the equilibrium bond length. The deviation of a bond distance from its equilibrium length determines the stretching potential energy of a bond.

The second term in this equation is the sum of the angle bending potentials in the system. This is also modeled after the spring equation and takes a nearly identical form to the first term. K_θ is the harmonic force constant representing the difficulty of bending an angle from its equilibrium angle. θ is the angle at a particular frame and θ_{eq} is the equilibrium angle.

The third term in the equation is the sum of the torsional twisting potentials in the system. V_n is the torsional force constant, n is the multiplicity, and γ is the phase shift angle. A single bond may have multiple torsional terms.

The last term describes noncovalent van der Waals and electrostatic potentials of the system. The values A and B in $\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6}$ are van der Waals parameters, whereas R_{ij} is the distance between the two interacting atoms. The values q_i and q_j are the partial charges of the two interacting atoms.

Apart from partial charges, all of the parameter values in the Amber equation are supplied by a force field. Many sets of force fields exist, but Amber commonly uses the force fields ff14SB and GAFF. Partial charges for common biomolecules are prebuilt into Amber and calculated by way of relative electrostatic potential fitting (RESP). Charges for organic molecules need to be calculated in Amber. This is usually done using the AM1-BCC or RESP method.

Basic ligand preparation

The Amber force fields already contain all the information (charges, bond distances, angles, torsions, etc) for typical biomolecular residues (e.g. amino acids, nucleic acids, lipids). However, if your system has an organic ligand (such as a drug molecule or potential inhibitor), then you will need to build the parameters for that molecule using Amber's *antechamber* and *parmchk* programs.

You will first need to make a PDB file of *only* your ligand molecule. So if the coordinates for your ligand are in PDB format as part of a single PDB that includes your biomolecule, also, then copy the PDB to another file and remove all coordinates for the biomolecule, leaving only the ligand coordinates. Once that is completed, if your ligand PDB file does not contain hydrogen atoms for the ligand, you will need to add those. You can do this with a program in Amber called *reduce*.

```
$AMBERHOME/bin/reduce ligand_no_H.pdb > ligand_with_H.pdb
```

Once you run this command, always visually check to make sure that *reduce* added hydrogen atoms correctly. This can be done just by opening *ligand_with_H.pdb* in Chimera. If it was not correctly protonated, you'll need to edit the PDB manually. This can be done fairly easily using the Build function in Schrödinger's Maestro program, or alternatively using Avogadro.

Once you've made sure your ligand was correctly protonated, you are ready to let *antechamber* calculate all the charges for every atom in your ligand PDF file (*ligand_with_H.pdb*). This will take a little bit of time since Amber must run a quantum calculation.

```
$AMBERHOME/bin/antechamber -i ligand_with_H.pdb -fi pdb -o ligand_with_H.mol2 -fo mol2 -c bcc -s 2 -nc 0 -m 1
```

antechamber will run a single-point quantum calculations (using Amber's *sqm* quantum program) to determine the charge of each atom of the system with the given coordinates and output the results into a mol2 structure file (*ligand_with_H.mol2* in the example above). In the

command, the ‘-c bcc’ denotes that we want to use AM1-BCC charges for our ligand (there are other charge types available through *antechamber* (see \$AMBERHOME/bin/antechamber -L). The ‘-s 2’ tells *antechamber* to be verbose, so any and all information is printed to the screen during the calculation. The last two flags in the command are options you might need to change. The total overall charge of your ligand is specified with the ‘-nc’ flag. In the example, the ligand has a neutral charge, but if your ligand has a +2 overall charge, then you should have ‘-nc 2’, or if your ligand has a -1 charge then put ‘-nc -1’ instead. So you need to know the overall charge of your ligand beforehand, and you should be able to determine this from a basic chemical perspective (ask for help if you are unsure). But if you guess the incorrect total charge, *antechamber* is likely to give you an error message saying that the charge is wrong. The last option ‘-m’ specifies the multiplicity of the ligand determined by $2S+1$ where S is the total number of unpaired electrons in the system. This is usually 1 (default) in the case of no unpaired electrons. Running *antechamber* will create quite a few files in your directory, most of which are unhelpful. I typically remove all the output files that start with ANTECHAMBER_ and the ATOMTYPE.INF files just to help keep my directory less cluttered.

Now that we have the charges in our mol2 file, we will use *parmchk2* to generate a force field modification (frcmod) file for our ligand.

```
$AMBERHOME/bin/parmchk2 -i ligand_with_H.mol2 -f mol2 -o ligand_with_H.frcmod
```

This command creates the `ligand_with_H.frcmod` file that contains different parameters specifically for your ligand based on the Generalized Amber Force Field (GAFF) that is parameterized for typical organic molecules. An frcmod file looks like the following example.

```
Remark line goes here
MASS
Si 28.085

BOND
Si-oh  339.5    1.624      gaff c3-oh values from Bowers
Si-os   339.5    1.624      gaff c3-os values from Bowers
Si-h3   209.9    1.482      gaff c3-h3 values from Shorb

ANGLE
os-Si-os   70.3      110.70    gaff Si -> c3 values from Bowers
Si-os-Si    8.5      159.00    gaff Si -> c3 values from Bowers
os-Si-h3   75.1      110.32    gaff Si -> c3 values from Shorb

DIHE
ca-ca-Si-c3  6      0.000      0.000      2.000      gaff X -c3-ca-X
ca-Si-c3-c3  9      1.400      0.000      3.000      gaff X -c3-c3-X
ca-Si-oh-ho  3      0.500      0.000      3.000      gaff X -c3-oh-X
c3-c3-Si-c3  1      0.200     180.000     1.000      gaff c3-c3-c3-c3
c3-c3-Si-oh  9      1.400      0.000      3.000      gaff X -c3-c3-X
c3-Si-c3-hc  1      0.160      0.000      3.000      gaff Si -> c3
c3-Si-oh-ho  1      0.160      0.000     -3.000      gaff ho-oh-c3-c3
c3-Si-oh-ho  1      0.250      0.000      1.000      gaff ho-oh-c3-c3
c3-Si-oh-ho  1      0.250      0.000      1.000      gaff Si -> c3
oh-Si-c3-hc  1      0.250      0.000      1.000      gaff Si -> c3

IMPROPER
```

Si-ca-ca-ca	1.1	180.0	2.0	Using default value
NONBON				
Si	2.220	0.320		DOCK vdw_AMBER_parm99.defn

The first line is simply a comment line. If you want, you can write a brief description of the `frmod` file here. The following sections detail the parameters for each atom type. The `MASS` section details the mass in AMU of an atom type. This is equivalent to the average mass of an element reported in the periodic table. The `BOND` section details the bond force constant (in $\text{kcal} \cdot \text{mol}^{-1} / \text{\AA}^2$) and equilibrium bond distance ($\text{\AA}$) of the bond between the two listed atoms. The `ANGLE` section details the angle force constant (in $\text{kcal} \cdot \text{mol}^{-1} / \text{rad}^2$) in and the equilibrium angle in degrees of the angle formed by the three listed atoms.

The `DIHE` section details the dihedral / torsional angle of the four atoms listed. The next four values fill the torsional potential equation $\frac{PK}{IDIVF} \cdot (1 + \cos(PN\phi - PHASE))$. The first value `IDIVF` is simply a division factor of the torsional barrier. The second value `PK` is the torsional barrier height divided by 2. The third value `PHASE` is the phase shift angle in degrees of the torsion. The last value `PN` is the periodicity of the torsional value. The `IMPROPER` section details improper dihedral / torsional angles using the same values. The only difference is that the division factor `IDIVF` is not used in the torsional potential equation $PK \cdot (1 + \cos(PN\phi - PHASE))$ for improper torsions.

Lastly, the `NONBON` section details the van der Waals interactions of the listed atom. The first term is the A term and the second is the B term in the equation $\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6}$.

We have now created the files necessary for *tleap* to assign parameters to our small molecule ligand.

Advanced ligand preparation

The previous steps work well when you have a typical organic molecule. Unfortunately, not all ligands are normal organic molecules. This includes molecules containing negatively-charged and hypervalent phosphorus (i.e. phosphates) or sulfur (i.e. sulfates). Other more complicated ligands exist, such as molecules with positive charges, other hypervalent groups (such as sulfonamides), and ligands covalently bound to amino acids. Alternatively, you also may have the desire to obtain better charges for your ligand than you would otherwise get with Amber's *antechamber* program. These ligands must be prepared differently.

Charged and Hypervalent Ligands

Advanced ligand preparation for charged and hypervalent ligands requires more advanced partial charge calculation. You will need to create a Gaussian input file for this task.

Gaussian will calculate the electrostatic potential (ESP) of your molecule. The program *antechamber* will then use this to calculate relative electrostatic potential (RESP) charges for each atom in your ligand. From here, a `PREPi` file will be generated using typical AMBER BCC charges. This can be done using the program *Gabedit* or through a series of commands.

Using Gabedit

In the past, Gabedit has been used to build ligands and set up Gaussian input files. While it is not the current preferred method, it is still valid. Furthermore, the Gaussian commands generated using Gabedit are also used in the method employing a series of commands. As such, this method is detailed here.

Gabedit does not come natively on the lab computers. You can download the source code for the program and compile it yourself using the instructions on the website: <http://gabedit.sourceforge.net/>. Alternatively, you should just be able to `rsync` the compiled source code from Pople. `cd` into a directory where you would like the Gabedit source code to be stored on your computer. Then, type

```
rsync -azvp --progress username@pople.richmond.edu:/home/bmiller4/Gabedit64 .
```

This places the Gabedit program on your computer. If you type

```
cd Gabedit64
```

and then

```
./gabedit
```

You should see the GUI for the Gabedit program open on your monitor. Then click

Geometry → Draw

Now a new Draw Geometry window should appear. Right click in the middle of the window, and click on

Read → PDB file

Browse to the directory with your PDB file and open it. You should now see your residue/ligand in the Draw Geometry window. Now in the main Gabedit window, click on

File → New → Gaussian input.

You should input the charge and multiplicity of your system at the top. The Amber force fields are parameterized using the same Quantum level of theory and basis sets, so for consistency, we will use the same. So under METHOD choose Hartree-Fock and the 6-31G* Basis set with No Diffuse function and an * for the Polarization function. Click on OK at the bottom of the window and a Gaussian-style file format should appear in the Gabedit main window. Gabedit will not automatically create a Gaussian input in the style that we want for Amber purposes, so you should take the example Gaussian input file (called `guassian.com`) shown below and replace the charge, multiplicity, and coordinate lines with your structure.

```
%NProcShared=8
#P HF/6-31G*
# Gfinput IOP(6/7=3) iop(6/33=2) iop(6/42=6) iop(6/50=1) Pop=full
Pop=SaveESP Pop=MK Density Test SCF=QC
# Units (Ang, Deg)
```

Input file generated by gabedit...

-2	2			
P	0	48.725	39.671	56.771
O	0	49.398	39.455	58.074
O	0	47.565	38.848	56.400
O	0	48.274	41.194	56.691
C	0	49.155	42.262	57.100
H	0	49.954	42.379	56.368
H	0	49.585	42.027	58.073
C	0	48.380	43.556	57.200
H	0	49.038	44.375	57.489
O	0	47.901	43.923	55.886
C	0	46.602	44.500	55.973
H	0	46.535	45.509	55.565
N	0	45.706	43.733	55.101
C	0	45.876	42.453	54.627
H	0	46.716	41.859	54.954
N	0	44.913	42.053	53.831
C	0	44.045	43.136	53.783
C	0	42.829	43.353	53.091
N	0	42.265	42.452	52.273
H	0	41.395	42.668	51.807
H	0	42.710	41.558	52.122
N	0	42.212	44.544	53.265
C	0	42.789	45.453	54.066
H	0	42.316	46.412	54.218
N	0	43.929	45.370	54.759
C	0	44.514	44.173	54.571
C	0	47.128	43.471	58.065
H	0	46.678	42.484	57.958
C	0	46.214	44.515	57.444
H	0	45.315	44.620	58.051
H	0	46.734	45.472	57.397
O	0	47.283	43.804	59.437

DA3H.esp

Leave the top 5 lines of text unchanged, and simply replace all the atomic information with the results of your structure. You can change the number of processors to be used for the calculation if you want, by changing the first line %NProcShared=8 to some number other than eight. You

should also change the name of the ESP output file at the very bottom of the file. This example is for a residue/ligand called DA3H. You should modify the `DA3H.esp` file so the residue name for your ligand is there instead of DA3H.

To run Gaussian, simply type

```
g16 gaussian.com
```

The time required for this calculation depends on a lot of variables (how many processors you use, how big your system is, how fast your computer is, etc). The example shown took about 14 minutes on the Pople workstation.

Now that Gaussian has calculated all of the atomic charges for your residue/ligand, you are ready to create a Mol2 file that includes the coordinates and charges. To make this Mol2 file, run the following command

```
$AMBERHOME/bin/antechamber -i DAH3.esp -fi gesp -o DAH3.mol2 -fo mol2 -c resp -rn DAH3
```

And you should replace every instance of `DAH3` with whatever you want your residue name to be (make sure it is in all caps and either 3 or 4 characters in length. The `-i` flag is followed by the input file name (which, in our case, in our Gaussian esp output file). The `-fi` flag is followed by the term `gesp` to let antechamber know the input file is a Gaussian esp output file. The `-o` flag is the name of the mol2 we want antechamber to create. The `-fo` flag is followed by `mol2` because we want antechamber to create a mol2 file. The `-c` flag is followed by `resp` because we are giving antechamber RESP atomic charges (that is the type of charges we had Gaussian compute). Finally, the `-rn` flag is followed by the name of the residue name we want Amber to recognize for our residue.

From here, to complete the advanced ligand preparation process, the following commands will need to be run using *antechamber*. Skip [here](#) to continue.

Using a script

While the method using Gabedit is perfectly valid, it requires one to first install Gabedit on their workstations (theoretically an easy process, but it is prone to issues) and then manually edit a lot of files. As such, the entire process detailed previously can be done systematically using a series of commands rather than with the Gabedit GUI. This is fairly complicated, but it significantly shortens the amount of time you'll spend typing later on. As such, I highly recommend this method.

As stated previously, you'll need a PDB file of your ligand before using this method. Always be sure that your ligand file is correct (i.e. correct number of hydrogens, correct bond orders, etc.) before beginning this process.

First things first, you'll need to convert your ligand PDB file to a Gaussian-styled COM file. This can be done fairly easily using a program called OpenBabel. First, we'll need to load the module file for OpenBabel. This can be done using the following command.

```
module load openbabel/2.3.2
```

Now that we've loaded the program, we'll need to convert the file. This can be done using the following command.

```
babel -ipdb ligand_w_H.pdb -ocom temp1
```

The command `babel` invokes OpenBabel. The flag `-ipdb` indicates that we are inputting a PDB file (in this case `ligand_w_H.pdb`). The flag `-ocom` indicates that we're outputting a COM file (in this case, a temporary file named `temp1`).

From here, we've created a COM file containing the coordinates of all the atoms in the ligand. The file will look like the following example.

```
#Put Keywords Here, check Charge and Multiplicity.

ligand_w_H.pdb

0  40141216
C      -13.56400      -9.69700      14.10000
C      -12.91100     -10.81100      13.13900
C      -13.93600     -11.57200      12.82100
C      -13.08400      -8.21800      14.22000
...
```

While the XYZ coordinates in this file will be correct, the charge and multiplicity will be ridiculous (in this case, the charge is 0 and the multiplicity is 40141216). Furthermore, as is likely obvious from the first line, this file is missing the necessary Gaussian keywords/commands needed to run a Gaussian calculation. As such, this file will need to be edited. This will be done using a series of commands.

First, we'll need to delete the first 5 lines of the COM file from OpenBabel. This can be done using the following command.

```
sed -i '1,5d' temp1
```

Next, we'll want to create a second temporary file (named `temp2`) containing the Gaussian commands we need to run our ESP calculation. These are the same commands as those generated by Gabedit. The file we create should look something like this.

```
%NProcShared=12
# P HF/6-31G*
# Gfinput IOP(6/7=3) iop(6/33=2) iop(6/42=6) iop(6/50=1) Pop=full Pop=SaveESP
Pop=MK
Density Test SCF=QC
# Units(Ang,Deg)
```

```
Input file generated by gabedit...
```

```
0 1
ligand.esp
```

Note that we can change the number of processors we run the calculation on. In this example, all 12 processors on Thais were used (`%NProcShared=12`). On a workstation like Cooper, we can run this command using a maximum of 20 processors. Generally, the more processors we run on, the faster the calculation will go.

The charge and multiplicity will also need to be changed to match your ligand. In this example, our ligand is neutrally charged. As such, the net charge is 0. It also has no unpaired electrons. This means our multiplicity (# unpaired electrons +1) is 1.

Everything else from this example will remain unchanged. As mentioned in the Gabedit section, we're simply using the Hartree-Fock method with the polarization-corrected 6-31G* basis set to calculate the electrostatic potential (ESP) of our molecule.

From here, we'll need to insert our ligand atomic coordinates into our file containing our Gaussian commands, charge, and multiplicity. This can be done using the text-editor command `ed`. First, we'll type the following line.

```
ed -s temp2 <<IN
```

This will edit the file containing our Gaussian commands, charge, and multiplicity (`temp2`) according to all the commands that follow until the flag `IN` is typed. From here, a question mark will appear in the terminal. After this question mark, we'll enter the following command.

```
/ligand.esp/-r temp1
```

This line will insert the ligand atomic coordinates (in the file named `temp1`) into the file containing our Gaussian commands, charge, and multiplicity (`temp2`) before the line that names our output file (`ligand.esp`). The process isn't done yet however. You'll note that another question mark will appear after we enter this command. We'll need to type a `w` and hit enter. This will simply write the lines in `temp1` into `temp2`. Next, we'll type `q`. This quits out of the command `ed`. Lastly, we'll type in `IN` to end the string of commands.

From here, we'll delete the file `temp1` as it is no longer necessary, and then rename the edited `temp2` file to `ligand_w_H.com`. This can be done using easily using the `rm` command to remove a file and the `mv` command to rename a file. Our fully edited file should look like the following example.

```
%NProcShared=12
# P HF/6-31G*
# Gfinput IOP(6/7=3) iop(6/33=2) iop(6/42=6) iop(6/50=1) Pop=full Pop=SaveESP
Pop=MK
Density Test SCF=QC
# Units (Ang, Deg)

Input file generated by gabedit...
```

```

0 1
C      -13.56400      -9.69700      14.10000
C      -12.91100      -10.81100      13.13900
C      -13.93600      -11.57200      12.82100
C      -13.08400      -8.21800      14.22000
...
ligand.esp

```

At this point, the next step will be to run the Gaussian calculation. This is a fairly simple process. First, load the Gaussian module file using the command that follows.

```
module load gaussian/16
```

Next, we can run the calculation using the following command.

```
g16 ligand_w_H.com
```

This process will take a bit of time. On Thais using 12 CPUs, a ligand with 97 atoms and 396 electrons, this calculation took about 9 minutes. Once the job has finished, it will output two files. The first will be named `ligand.esp`, and the second will be named `ligand_w_H.log`. We're interested in the ESP file.

From here, we'll use Amber's *antechamber* program to convert our ESP to RESP charges. First, we'll need to load in the Amber program module using the following command.

```
module load amber/18
```

Next, we'll enter the following command.

```
$AMBERHOME/bin/antechamber -i ligand.esp -fi gesp -o ligand.mol2 -fo mol2 -c
resp -rn LIG
```

This simply inputs the ligand ESP file (`-i ligand.esp`) in the form of a Gaussian ESP file (`-fi gesp`). It will then output a Mol2 file in the format of a Mol2 file (`-o ligand.mol2 -fo mol2`) containing a residue named `LIG` with RESP partial charges (`-c resp -rn LIG`). This calculation will not take too long.

From here, another command will need to be run. Due to the fact that Gaussian outputs both new atomic coordinates and atom names, it is not possible to convert a docked ligand pose PDB file into a Mol2 file using this method. Instead, a PREPi file will need to be created to define the bonds and charges of a docked ligand PDB file. This will significantly cut back on time in the long run as this PREPi file is applicable to any docking pose for this ligand. Type in the following command.

```
$AMBERHOME/bin/antechamber -fi gesp -fo prepri -i ligand.esp -o ligand.prepi -
c resp -j 4 -at gaff
```

This command will input your Mol2 file and output a PREPi file (`-fi gesp -fo prepi -i ligand.esp -o ligand.prepi`) with RESP partial charges, full atom and bond type designation, and atom typing according to the General Amber Force Field named GAFF (`-c resp -j 4 -at gaff`).

From here, there's another step. When generating the PREPi file, Amber will output a new PDB, creatively named `NEWPDB.PDB`. As stated previously, Gaussian changes the atomic coordinates and atom names from the original PDB. It is important that any subsequent ligand docking is conducted using a PDB with atom names matching that of the PREPi file. This file `NEWPDB.PDB` will contain the new atom names. This will be the PDB we'll want to use for ligand docking. As such, we'll rename this PDB to something more descriptive. That can be done using the following command.

```
mv NEWPDB.PDB ligand_NEW.pdb
```

At this point, if your ligand is an organic molecule, you should be able to run Amber's *parmchk2* program using the following command. Note that while *parmchk2* will use the Mol2 file generated from this process, it is not needed at any other point.

```
$AMBERHOME/bin/parmchk2 -i ligand.mol2 -f mol2 -o ligand.frcmod
```

Like the PREPi file, the resultant frcmod file is applicable to all ligand docking poses. From here, you can move on to ligand docking, and eventually the section Creating initial Amber topology and coordinate files. Furthermore, [this section](#) details how to incorporate PREPi files into *tleap*. Note that all the basic ligand preparation steps incorporated into the late script `ligprep.sh` are not necessary if using advanced ligand preparation.

Thankfully, this entire process can be streamlined using a script. The only additional command used in this script is the `cat` command with an EOF flag to write out the `temp2` file. A template for such a script (`Adv_ligprep.sh`) is detailed below.

```
#!/bin/csh
module load openbabel/2.3.2
module load gaussian/16
module load amber/18

# Convert the PDB file to a temporary Gaussian COM file
babel -ipdb ligand_w_H.pdb -ocom temp1

# Edit the temporary COM file -- takes many steps to complete
#
# Delete the 5 lines auto-generated by OpenBabel
sed -i '1,5d' temp1

# Create another temporary file containing the necessary Gaussian commands
cat > temp2 << EOF
%NProcShared=12
# P HF/6-31G*
```

```

# Gfinput IOP(6/7=3) iop(6/33=2) iop(6/42=6) iop(6/50=1) Pop=full Pop=SaveESP
Pop=MK
Density Test SCF=QC
# Units(Ang,Deg)

Input file generated by gabedit...

0 1
ligand.esp
EOF

# Insert the ligand atomic coordinates into the file containing the Gaussian
commands
ed -s temp2 <<IN
/ligand.esp/-r templ
w
q
IN

# Remove and rename the temporary files
rm temp1
mv temp2 ligand_w_H.com

# Run the Gaussian Input file to calculate the molecule's electrostatic
potential (ESP)
g16 ligand_w_H.com

# Using the ESP, create a Mol2 file with RESP charges
$AMBERHOME/bin/antechamber -i ligand.esp -fi gesp -o ligand.mol2 -fo mol2 -c
resp -rn LIG

# Using the ESP file, create a pose-independent PREPi file
$AMBERHOME/bin/antechamber -fi gesp -fo prepi -i ligand.esp -o ligand.prepi -
c resp -j 4 -at gaff

# Rename the PDB generated by Antechamber to that of your ligand -- IMPORTANT
-- Use this new PDB for all subsequent ligand docking, using the original
file will cause the docking poses to have atom names that do not match that
of the PREPi file
mv NEWPDB.PDB ligand_NEW.pdb

# Create a force field modification file (frcmod) using parmchk2
$AMBERHOME/bin/parmchk2 -i ligand.mol2 -f mol2 -o ligand.frcmod

# From here, you're ready for ligand docking. This is just a congratulatory
line (congratulations!!)
echo "ligand_NEW.pdb is ready for ligand docking!"

```

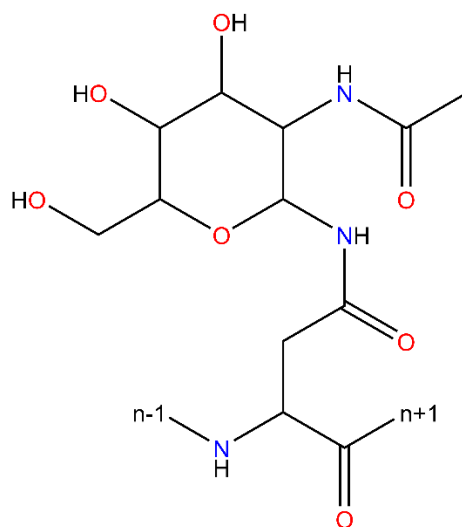
From here, your journey into advanced ligand preparation is complete! This script will make any advanced ligand preparation done in the future significantly more intuitive.

Modified Residues and Covalent Ligands

NOTE: Phosphorylated amino acids do not require these steps. Simply use the phosaa10 force field and follow the instructions [here](#) instead.

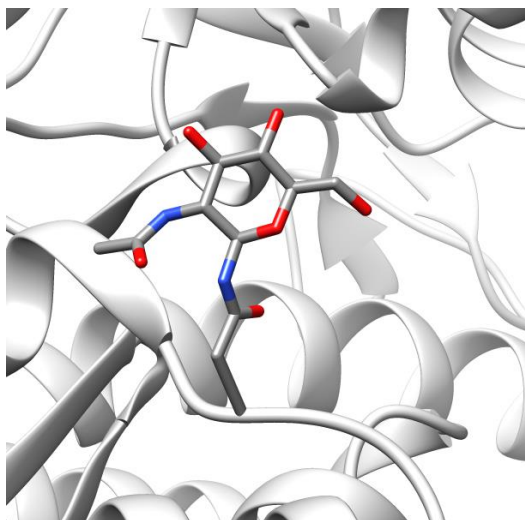
Certain proteins will contain amino acids that have been modified in some way. Some common amino acid modifications include phosphorylation and glycosylation. There are also certain ligands that will form covalent bonds to certain amino acids in a protein. An example of this is the experimental SARS-CoV-2 main protease inhibitor N3, which upon entering the active site forms a covalent bond to the catalytic cysteine residue.

If you are creating a modified residue of a protein or nucleic acid, then you have more work cut out for you before you can move on. This process is far more involved and will require some perseverance and creativity. While this walkthrough will by no means be a complete guide, it should at least provide a framework for you to work with. In addition to the advanced ligand preparation described above, you'll need to develop parameter files describing your modified amino / nucleic acid. For this walkthrough we'll focus on a specific example of a modified amino acid, but a similar process should be applicable for other modified residues and nucleic acids as well. This specific example we'll focus on is an asparagine residue (Asn) bound to the glycan N-acetyl-D-Glucosamine (NAG). The structure of this modified amino acid is detailed below.



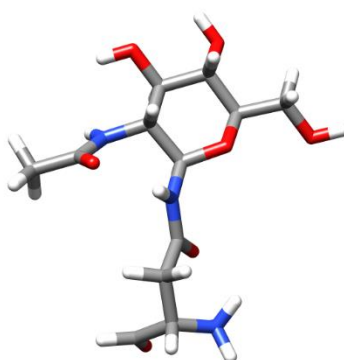
Preparing an Input Structure for RESP Fitting

First and foremost, you will need to identify and isolate your modified amino acid. Open your PDB containing the modified amino acid in Chimera and search for it.

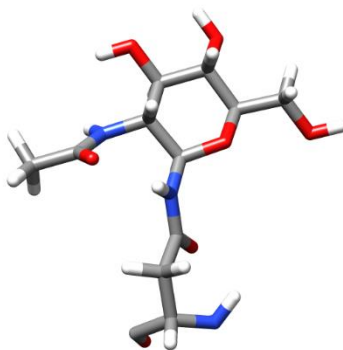


Here we see Asn bound to NAG. From here, you would need to select both Asn and NAG, invert your selection, and delete all selected atoms and bonds. This will leave you with only Asn and NAG remaining in your structure. Save this as a PDB, and then we can move to the next step.

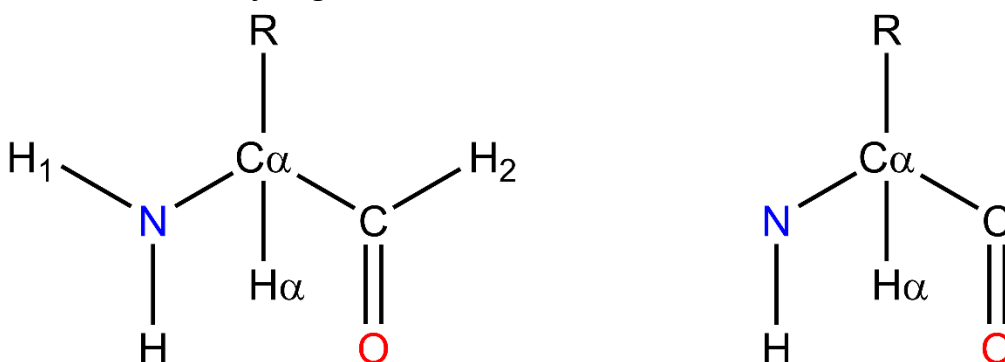
Now that we've isolated our Asn-NAG structure, we'll need to make some slight modifications to it. Most raw PDB files do not contain hydrogen atoms, and as such we'll need to add them ourselves. Open your new PDB in Chimera. From here, in the top menu click on Tools, hover over Structure Editing, and click on AddH. This will bring up a window for the add hydrogens tool. We'll keep all settings at their default, so click okay without changing anything in this window. This should add hydrogens to the Asn-NAG complex. Inspect your structure to ensure that hydrogens have been added correctly (i.e. check for too many or too few hydrogens). Your structure should now look something like this.



There's just one more modification we'll need to make from here. When isolated, the amine nitrogen and carboxylic acid carbonyl carbon of your amino acid backbone will each have a hydrogen atom that would normally not be there if this residue was a part of a protein. This is because these atoms would normally be bound to the backbone of the n-1 and n+1 amino acids. To fix this, we'll just delete these unnecessary hydrogens, and your final structure should look like the following.



Just in case the above images aren't clear enough, the following image provides a 2D structural image of the what the amino acid backbone should look like before (left) and after (right) deleting the two additional hydrogens.



It should be noted that this will result in improper valences on the amine nitrogen and the carbonyl carbon. However, as will be explained shortly, this will not be a problem for our final parameters. Save this structure as a new PDB, and now we can move on to the next step.

Now that we have our final protonated Asn-NAG structure, we'll need to run it through the advanced ligand preparation script (`Adv_ligprep.sh`) described [here](#). Note that despite incorrect valences, we will still be setting the net charge of this structure to 0. This is to ensure that the modification to our base amino acid (NAG in this case) will have correct charges. Once this process finishes, you'll get several new files that we'll be interested in. Following the file names used in the template script, we'll be interested in `ligand_NEW.pdb` and `ligand.prepi`.

First, open the file `ligand_NEW.pdb` in Chimera and make sure there is nothing glaringly wrong with the structure. This new PDB is produced by *antechamber* when creating your PREPi file. It contains updated atom names that correspond to the atom names present in the PREPi file, and as such should be referenced when editing the PREPi. Once opened in Chimera, the structure of `ligand_NEW.pdb` should look similar to the PDB you entered into the `Adv_ligprep.sh` script.

PREP and PREPi File Format

Before we begin editing our PREPi file, we'll need to talk about the PREP and PREPi file format. You'll need to have an understanding of these file formats to properly incorporate any modified amino acid parameters. Amber uses PREP files to build the structures of all standard

amino acids present in proteins. A typical amino acid PREP file will be formatted according to the following outline.

```

0      0      2

This is a remark line
molecule.res
MOL    INT    0
CORRECT      OMIT DU    BEG
0.0000
1  DUMM
2  DUMM
3  DUMM
4  N
5  H
6  CA
7  HA
8  Sidechain starts here...
...
X  Sidechain ends here...
Y  C
Z  O

Loop

Improper

DONE
STOP

```

You'll note the first line contains three numbers. The first 0 indicates that this file does not come from a pre-existing database. The other two numbers further detail the database (0 meaning it will save this to a new database) and the force field type used (2 indicating an all atom model). However, since the first number 0 indicates that there is no associated database, these last two numbers are ignored. The second line indicates which pre-existing database this residue is from. You'll note that this line is blank. This is of course because this residue does not come from a pre-existing database.

The third line is nothing more than a descriptive title line, and as such you can write in any description you want here. The fourth line is the name that would be given to an output file containing only this individual residue. However, since this PREP file will be used to describe a modified residue that is incorporated into a PDB containing multiple residues, this line will not be used.

Next, we have the fifth line. The first term `MOL` is simply the 3-letter amino acid code that will be read from the PDB to indicate that this PREP file should be used to parameterize that residue. The second term `INT` means that internal coordinates will be used rather than cartesian coordinates. This is to say that rather than give XYZ coordinates of atoms in 3D space, the position of each atom relative to other atoms in the residue is detailed instead. This allows the PREP file to parameterize any residue with the 3-letter code `MOL` regardless of its cartesian location in the PDB. Lastly, the 0 indicates that the *tleap* output will not be in binary.

The second to last line before we get to the atom description section provides more information on how to build the residue `MOL`. The term `CORRECT` tells *tleap* to read internal coordinates and build each atom according to its indicated tree structure (more on this soon). The term `OMIT` tells *tleap* to delete dummy atoms. Dummy atoms only function as internal coordinate references to set up the first few atoms in our residue, and in general should be deleted after this use has been exhausted. The next term `DU` tells *tleap* which atom type corresponds to a dummy atom. The last term `BEG` tells *tleap* to only delete dummy atoms present before the first atom in our residue. Most PREP files we deal with will only have dummy atoms before the first atom.

The final line before the atom description provides a distance cutoff value in angstroms (Å) for loop closing or ring structure forming atoms. This is usually set to 0.0000 and instead loop closing / ring forming atoms will be explicitly described in a later section of the file. This ensures no incorrect rings are formed.

Now we move into the atom description section. We'll first discuss the order atoms are listed in in a standard amino acid PREP file. Atoms 1, 2, and 3 will all be dummy atoms. Atom 4 will always be the amino acid backbone nitrogen, and 5 will be the hydrogen bound to it. Atom 6 will be the backbone alpha carbon, and 7 will be the hydrogen bound to it. Atom 8 begins the amino acid sidechain. Once all sidechain atoms are described, at the end of the atom description we'll find the backbone carbonyl carbon followed by the backbone carbonyl oxygen. This format is necessary for any standard or modified amino acid to be parameterized correctly by *tleap*.

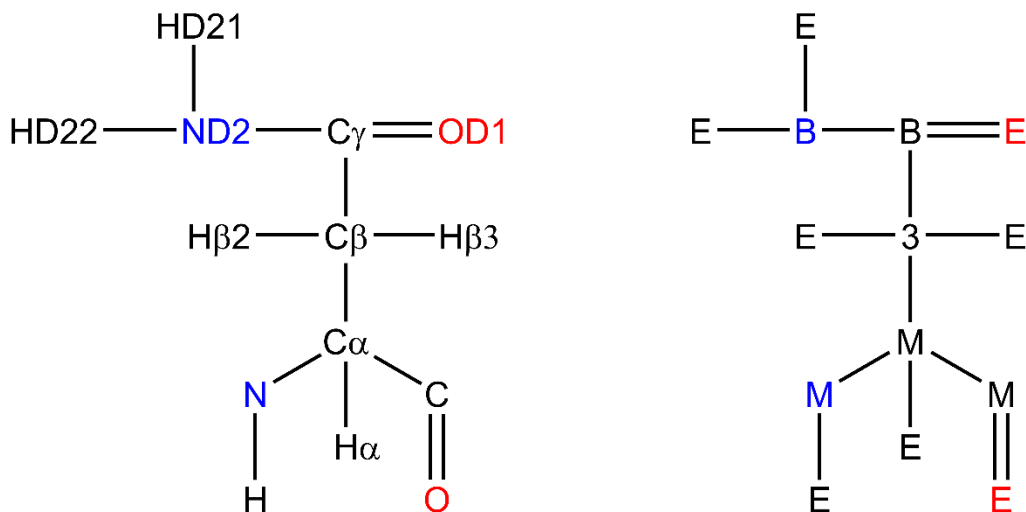
When building a protein, *tleap* will always bond the first non-dummy atom in the PREP file describing that residue to the second to last atom in the PREP file describing the preceding residue. This attachment order governs how bonds, angles, and torsions of other atoms are parameterized. As such, this format is essential for the correct parameterization of both a PREP file describing a standard amino and a PREPi file describing a modified amino acid.

Each line describing an atom contains further information detailing its properties. All lines describing an atom will take the following format.

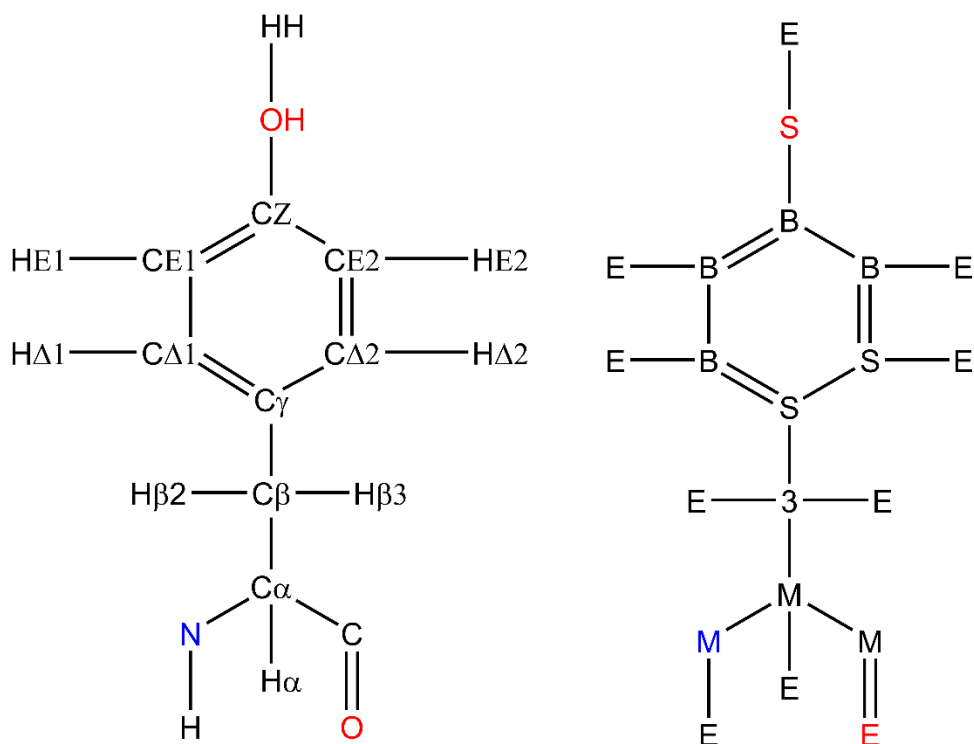
X	Name	Type	Tree	Bond	Ang	Dih	Dist	Adegrees	Ddegrees	Pcharge
---	------	------	------	------	-----	-----	------	----------	----------	---------

The term `x` will simply be a number assigned to the atom in the PREP format atom order. The term `Name` is the label applied to that atom. This is what will be displayed when visualizing a structure in a program like Chimera. The term `Type` assigns an atom type according to the force field used to describe this residue (usually GAFF or ff14SB).

The term `Tree` assigns a tree structure label indicating what branch and which connections an atom contains. Residues are constructed in *tleap* according to this tree structure. As you are likely aware, a tree consists of a trunk with many branches attached to it. Some of these branches have their own branches, until eventually the end of a branch is reached. This analogy is how the tree structure functions. An atom that is assigned an `M` is part of the main branch. The amino acid backbone atoms `N`, `CA`, and `C` will always be assigned `M` as the backbone is the main branch. Side branch atoms connected to the main branch (i.e. sidechain atoms) will be assigned either an `S` (indicating the atom is connected to two other atoms in total), a `B` (indicating the atom has three connections), a `3` (four connections), a `4` (five connections), or a `5` (six connections). An atom at the end of a branch will only have one connection to another atom, and will be assigned the label `E`. The following displays the tree structure labels applied to the atoms of a standard Asn residue (left is the Asn atom names, right is the tree structure labels).



It should be noted that if your sidechain contains a ring structure, two atoms will be arbitrarily picked to be loop closing atoms. Atoms designated as loop closing atoms will have one connection subtracted from the true number of connections to other atoms before assigning a tree structure label. The following displays this for tyrosine, where the atoms designated as loop closing atoms (C γ and C Δ 2) are assigned a tree structure label of s (for 2 total connections) despite having 3 total connections to other atoms.



The next six terms are the internal coordinates of each atom in the residue. As stated previously, internal coordinates differ from XYZ coordinates in that they aim to describe the position of each atom relative to other atoms in the residue.

The term `Bond` lists the atom number of an atom that the atom described in this line is directly bound to. The term `Ang` lists the atom number of an atom that forms a 3-point angle with the atom indicated by the term `Bond` and the atom in this line. The term `Dih` simply indicates the atom number of an atom that forms a 4-point dihedral angle / torsional angle with the atoms indicated by the term `Bond`, `Ang`, and the atom in this line.

The three terms following this quantify the connections described by the `Bond`, `Ang`, and `Dih` terms. The term `Dist` is the bond distance in angstroms (Å) between the atom indicated by the term `Bond` and the atom in this line. The term `Adegrees` is the 3-point angle in degrees formed by the atoms indicated by the terms `Bond` and `Ang` and the atom in this line. Lastly, the term `Ddegrees` is the 4-point angle in degrees formed by the atoms indicated by the terms `Bond`, `Ang`, and `Dih`, and the atom in this line.

The final atom description term is the partial charge of the atom in this line in atomic units. In the method we've used here, this will be a RESP partial charge.

From here, there are only two more sections of the PREP file format to discuss. The `Loop` section is simply a list of atoms that have been assigned to be loop or ring closing atoms. Note that this assignment is arbitrary, and that any pairs of atoms that are directly bound in a ring may be selected as ring closing. Using the branch structure example for the amino acid tyrosine, the atoms named `Cy` and `CA2` would be listed under this `Loop` section.

The final section `Improper` simply lists sets of 4 atoms that form an improper torsion. An improper torsion is, as the name implies, a dihedral / torsional angle that should not appear within the structure.

The last two lines `DONE` and `STOP` are relatively self-explanatory. `DONE` tells *tleap* that it has finished reading the PREPi file, whereas `STOP` tells it to stop the process that began reading the file. Both are necessary for a proper exit.

With that, we've described the PREP file. In this walkthrough, we will be making PREPi files for our modified amino acids. The PREPi file is a similarly formatted file used to describe nonstandard "amino acids" or modified amino acids, and is the format outputted by our `Adv_ligprep.sh` script.

Editing the PREPi File

Next, we'll be performing our first edits to the PREPi file. As mentioned previously, you will need to keep `ligand_NEW.pdb` open in Chimera as a visual reference for the atom names used in the PREPi file. Your PREPi file should look like the following example.

```

0      0      2
This is a remark line
molecule.res
MOL    INT    0
CORRECT      OMIT DU    BEG
0.0000
1  DUMM  DU    M    0  -1  -2    0.000    .0    .0    .00000
2  DUMM  DU    M    1   0  -1    1.449    .0    .0    .00000
3  DUMM  DU    M    2   1   0    1.523  111.21    .0    .00000
4  O1    o    M    3   2   1    1.540  111.208  -180.000  -0.254431
5  C2    c1    M    4   3   2    1.235    86.890  117.272   0.109915
6  C1    c3    M    5   4   3    1.531  121.850   -68.394   0.461705

```

7	N1	n2	S	6	5	4	1.453	109.768	-5.244	-0.628618
8	H1	hn	E	7	6	5	1.021	109.456	-55.902	0.350098
9	H2	h1	E	6	5	4	1.069	107.764	-124.537	0.026947
10	C3	c3	M	6	5	4	1.529	112.069	117.843	-0.490680
11	H3	hc	E	10	6	5	1.070	108.552	162.804	0.123878
12	H4	hc	E	10	6	5	1.070	107.480	41.615	0.123878
13	C4	c2	M	10	6	5	1.518	112.922	-76.740	0.976292
14	O2	o	E	13	10	6	1.233	120.990	-0.757	-0.618200
15	N2	n	M	13	10	6	1.332	116.319	179.263	-0.792554
16	H5	hn	E	15	13	10	0.984	118.388	0.871	0.432715
17	C5	c3	M	15	13	10	1.445	123.267	-179.175	0.339783
18	O5	os	E	17	15	13	1.436	112.404	-90.137	-0.466391
19	H6	h2	E	17	15	13	1.070	106.610	27.796	0.069679
20	C6	c3	M	17	15	13	1.533	110.765	146.873	-0.021588
21	N3	n	B	20	17	15	1.456	111.173	-56.673	-0.741150
22	C11	c2	B	21	20	17	1.338	123.198	84.294	0.923729
23	C12	c3	3	22	21	20	1.500	116.017	-179.873	-0.629818
24	H13	hc	E	23	22	21	1.070	109.479	-0.331	0.172764
25	H14	hc	E	23	22	21	1.070	109.460	-120.381	0.172764
26	H15	hc	E	23	22	21	1.070	109.465	119.653	0.172764
27	O7	o	E	22	21	20	1.229	121.995	-0.259	-0.687145
28	H16	hn	E	21	20	17	0.985	118.374	-95.736	0.385733
29	H7	h1	E	20	17	15	1.069	108.187	61.484	0.126035
30	C7	c3	M	20	17	15	1.527	110.266	-179.703	0.382185
31	O3	oh	S	30	20	17	1.425	109.658	-175.173	-0.713020
32	H17	ho	E	31	30	20	0.970	109.504	60.665	0.440166
33	H8	h1	E	30	20	17	1.070	108.911	65.295	0.061101
34	C8	c3	M	30	20	17	1.523	110.421	-53.844	0.089561
35	O4	oh	S	34	30	20	1.423	110.285	174.626	-0.658322
36	H18	ho	E	35	34	30	0.971	109.429	-60.984	0.420322
37	H9	h1	E	34	30	20	1.070	108.419	-65.738	0.093684
38	C9	c3	M	34	30	20	1.527	110.176	53.907	0.091800
39	H10	h1	E	38	34	30	1.070	105.920	65.453	0.046744
40	C10	c3	M	38	34	30	1.511	112.896	-175.072	0.374330
41	H11	h1	E	40	38	34	1.071	109.017	72.367	0.004798
42	H12	h1	E	40	38	34	1.070	108.403	-48.290	0.004798
43	O6	oh	M	40	38	34	1.420	111.284	-167.383	-0.718329
44	H19	ho	E	43	40	38	0.970	109.488	-60.898	0.442078
LOOP										
C9 O5										
IMPROPER										
C3 N2 C4 O2										
C12 N3 C11 O7										
DONE										
STOP										

First and foremost, you'll want to apply new atom names to all of the atoms in your PREPi. The atoms that make up the standard amino acid should have their standard names applied to them. Note that you can find these standard atom names in the file

/usr/local/amber/amber18/dat/leap/prop/aminol2.in on any of the Linux workstations.

For the remaining atoms that are a part of your modification, I'd suggest simply renaming them to match the changed names of the standard atoms (i.e. C1 is changed to CA as it was a backbone atom, so since C5 is not a backbone atom and will not be given a specific name, simply change it to C1). You'll also want to change the default 3-letter code in line 5 from MOL to whatever code you want to assign to your modified amino acid (in this case, NAG).

You'll also want to be sure to change the atom typing of the atoms associated with your backbone carbonyl carbon and amine nitrogen. Since these atoms each had an attached hydrogen deleted from them in the inputted PDB file, they will be assigned incorrect atom types by *antechamber*. The amine nitrogen will be assigned the GAFF atom type `n2` and the carbonyl carbon will be assigned `c1`. These will need to be changed to the GAFF atom types `n` and `c` respectively (note that this incorrect atom typing will have other effects that will be addressed later on).

Once you've finished this, your file should look something like the following example.

```

0      0      2
This is a remark line
molecule.res
NAG      INT      0
CORRECT      OMIT DU      BEG
0.0000
1  DUMM  DU    M    0  -1  -2    0.000    .0    .0    .00000
2  DUMM  DU    M    1   0  -1    1.449    .0    .0    .00000
3  DUMM  DU    M    2   1   0    1.523   111.21    .0    .00000
4  O      o    M    3   2   1    1.540   111.208  -180.000  -0.254431
5  C      c    M    4   3   2    1.235    86.890   117.272   0.109915
6  CA     c3   M    5   4   3    1.531   121.850   -68.394   0.461705
7  N      n    S    6   5   4    1.453   109.768    -5.244   -0.628618
8  H      hn   E    7   6   5    1.021   109.456   -55.902   0.350098
9  HA     h1   E    6   5   4    1.069   107.764  -124.537   0.026947
10 CB     c3   M    6   5   4    1.529   112.069   117.843  -0.490680
11 HB2    hc   E   10   6   5    1.070   108.552   162.804   0.123878
12 HB3    hc   E   10   6   5    1.070   107.480    41.615   0.123878
13 CG     c2   M   10   6   5    1.518   112.922   -76.740   0.976292
14 OD1    o    E   13  10   6    1.233   120.990    -0.757  -0.618200
15 ND2    n    M   13  10   6    1.332   116.319   179.263  -0.792554
16 HD21   hn   E   15  13  10    0.984   118.388     0.871   0.432715
17 C1     c3   M   15  13  10    1.445   123.267  -179.175   0.339783
18 O5     os   E   17  15  13    1.436   112.404   -90.137  -0.466391
19 H1     h2   E   17  15  13    1.070   106.610    27.796   0.069679
20 C2     c3   M   17  15  13    1.533   110.765   146.873  -0.021588
21 N2     n    B   20  17  15    1.456   111.173   -56.673  -0.741150
22 C7     c2   B   21  20  17    1.338   123.198    84.294   0.923729
23 C8     c3   3    22  21  20    1.500   116.017  -179.873  -0.629818
24 H81    hc   E   23  22  21    1.070   109.479    -0.331   0.172764
25 H82    hc   E   23  22  21    1.070   109.460  -120.381   0.172764
26 H83    hc   E   23  22  21    1.070   109.465   119.653   0.172764
27 O7     o    E   22  21  20    1.229   121.995    -0.259  -0.687145
28 HN2    hn   E   21  20  17    0.985   118.374   -95.736   0.385733
29 H2     h1   E   20  17  15    1.069   108.187    61.484   0.126035
30 C3     c3   M   20  17  15    1.527   110.266  -179.703   0.382185
31 O3     oh   S   30  20  17    1.425   109.658  -175.173  -0.713020

```

```

32 HO3 ho E 31 30 20 0.970 109.504 60.665 0.440166
33 H3 h1 E 30 20 17 1.070 108.911 65.295 0.061101
34 C4 c3 M 30 20 17 1.523 110.421 -53.844 0.089561
35 O4 oh S 34 30 20 1.423 110.285 174.626 -0.658322
36 HO4 ho E 35 34 30 0.971 109.429 -60.984 0.420322
37 H4 h1 E 34 30 20 1.070 108.419 -65.738 0.093684
38 C5 c3 M 34 30 20 1.527 110.176 53.907 0.091800
39 H5 h1 E 38 34 30 1.070 105.920 65.453 0.046744
40 C6 c3 M 38 34 30 1.511 112.896 -175.072 0.374330
41 H61 h1 E 40 38 34 1.071 109.017 72.367 0.004798
42 H62 h1 E 40 38 34 1.070 108.403 -48.290 0.004798
43 O6 oh M 40 38 34 1.420 111.284 -167.383 -0.718329
44 HO6 ho E 43 40 38 0.970 109.488 -60.898 0.442078

LOOP
  C5 O5

IMPROPER
  CB ND2 CG OD1
  C8 N2 C7 O7

DONE
STOP

```

Once this is completed, we'll next need to change the atom typing. In the atom type column, you'll note that by default our `Adv_ligprep.sh` script will write out GAFF atom types. For a modified amino acid / covalent ligand that is to be read by AMBER as part of your protein, it will be necessary to convert these atom types to that of ff14SB. The program *antechamber* does not provide us with an option to write out an ff14SB atom typed PREPi file. As such, we will need to assign these atom types manually. The following table provides a list of GAFF carbon atom types and their ff14SB equivalents.

GAFF Atom Type	ff14SB Atom Type	Description
c	C	sp2 C carbonyl group
c1	CZ	sp C
c2	No simple equivalent	sp2 C
	CB	sp2 aromatic C, 5&6 membered ring junction
	CK	sp2 C 5 memb.ring in purines (dA,dG)
	CP	sp2 C 5 memb.ring in purines (G)
	CM	sp2 C pyrimidines in pos. 5 & 6 (dT,dC)
	CS	sp2 C pyrimidines in pos. 5 & 6 (U)

	CN	sp ² C aromatic 5&6 memb.ring junct.(TRP)
	CQ	sp ² C in 5 memb.ring of purines between 2 N
	CR	sp ² arom as CQ but in HIS
	CV	sp ² arom. 5 memb.ring w/1 N and 1 H (HIS)
	CW	sp ² arom. 5 memb.ring w/1 N-H and 1 H (HIS)
	C*	sp ² arom. 5 memb.ring w/1 subst. (TRP)
	C5	sp ² C 5 memb.ring in purines (A)
	C4	sp ² C pyrimidines in pos. 5 & 6 (C)
c3	CT	sp ³ C
ca	CA	sp ² C in pure aromatic system
cp / cq	No equivalent	Head sp ² C that connect two rings in biphenyl system
cc / cd	CC	sp ² carbons in non-pure aromatic systems
ce / cf	CD	Inner sp ² carbons in conjugated systems
cg / ch	No equivalent	Inner sp carbons in conjugated systems
cx	No equivalent	sp ³ carbons in triangle systems
cy	No equivalent	sp ³ carbons in square systems
cu	No equivalent	sp ² carbons in triangle systems
cv	No equivalent	sp ² carbons in square systems
cz	No equivalent	sp ² carbon in guanidine group
No equivalent	CX	protein C-alpha
No equivalent	CY	nitrile C

It should be noted that there are certain atom types that do not have equivalents. This is because GAFF and ff14SB were both designed to perform different functions. GAFF functions to parameterize organic molecules whereas ff14SB functions to parameterize amino acids. In the event that you cannot find a direct ff14SB equivalent to a GAFF atom type, you'll need to rely on your own chemical intuition to assign the closest equivalent type.

The following table provides a list of GAFF hydrogen atom types and their ff14SB equivalents. You'll note that for most hydrogens there is an equivalent atom type in both force fields.

GAFF Atom Type	ff14SB Atom Type	Description
h1	H1	H bonded to aliphatic carbon with 1 electrwd. group
h2	H2	H bonded to aliphatic carbon with 2 electrwd. group
h3	H3	H bonded to aliphatic carbon with 3 electrwd. group
h4	H4	H bonded to non-sp ³ carbon with 1 electrwd. group
h5	H5	H bonded to non-sp ³ carbon with 2 electrwd. group
ha	HA	H bonded to aromatic carbon
hc	HC	H bonded to aliphatic carbon without electrwd. group
hn	H	H bonded to nitrogen atoms
ho	HO	Hydroxyl group
hp	No equivalent	H bonded to phosphate
hs	HS	Hydrogen bonded to sulphur
hw	HW	Hydrogen in water
hx	HP	H bonded to C next to positively charged group
No equivalent	HZ	H bond sp C

The following table provides a list of GAFF nitrogen atom types and their ff14SB equivalents.

GAFF Atom Type	ff14SB Atom Type	Description
n	N	sp ² nitrogen in amide groups
n1	No equivalent	sp N
n2	No simple equivalent	aliphatic sp ² N with two connected atoms
	N*	sp ² N
	NA	sp ² N in 5 memb.ring w/H atom (HIS)
	NB	sp ² N in 5 memb.ring w/LP (HIS,ADE,GUA)
	NC	sp ² N in 6 memb.ring w/LP (ADE,GUA)
n3	NT	sp ³ N with three connected atoms

n4	N3	sp3 N with four connected atoms
na	N2	sp2 N with three connected atoms
nb	No equivalent	sp2 N in pure aromatic systems
nc / nd	No equivalent	sp2 N in pure aromatic systems
ne / nf	No equivalent	Inner sp2 N in conjugated systems
nh	No equivalent	Amine N connected one or more aromatic rings
no	No equivalent	Nitro N
ni	No equivalent	n in 3-membered rings
nj	No equivalent	n in 4-membered rings
nk	No equivalent	n4 in 3-membered rings
nl	No equivalent	n4 in 4-membered rings
nm	No equivalent	nh in 3-membered rings
nn	No equivalent	nh in 4-membered rings
np	No equivalent	n3 in 3-membered rings
nq	No equivalent	n3 in 4-membered rings
No equivalent	NY	Nitrile N

It should be noted that this is where major problems may begin to arise from nonequivalent atom types across both force fields. Many of the GAFF nitrogen atom types are simply not described in ff14SB. In the event that you cannot find a suitable equivalent, it might be necessary for you to manually add in the GAFF atom type in *tleap* and describe its parameters using an *frmod* file. This will be described [in more depth later](#).

The following table provides a list of other GAFF atom types and their ff14SB equivalents.

GAFF Atom Type	ff14SB Atom Type	Description
f	F	Fluorine
cl	Cl	Chlorine
br	Br	Bromine
i	I	Iodine
o	O	Oxygen with one connected atom
oh	OH	Oxygen in hydroxyl group
os	OS	Ether and ester oxygen
op	No equivalent	os in 3-membered rings
oq	No equivalent	os in 4-membered rings
ow	OW	Oxygen in water

No equivalent	O2	Carboxyl and phosphate oxygen
p2	No equivalent	Phosphate with two connected atoms
p3	No equivalent	Phosphate with three connected atoms, such as PH3
p4	No equivalent	Phosphate with three connected atoms, such as O=P(CH3)2
p5	P	Phosphate with four connected atoms, such as O=P(OH)3
pb	No equivalent	sp2 P in pure aromatic systems
pc / pd	No equivalent	sp2 P in non-pure aromatic systems
pe / pf	No equivalent	Inner Sp2 P in conjugated systems
px	No equivalent	Special p4 in conjugated systems
py	No equivalent	Special p5 in conjugated systems
s	No equivalent	S with one connected atom
s2	No equivalent	S with two connected atoms, involved at least one double bond
s4	No equivalent	S with three connected atoms
s6	No equivalent	S with four connected atoms
sh	SH	sp3 S connected with hydrogen
ss	S	sp3 S in thio-ester and thio-ether
sp	No equivalent	ss in 3-membered rings
sq	No equivalent	ss in 4-membered rings
sx	No equivalent	Special s4 in conjugated systems
sy	No equivalent	Special s6 in conjugated systems

It should be noted that ff14SB is largely lacking on parameters for sulfur and phosphorous atoms. Once again, given that ff14SB was designed to function primarily with amino acids, this is not particularly surprising. In the event that the specific atom type you need has no suitable equivalent, you'll need to manually add the GAFF atom type in *tleap* and describe its parameters using an frcmod file. This process is described [here](#).

Note that if you're looking for more information on the particular force fields and their atom types, you can find the force field parameters in the directory

/usr/local/amber/amber18/dat/leap/parm on any Linux workstation. Furthermore, you could search the literature and read publications describing both force fields to get a better idea for examples of how each atom type should be used. The publication describing GAFF can be found [here](#) along with Amber's description of GAFF [here](#). The paper describing ff14SB can be found [here](#).

Now that we've converted our GAFF atom types to ff14SB, your new PREPi file should look like the following example.

```

0      0      2
This is a remark line
molecule.res
NAG      INT      0
CORRECT      OMIT DU      BEG
0.0000
1  DUMM  DU      M      0  -1  -2      0.000      .0      .0      .00000
2  DUMM  DU      M      1   0  -1      1.449      .0      .0      .00000
3  DUMM  DU      M      2   1   0      1.523     111.21      .0      .00000
4  O      O      M      3   2   1      1.540     111.208    -180.000    -0.254431
5  C      C      M      4   3   2      1.235      86.890     117.272     0.109915
6  CA     CX      M      5   4   3      1.531     121.850     -68.394     0.461705
7  N      N      S      6   5   4      1.453     109.768     -5.244     -0.628618
8  H      H      E      7   6   5      1.021     109.456     -55.902     0.350098
9  HA     H1      E      6   5   4      1.069     107.764    -124.537     0.026947
10 CB     CT      M      6   5   4      1.529     112.069     117.843    -0.490680
11 HB2    HC      E     10   6   5      1.070     108.552     162.804     0.123878
12 HB3    HC      E     10   6   5      1.070     107.480      41.615     0.123878
13 CG     C      M     10   6   5      1.518     112.922     -76.740     0.976292
14 OD1    O      E     13  10   6      1.233     120.990      -0.757    -0.618200
15 ND2    N      M     13  10   6      1.332     116.319     179.263    -0.792554
16 HD21   H      E     15  13  10      0.984     118.388       0.871     0.432715
17 C1     CT      M     15  13  10      1.445     123.267    -179.175     0.339783
18 O5     OS      E     17  15  13      1.436     112.404     -90.137    -0.466391
19 H1     H2      E     17  15  13      1.070     106.610      27.796     0.069679
20 C2     CT      M     17  15  13      1.533     110.765     146.873    -0.021588
21 N2     N      B     20  17  15      1.456     111.173     -56.673    -0.741150
22 C7     C      B     21  20  17      1.338     123.198      84.294     0.923729
23 C8     CT      3     22  21  20      1.500     116.017    -179.873    -0.629818
24 H81    HC      E     23  22  21      1.070     109.479      -0.331     0.172764
25 H82    HC      E     23  22  21      1.070     109.460    -120.381     0.172764
26 H83    HC      E     23  22  21      1.070     109.465     119.653     0.172764
27 O7     O      E     22  21  20      1.229     121.995      -0.259    -0.687145
28 HN2    H      E     21  20  17      0.985     118.374     -95.736     0.385733
29 H2     H1      E     20  17  15      1.069     108.187      61.484     0.126035
30 C3     CT      M     20  17  15      1.527     110.266    -179.703     0.382185
31 O3     OH      S     30  20  17      1.425     109.658    -175.173    -0.713020
32 HO3    HO      E     31  30  20      0.970     109.504      60.665     0.440166
33 H3     H1      E     30  20  17      1.070     108.911      65.295     0.061101
34 C4     CT      M     30  20  17      1.523     110.421     -53.844     0.089561

```

35	O4	OH	S	34	30	20	1.423	110.285	174.626	-0.658322
36	HO4	HO	E	35	34	30	0.971	109.429	-60.984	0.420322
37	H4	H1	E	34	30	20	1.070	108.419	-65.738	0.093684
38	C5	CT	M	34	30	20	1.527	110.176	53.907	0.091800
39	H5	H1	E	38	34	30	1.070	105.920	65.453	0.046744
40	C6	CT	M	38	34	30	1.511	112.896	-175.072	0.374330
41	H61	H1	E	40	38	34	1.071	109.017	72.367	0.004798
42	H62	H1	E	40	38	34	1.070	108.403	-48.290	0.004798
43	O6	OH	M	40	38	34	1.420	111.284	-167.383	-0.718329
44	HO6	HO	E	43	40	38	0.970	109.488	-60.898	0.442078

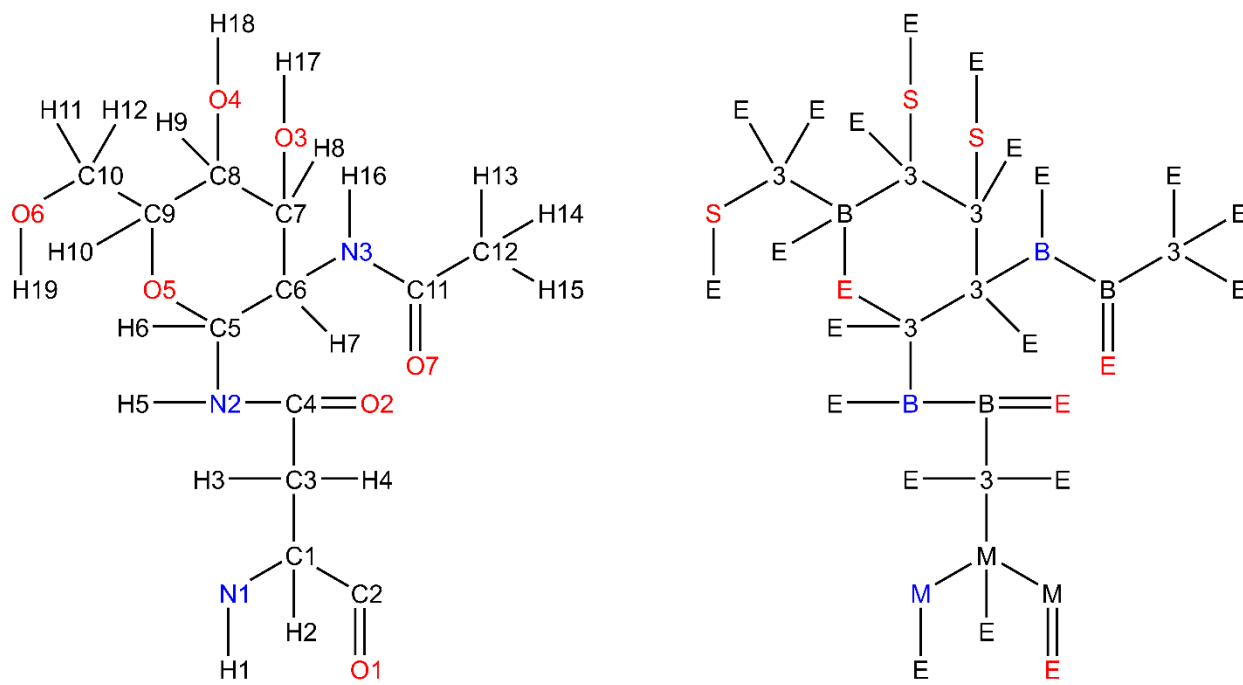
LOOP
C5 O5

IMPROPER
CB ND2 CG OD1
C8 N2 C7 O7

DONE
STOP

From here, you'll next need to change the tree structure label assigned to each atom. By default, *antechamber* will simply pick the longest carbon chain and label this as the main branch. However, as [described in detail above](#), this is not sufficient for describing an amino acid. As such, the backbone atoms will need to be manually relabeled as main branch atoms. All other atoms should then be relabeled accordingly using the labels described above.

The correct branch structure for NAG looks like the following.



Once these changes have been made, your new PREPi file should look like the following example.

0	0	2								
This is a remark line										
molecule.res										
NAG	INT	0								
CORRECT	OMIT	DU	BEG							
0.0000										
1	DUMM	DU	M	0	-1	-2	0.000	.0	.0	.00000
2	DUMM	DU	M	1	0	-1	1.449	.0	.0	.00000
3	DUMM	DU	M	2	1	0	1.523	111.21	.0	.00000
4	O	O	E	3	2	1	1.540	111.208	-180.000	-0.254431
5	C	C	M	4	3	2	1.235	86.890	117.272	0.109915
6	CA	CX	M	5	4	3	1.531	121.850	-68.394	0.461705
7	N	N	M	6	5	4	1.453	109.768	-5.244	-0.628618
8	H	H	E	7	6	5	1.021	109.456	-55.902	0.350098
9	HA	H1	E	6	5	4	1.069	107.764	-124.537	0.026947
10	CB	2C	3	6	5	4	1.529	112.069	117.843	-0.490680
11	HB2	HC	E	10	6	5	1.070	108.552	162.804	0.123878
12	HB3	HC	E	10	6	5	1.070	107.480	41.615	0.123878
13	CG	C	B	10	6	5	1.518	112.922	-76.740	0.976292
14	OD1	O	E	13	10	6	1.233	120.990	-0.757	-0.618200
15	ND2	N	B	13	10	6	1.332	116.319	179.263	-0.792554
16	HD21	H	E	15	13	10	0.984	118.388	0.871	0.432715
17	C1	CT	3	15	13	10	1.445	123.267	-179.175	0.339783
18	O5	OS	E	17	15	13	1.436	112.404	-90.137	-0.466391
19	H1	H2	E	17	15	13	1.070	106.610	27.796	0.069679
20	C2	CT	3	17	15	13	1.533	110.765	146.873	-0.021588
21	N2	N	B	20	17	15	1.456	111.173	-56.673	-0.741150
22	C7	C	B	21	20	17	1.338	123.198	84.294	0.923729
23	C8	CT	3	22	21	20	1.500	116.017	-179.873	-0.629818
24	H81	HC	E	23	22	21	1.070	109.479	-0.331	0.172764
25	H82	HC	E	23	22	21	1.070	109.460	-120.381	0.172764
26	H83	HC	E	23	22	21	1.070	109.465	119.653	0.172764
27	O7	O	E	22	21	20	1.229	121.995	-0.259	-0.687145
28	HN2	H	E	21	20	17	0.985	118.374	-95.736	0.385733
29	H2	H1	E	20	17	15	1.069	108.187	61.484	0.126035
30	C3	CT	3	20	17	15	1.527	110.266	-179.703	0.382185
31	O3	OH	S	30	20	17	1.425	109.658	-175.173	-0.713020
32	HO3	HO	E	31	30	20	0.970	109.504	60.665	0.440166
33	H3	H1	E	30	20	17	1.070	108.911	65.295	0.061101
34	C4	CT	3	30	20	17	1.523	110.421	-53.844	0.089561
35	O4	OH	S	34	30	20	1.423	110.285	174.626	-0.658322
36	HO4	HO	E	35	34	30	0.971	109.429	-60.984	0.420322
37	H4	H1	E	34	30	20	1.070	108.419	-65.738	0.093684
38	C5	CT	B	34	30	20	1.527	110.176	53.907	0.091800
39	H5	H1	E	38	34	30	1.070	105.920	65.453	0.046744
40	C6	CT	3	38	34	30	1.511	112.896	-175.072	0.374330
41	H61	H1	E	40	38	34	1.071	109.017	72.367	0.004798
42	H62	H1	E	40	38	34	1.070	108.403	-48.290	0.004798
43	O6	OH	S	40	38	34	1.420	111.284	-167.383	-0.718329
44	HO6	HO	E	43	40	38	0.970	109.488	-60.898	0.442078

```

LOOP
  C5   O5

IMPROPER
  CB   ND2  CG   OD1
  C8   N2   C7   O7

DONE
STOP

```

One more change to the PREPi file will be necessary before we can move on. As noted above, all amino acid PREP files follow a very specific format. As you might expect by now, *antechamber* does not follow this format by default when creating a PREPi file. As such, we'll need to change the format ourselves. Your new file will look like the following.

```

      0      0      2

This is a remark line
molecule.res
NAG      INT      0
CORRECT      OMIT DU      BEG
0.0000
  1  DUMM  DU      M      0  -1  -2      0.000      .0      .0      .00000
  2  DUMM  DU      M      1   0  -1      1.449      .0      .0      .00000
  3  DUMM  DU      M      2   1   0      1.523     111.21      .0      .00000
  7  N      N      M      6   5   4      1.453     109.768     -5.244    -0.628618
  8  H      H      E      7   6   5      1.021     109.456     -55.902    0.350098
  6  CA     CX      M      5   4   3      1.531     121.850     -68.394    0.461705
  9  HA     H1      E      6   5   4      1.069     107.764    -124.537    0.026947
 10  CB     2C      3      6   5   4      1.529     112.069     117.843   -0.490680
 11  HB2    HC      E     10   6   5      1.070     108.552     162.804    0.123878
 12  HB3    HC      E     10   6   5      1.070     107.480      41.615    0.123878
 13  CG     C      B     10   6   5      1.518     112.922     -76.740    0.976292
 14  OD1    O      E     13  10   6      1.233     120.990     -0.757   -0.618200
 15  ND2    N      B     13  10   6      1.332     116.319     179.263   -0.792554
 16  HD21   H      E     15  13  10      0.984     118.388       0.871    0.432715
 17  C1     CT      3     15  13  10      1.445     123.267   -179.175    0.339783
 18  O5     OS      E     17  15  13      1.436     112.404     -90.137   -0.466391
 19  H1     H2      E     17  15  13      1.070     106.610      27.796    0.069679
 20  C2     CT      3     17  15  13      1.533     110.765     146.873   -0.021588
 21  N2     N      B     20  17  15      1.456     111.173     -56.673   -0.741150
 22  C7     C      B     21  20  17      1.338     123.198      84.294    0.923729
 23  C8     CT      3     22  21  20      1.500     116.017   -179.873   -0.629818
 24  H81    HC      E     23  22  21      1.070     109.479      -0.331    0.172764
 25  H82    HC      E     23  22  21      1.070     109.460   -120.381    0.172764
 26  H83    HC      E     23  22  21      1.070     109.465     119.653    0.172764
 27  O7     O      E     22  21  20      1.229     121.995      -0.259   -0.687145
 28  HN2    H      E     21  20  17      0.985     118.374     -95.736    0.385733
 29  H2     H1      E     20  17  15      1.069     108.187      61.484    0.126035
 30  C3     CT      3     20  17  15      1.527     110.266   -179.703    0.382185
 31  O3     OH      S     30  20  17      1.425     109.658   -175.173   -0.713020
 32  HO3    HO      E     31  30  20      0.970     109.504      60.665    0.440166
 33  H3     H1      E     30  20  17      1.070     108.911      65.295    0.061101
 34  C4     CT      3     30  20  17      1.523     110.421    -53.844    0.089561

```

```

35 O4 OH S 34 30 20 1.423 110.285 174.626 -0.658322
36 HO4 HO E 35 34 30 0.971 109.429 -60.984 0.420322
37 H4 H1 E 34 30 20 1.070 108.419 -65.738 0.093684
38 C5 CT B 34 30 20 1.527 110.176 53.907 0.091800
39 H5 H1 E 38 34 30 1.070 105.920 65.453 0.046744
40 C6 CT 3 38 34 30 1.511 112.896 -175.072 0.374330
41 H61 H1 E 40 38 34 1.071 109.017 72.367 0.004798
42 H62 H1 E 40 38 34 1.070 108.403 -48.290 0.004798
43 O6 OH S 40 38 34 1.420 111.284 -167.383 -0.718329
44 HO6 HO E 43 40 38 0.970 109.488 -60.898 0.442078
5 C C M 4 3 2 1.235 86.890 117.272 0.109915
4 O O E 3 2 1 1.540 111.208 -180.000 -0.254431

```

LOOP

C5 O5

IMPROPER

CB ND2 CG OD1
C8 N2 C7 O7

DONE

STOP

This, however, creates a major problem. Switching the order of these atoms breaks the atom numbering in the PREPi file. As such, all atoms, including all atom numbers listed in the three connection columns, will need to be renumbered. Given the sheer amount of changes this necessitates, I recommend using the vim command `:s/old/new/g` to renumber each instance of each atom in the file. For example, to change the numbering of the atom 5 C to 45 C I would recommend entering the command `:s/ 5 / 45 /g` where the spaces before and after are included to prevent other instances where the number 4 occurs (for example, in the internal coordinates) from being changed.

After making these changes, your new file should look like the following.

```

0 0 2

This is a remark line
molecule.res
NAG INT 0
CORRECT OMIT DU BEG
0.0000
1 DUMM DU M 0 -1 -2 0.000 0.000 0.000 0.00000
2 DUMM DU M 1 0 -1 1.449 0.000 0.000 0.00000
3 DUMM DU M 2 1 0 1.522 111.100 0.000 0.00000
4 N N M 3 2 1 1.453 109.768 -5.244 -0.628618
5 H H E 4 3 2 1.021 109.456 -55.902 0.350098
6 CA CX M 4 3 2 1.531 121.850 -68.394 0.461705
7 HA H1 E 6 4 3 1.069 107.764 -124.537 0.026947
8 CB 2C 3 6 4 3 1.529 112.069 117.843 -0.490680
9 HB2 HC E 8 6 4 1.070 108.552 162.804 0.123878
10 HB3 HC E 8 6 4 1.070 107.480 41.615 0.123878
11 CG C B 8 6 4 1.518 112.922 -76.740 0.976292

```

12	OD1	O	E	11	8	6	1.233	120.990	-0.757	-0.618200
13	ND2	N	B	11	8	6	1.332	116.319	179.263	-0.792554
14	HD21	H	E	13	11	8	0.984	118.388	0.871	0.432715
15	C1	CT	3	13	11	8	1.445	123.267	-179.175	0.339783
16	O5	OS	E	15	13	11	1.436	112.404	-90.137	-0.466391
17	H1	H2	E	15	13	11	1.070	106.610	27.796	0.069679
18	C2	CT	3	15	13	11	1.533	110.765	146.873	-0.021588
19	N2	N	B	18	15	13	1.456	111.173	-56.673	-0.741150
20	C7	C	B	19	18	15	1.338	123.198	84.294	0.923729
21	C8	CT	3	20	19	18	1.500	116.017	-179.873	-0.629818
22	H81	HC	E	21	20	19	1.070	109.479	-0.331	0.172764
23	H82	HC	E	21	20	19	1.070	109.460	-120.381	0.172764
24	H83	HC	E	21	20	19	1.070	109.465	119.653	0.172764
25	O7	O	E	20	19	18	1.229	121.995	-0.259	-0.687145
26	HN2	H	E	19	18	15	0.985	118.374	-95.736	0.385733
27	H2	H1	E	18	15	13	1.069	108.187	61.484	0.126035
28	C3	CT	3	18	15	13	1.527	110.266	-179.703	0.382185
29	O3	OH	S	28	18	15	1.425	109.658	-175.173	-0.713020
30	HO3	HO	E	29	28	18	0.970	109.504	60.665	0.440166
31	H3	H1	E	28	18	15	1.070	108.911	65.295	0.061101
32	C4	CT	3	28	18	15	1.523	110.421	-53.844	0.089561
33	O4	OH	S	32	28	18	1.423	110.285	174.626	-0.658322
34	HO4	HO	E	33	32	28	0.971	109.429	-60.984	0.420322
35	H4	H1	E	32	28	18	1.070	108.419	-65.738	0.093684
36	C5	CT	B	32	28	18	1.527	110.176	53.907	0.091800
37	H5	H1	E	36	32	28	1.070	105.920	65.453	0.046744
38	C6	CT	3	36	32	28	1.511	112.896	-175.072	0.374330
39	H61	H1	E	38	36	32	1.071	109.017	72.367	0.004798
40	H62	H1	E	38	36	32	1.070	108.403	-48.290	0.004798
41	O6	OH	S	38	36	32	1.420	111.284	-167.383	-0.718329
42	HO6	HO	E	41	38	36	0.970	109.488	-60.898	0.442078
43	C	C	M	6	4	3	1.235	86.890	117.272	0.109915
44	O	O	E	43	6	4	1.540	111.208	-180.000	-0.254431
LOOP										
C5 O5										
IMPROPER										
CB ND2 CG OD1										
C8 N2 C7 O7										
DONE										
STOP										

There is still one last change that we need to make. The `Adv_ligprep.sh` script uses Gaussian 16 to calculate the electrostatic potential (ESP) of the molecule, and *antechamber* performs a relative electrostatic potential (RESP) fitting to assign RESP partial charges to each atom.

In the structure we originally used, the backbone amine nitrogen and the carboxyl carbon are both missing the bonds they would typically have to the backbone atoms of other amino acids. This affects the partial charges on these atoms significantly. This means that the closer an atom is to the backbone amine nitrogen and the carboxyl carbon, the more incorrect the partial charge calculated for it will be. The atoms in the modification are sufficiently far from the

backbone as to not have any major inaccuracies. However, the backbone itself will have largely inaccurate partial charges.

To correct this issue, I recommend copying the partial charges detailed in the file `/usr/local/amber/amber18/dat/leap/rep/aminol2.in` for the atoms in the standard amino acid and pasting these in for the corresponding atoms in your PREPi file. For example, the atoms in the NAG modified amino acid corresponding to the standard Asn residue are shown below, with the partial charges detailed in the final column.

4	N	N	M	3	2	1	1.453	109.768	-5.244	-0.628618
5	H	H	E	4	3	2	1.021	109.456	-55.902	0.350098
6	CA	CX	M	4	3	2	1.531	121.850	-68.394	0.461705
7	HA	H1	E	6	4	3	1.069	107.764	-124.537	0.026947
8	CB	2C	3	6	4	3	1.529	112.069	117.843	-0.490680
9	HB2	HC	E	8	6	4	1.070	108.552	162.804	0.123878
10	HB3	HC	E	8	6	4	1.070	107.480	41.615	0.123878
11	CG	C	B	8	6	4	1.518	112.922	-76.740	0.976292
12	OD1	O	E	11	8	6	1.233	120.990	-0.757	-0.618200
13	ND2	N	B	11	8	6	1.332	116.319	179.263	-0.792554
14	HD21	H	E	13	11	8	0.984	118.388	0.871	0.432715
...										
43	C	C	M	6	4	3	1.235	86.890	117.272	0.109915
44	O	O	E	43	6	4	1.540	111.208	-180.000	-0.254431

The partial charges for these same atoms in Asn as detailed in the file `/usr/local/amber/amber18/dat/leap/rep/aminol2.in` can be seen in the following example.

4	N	N	M	3	2	1	1.335	116.600	180.000	-0.41570
5	H	H	E	4	3	2	1.010	119.800	0.000	0.27190
6	CA	CX	M	4	3	2	1.449	121.900	180.000	0.01430
7	HA	H1	E	6	4	3	1.090	109.500	300.000	0.10480
8	CB	2C	3	6	4	3	1.525	111.100	60.000	-0.20410
9	HB2	HC	E	8	6	4	1.090	109.500	300.000	0.07970
10	HB3	HC	E	8	6	4	1.090	109.500	60.000	0.07970
11	CG	C	B	8	6	4	1.522	111.100	180.000	0.71300
12	OD1	O	E	11	8	6	1.229	120.500	0.000	-0.59310
13	ND2	N	B	11	8	6	1.335	116.600	180.000	-0.91910
14	HD21	H	E	13	11	8	1.010	119.800	180.000	0.41960
...										
43	C	C	M	6	4	3	1.522	111.100	180.000	0.59730
44	O	O	E	43	6	4	1.229	120.500	0.000	-0.56790

You'll also note that there are some differences in the bond distances, angles, and torsions reported in the file above. I would highly recommend copy and pasting these from the file `/usr/local/amber/amber18/dat/leap/rep/aminol2.in` as well. This further corrects issues associated with the atypical bond orders and subsequent incorrect GAFF atom typing on our backbone carbonyl carbon and amine nitrogen in the original inputted PDB.

After making these last changes, your completed PREPi file will look like the following example.

```

0      0      2

This is a remark line
molecule.res
NAG      INT      0
CORRECT      OMIT DU      BEG
0.0000
  1  DUMM  DU      M      0  -1  -2      0.000      0.000      0.000      0.00000
  2  DUMM  DU      M      1   0  -1      1.449      0.000      0.000      0.00000
  3  DUMM  DU      M      2   1   0      1.522     111.100      0.000      0.00000
  4   N      N      M      3   2   1      1.335     116.600     180.000     -0.41570
  5   H      H      E      4   3   2      1.010     119.800      0.000      0.27190
  6  CA     CX      M      4   3   2      1.449     121.900     180.000      0.01430
  7  HA     H1     E      6   4   3      1.090     109.500     300.000      0.10480
  8  CB     2C     3      6   4   3      1.525     111.100      60.000     -0.20410
  9  HB2    HC     E      8   6   4      1.090     109.500     300.000      0.07970
 10  HB3    HC     E      8   6   4      1.090     109.500      60.000      0.07970
 11  CG     C      B      8   6   4      1.522     111.100     180.000      0.71300
 12  OD1    O      E     11   8   6      1.229     120.500      0.000     -0.59310
 13  ND2    N      B     11   8   6      1.335     116.600     180.000     -0.91910
 14  HD21   H      E     13  11   8      1.010     119.800     180.000      0.41960
 15  C1     CT     3     13  11   8      1.445     123.267    -179.175      0.339783
 16  O5     OS     E     15  13  11      1.436     112.404     -90.137     -0.466391
 17  H1     H2     E     15  13  11      1.070     106.610      27.796      0.069679
 18  C2     CT     3     15  13  11      1.533     110.765     146.873     -0.021588
 19  N2     N      B     18  15  13      1.456     111.173     -56.673     -0.741150
 20  C7     C      B     19  18  15      1.338     123.198      84.294      0.923729
 21  C8     CT     3     20  19  18      1.500     116.017    -179.873     -0.629818
 22  H81    HC     E     21  20  19      1.070     109.479      -0.331      0.172764
 23  H82    HC     E     21  20  19      1.070     109.460    -120.381      0.172764
 24  H83    HC     E     21  20  19      1.070     109.465     119.653      0.172764
 25  O7     O      E     20  19  18      1.229     121.995      -0.259     -0.687145
 26  HN2    H      E     19  18  15      0.985     118.374     -95.736      0.385733
 27  H2     H1     E     18  15  13      1.069     108.187      61.484      0.126035
 28  C3     CT     3     18  15  13      1.527     110.266    -179.703      0.382185
 29  O3     OH     S     28  18  15      1.425     109.658    -175.173     -0.713020
 30  HO3    HO     E     29  28  18      0.970     109.504      60.665      0.440166
 31  H3     H1     E     28  18  15      1.070     108.911      65.295      0.061101
 32  C4     CT     3     28  18  15      1.523     110.421     -53.844      0.089561
 33  O4     OH     S     32  28  18      1.423     110.285     174.626     -0.658322
 34  HO4    HO     E     33  32  28      0.971     109.429     -60.984      0.420322
 35  H4     H1     E     32  28  18      1.070     108.419     -65.738      0.093684
 36  C5     CT     B     32  28  18      1.527     110.176      53.907      0.091800
 37  H5     H1     E     36  32  28      1.070     105.920      65.453      0.046744
 38  C6     CT     3     36  32  28      1.511     112.896    -175.072      0.374330
 39  H61    H1     E     38  36  32      1.071     109.017      72.367      0.004798
 40  H62    H1     E     38  36  32      1.070     108.403     -48.290      0.004798
 41  O6     OH     S     38  36  32      1.420     111.284    -167.383     -0.718329
 42  HO6    HO     E     41  38  36      0.970     109.488     -60.898      0.442078
 43  C      C      M      6   4   3      1.522     111.100     180.000      0.59730
 44  O      O      E     43   6   4      1.229     120.500      0.000     -0.56790

LOOP
  C5      O5

```

```

IMPROPER
  CB   ND2   CG   OD1
  C8   N2    C7   O7

DONE
STOP

```

From here, your PREPi file is complete, and the next task will be to do some light editing to your PDB that you intend to run MD on.

Editing the PDB for MD

From here, the most difficult task is behind us. However, we still have a bit of work to do. Next, we'll need to edit our PDB file so that our modified residue exists within it as a single, uniquely named amino acid. Usually a PDB downloaded from the Protein Data Bank will have any amino acid modification listed as a separate residue from the amino acid that is being modified. However, for our PREPi parameters to be read correctly, these need to be combined into a single residue. The following displays our Asn residue from our example and the NAG glycan that is modifying it.

```

...
ATOM  1124  N   ASN  A  90    -21.795  43.653  -6.801  1.00  68.31      N
ATOM  1125  CA  ASN  A  90    -21.414  42.298  -6.398  1.00  67.87      C
ATOM  1126  C   ASN  A  90    -21.836  41.281  -7.476  1.00  68.96      C
ATOM  1127  O   ASN  A  90    -23.025  41.160  -7.766  1.00  68.43      O
ATOM  1128  CB  ASN  A  90    -22.021  41.971  -5.011  1.00  63.46      C
ATOM  1129  CG  ASN  A  90    -21.329  40.809  -4.285  1.00  66.69      C
ATOM  1130  OD1 ASN  A  90    -20.731  39.945  -4.926  1.00  70.17      O
ATOM  1131  ND2 ASN  A  90    -21.378  40.828  -2.939  1.00  67.13      N
ATOM  1132  H   ASN  A  90    -22.538  44.091  -6.269  1.00  68.31      H
ATOM  1133  HA  ASN  A  90    -20.325  42.245  -6.319  1.00  67.87      H
ATOM  1134  HB3 ASN  A  90    -23.096  41.805  -5.068  1.00  63.46      H
ATOM  1135  HB2 ASN  A  90    -21.881  42.846  -4.377  1.00  63.46      H
ATOM  1136  HD21 ASN  A  90    -22.225  41.283  -2.638  1.00  67.13      H
ATOM  1137  HD22 ASN  A  90    -20.615  41.439  -2.675  1.00  67.13      H
...
HETATM12542  C1  NAG  A  702    -21.126  39.613  -2.148  1.00  70.25      C
HETATM12543  C2  NAG  A  702    -19.714  39.515  -1.483  1.00  70.70      C
HETATM12544  C3  NAG  A  702    -19.634  38.210  -0.645  1.00  72.39      C
HETATM12545  C4  NAG  A  702    -20.827  38.093   0.338  1.00  73.12      C
HETATM12546  C5  NAG  A  702    -22.171  38.315  -0.406  1.00  72.99      C
HETATM12547  C6  NAG  A  702    -23.366  38.370   0.573  1.00  73.90      C
HETATM12548  C7  NAG  A  702    -18.136  40.846  -2.917  1.00  79.11      C
HETATM12549  C8  NAG  A  702    -16.947  40.773  -3.867  1.00  74.20      C
HETATM12550  N2  NAG  A  702    -18.558  39.687  -2.380  1.00  77.91      N
HETATM12551  O3  NAG  A  702    -18.388  38.096   0.027  1.00  80.19      O
HETATM12552  O4  NAG  A  702    -20.813  36.849   1.019  1.00  71.88      O
HETATM12553  O5  NAG  A  702    -22.103  39.561  -1.109  1.00  72.31      O
HETATM12554  O6  NAG  A  702    -24.591  38.483  -0.125  1.00  68.80      O
HETATM12555  O7  NAG  A  702    -18.670  41.932  -2.686  1.00  77.68      O
HETATM12556  H1  NAG  A  702    -21.280  38.737  -2.785  1.00  70.25      H
HETATM12557  H2  NAG  A  702    -19.629  40.329  -0.758  1.00  70.70      H
HETATM12558  H3  NAG  A  702    -19.666  37.364  -1.335  1.00  72.39      H
HETATM12559  H4  NAG  A  702    -20.722  38.862   1.106  1.00  73.12      H
HETATM12560  H5  NAG  A  702    -22.352  37.504  -1.117  1.00  72.99      H

```

HETATM12561	H61	NAG	A	702	-23.269	39.207	1.266	1.00	73.90	H
HETATM12562	H62	NAG	A	702	-23.410	37.463	1.178	1.00	73.90	H
HETATM12563	H81	NAG	A	702	-16.139	41.409	-3.505	1.00	74.20	H
HETATM12564	H82	NAG	A	702	-16.564	39.755	-3.959	1.00	74.20	H
HETATM12565	H83	NAG	A	702	-17.236	41.120	-4.859	1.00	74.20	H
HETATM12566	HN2	NAG	A	702	-18.114	38.836	-2.695	1.00	77.91	H
HETATM12567	HO3	NAG	A	702	-18.330	37.226	0.427	1.00	80.19	H
HETATM12568	HO4	NAG	A	702	-20.098	36.853	1.661	1.00	71.88	H
HETATM12569	HO6	NAG	A	702	-25.312	38.394	0.504	1.00	68.80	H
...										

Before doing anything, we'll first want to make sure that the atom names in the 3rd column of our PDB match the atoms names in the 2nd column of the PREPi file. Once you've made sure these atom names match, we can start our edits.

Our PREPi file contains parameters for both the Asn and NAG glycans combined. As such, we'll need to combine these two residues into one. First, you'll need to copy all information for the atoms associated with the modified section of your amino acids. In this example, this will be all atoms / lines of NAG 702. You'll then need to paste these atoms directly below the line detailing the last atom of the amino acid that is being modified. After this, you'll have to delete the original NAG 702 atoms / lines to avoid having duplicate copies of the same atoms.

Next, you'll want to change the amino acid number of all atoms associated with the modification to match the number of the residue being modified. In this example, you'd want to use the command `:s/NAG A 702/NAG A 90/g` to change the residue number of all atoms in the NAG 702 residue to NAG 90. From here, you'll want to make sure both the amino acid being modified and the modification have the same 3-letter amino acid code. Earlier on we indicated in our PREPi file that it would describe a modified amino acid with the 3-letter code NAG. As such, we'll want all atoms in this line to have the 3-letter code NAG. We can do this for all atoms using the command `:s/ASN A 90/NAG A 90/g` in our example.

From here, the last thing we'll want to do is make sure that all atoms in our modified amino acid begin with ATOM and not HETATM. Using the same type of command `:s/HETATM/ATOM /g` will perform this change for all of our atoms. Our new PDB should look something like the following example.

...											
ATOM	1124	N	NAG	A	90	-21.795	43.653	-6.801	1.00	68.31	N
ATOM	1125	CA	NAG	A	90	-21.414	42.298	-6.398	1.00	67.87	C
ATOM	1126	C	NAG	A	90	-21.836	41.281	-7.476	1.00	68.96	C
ATOM	1127	O	NAG	A	90	-23.025	41.160	-7.766	1.00	68.43	O
ATOM	1128	CB	NAG	A	90	-22.021	41.971	-5.011	1.00	63.46	C
ATOM	1129	CG	NAG	A	90	-21.329	40.809	-4.285	1.00	66.69	C
ATOM	1130	OD1	NAG	A	90	-20.731	39.945	-4.926	1.00	70.17	O
ATOM	1131	ND2	NAG	A	90	-21.378	40.828	-2.939	1.00	67.13	N
ATOM	1132	H	NAG	A	90	-22.538	44.091	-6.269	1.00	68.31	H
ATOM	1133	HA	NAG	A	90	-20.325	42.245	-6.319	1.00	67.87	H
ATOM	1134	HB3	NAG	A	90	-23.096	41.805	-5.068	1.00	63.46	H
ATOM	1135	HB2	NAG	A	90	-21.881	42.846	-4.377	1.00	63.46	H
ATOM	1136	HD21	NAG	A	90	-22.225	41.283	-2.638	1.00	67.13	H
ATOM	12542	C1	NAG	A	90	-21.126	39.613	-2.148	1.00	70.25	C
ATOM	12543	C2	NAG	A	90	-19.714	39.515	-1.483	1.00	70.70	C
ATOM	12544	C3	NAG	A	90	-19.634	38.210	-0.645	1.00	72.39	C
ATOM	12545	C4	NAG	A	90	-20.827	38.093	0.338	1.00	73.12	C

ATOM	12546	C5	NAG	A	90	-22.171	38.315	-0.406	1.00	72.99	C
ATOM	12547	C6	NAG	A	90	-23.366	38.370	0.573	1.00	73.90	C
ATOM	12548	C7	NAG	A	90	-18.136	40.846	-2.917	1.00	79.11	C
ATOM	12549	C8	NAG	A	90	-16.947	40.773	-3.867	1.00	74.20	C
ATOM	12550	N2	NAG	A	90	-18.558	39.687	-2.380	1.00	77.91	N
ATOM	12551	O3	NAG	A	90	-18.388	38.096	0.027	1.00	80.19	O
ATOM	12552	O4	NAG	A	90	-20.813	36.849	1.019	1.00	71.88	O
ATOM	12553	O5	NAG	A	90	-22.103	39.561	-1.109	1.00	72.31	O
ATOM	12554	O6	NAG	A	90	-24.591	38.483	-0.125	1.00	68.80	O
ATOM	12555	O7	NAG	A	90	-18.670	41.932	-2.686	1.00	77.68	O
ATOM	12556	H1	NAG	A	90	-21.280	38.737	-2.785	1.00	70.25	H
ATOM	12557	H2	NAG	A	90	-19.629	40.329	-0.758	1.00	70.70	H
ATOM	12558	H3	NAG	A	90	-19.666	37.364	-1.335	1.00	72.39	H
ATOM	12559	H4	NAG	A	90	-20.722	38.862	1.106	1.00	73.12	H
ATOM	12560	H5	NAG	A	90	-22.352	37.504	-1.117	1.00	72.99	H
ATOM	12561	H61	NAG	A	90	-23.269	39.207	1.266	1.00	73.90	H
ATOM	12562	H62	NAG	A	90	-23.410	37.463	1.178	1.00	73.90	H
ATOM	12563	H81	NAG	A	90	-16.139	41.409	-3.505	1.00	74.20	H
ATOM	12564	H82	NAG	A	90	-16.564	39.755	-3.959	1.00	74.20	H
ATOM	12565	H83	NAG	A	90	-17.236	41.120	-4.859	1.00	74.20	H
ATOM	12566	HN2	NAG	A	90	-18.114	38.836	-2.695	1.00	77.91	H
ATOM	12567	HO3	NAG	A	90	-18.330	37.226	0.427	1.00	80.19	H
ATOM	12568	HO4	NAG	A	90	-20.098	36.853	1.661	1.00	71.88	H
ATOM	12569	HO6	NAG	A	90	-25.312	38.394	0.504	1.00	68.80	H
...											

From here, your PDB should be complete! You'll note that the atom numbers in the 2nd column are not in order. This is not a major problem at all, and it will not cause any issues. *tleap* will renumber these atoms when creating the prmtop and inpcrd files.

You can now try and run *tleap* and see if it completes successfully. Your input file will not look too different from the basic example explained in the [tleap section of this protocol](#). If *tleap* runs to completion without any note of missing parameters, you'll be good to run MD.

Addressing Missing Parameters

As mentioned above, there are some cases where parameters for an equivalent atom type from GAFF will not be present in ff14SB. There are also cases where even if an equivalent atom type does exist, there still may be angle or torsional parameters that are missing. In this event, it will be necessary to manually create an frcmod file adding in these parameters. The format of an frcmod file is detailed [here](#).

In the event that a particular set of angles, torsions, or improper torsions are missing for existing ff14SB atom types, you'll need copy parameters for equivalent atom types from GAFF and change the atom types to match their equivalents in ff14SB. For example, if the angle OS-CT-O happened to be missing from ff14SB, you would need to copy the GAFF parameters for os-c3-o into an frcmod file. These values would need to be assigned to the atom types OS-CT-O for proper incorporation into ff14SB.

In the event that a specific atom type is missing, you'll need to create an frcmod file and also manually define the atom type in a *tleap* input file as described [here](#).

Creating initial Amber topology and coordinate files

When you run *tLeap* you will enter a new prompt on the command line that is for *tLeap* commands instead of regular unix commands. You can run *tLeap* using the following command.

```
$AMBERHOME/bin/tLeap
```

tLeap will then inform you of directories it has added to the search path and will tell you welcome (*tLeap* is so polite). Now we need to load the force fields we want to use. The first force field we'll load in will parameterize our protein. I suggest using the Amber ff14SB force field for this as it is the most recent Amber force field, but many in the lab like to use the Amber ff99SB force field because it is also released with Desmond. To load the ff14SB protein force field, type

```
source leaprc.protein.ff14SB
```

There are several force fields available within Amber, but you must explicitly load the force fields you want to use. A list of the available force fields can be found in the Amber source code in the following directory

```
$AMBERHOME/dat/leap/cmd/
```

To load any of these force fields while you are in *tLeap* you can just type

```
source leaprc.force_field_name
```

where *force_field_name* is the exact name of the force field you want to use (for example, 'source leaprc.ff14SB' shown earlier to load the Amber ff14SB force field). Some important available force fields are listed below, along with the type of molecules/simulations they can be used to parameterize.

leaprc.ff14SB	Amino acids, nucleic acids, ions, and solvents
leaprc.ff99SB	Older parameters for amino acids, nucleic acids, ions, and solvents
leaprc.gaff	General organic molecules
leaprc.GLYCAM_06h-12SB	Carbohydrates
leaprc.lipid11	Lipids
leaprc.constph	Used for constant pH simulations

If you are using the Amber ff14SB force field, then you will also need to separately load in the parameters for the ions and water. If you are going to use TIP3P water, that command would look like this.

```
source leaprc.water.tip3p
```

Next, load in the force field parameters for your ligand. Most ligands can be characterized using the General Amber Force Field for organic molecules (GAFF). To load this force field in, use the following command.

```
source leaprc.gaff
```

Next, load in the parameters for TIP3P ions. The command should look like this.

```
loadamberparams frcmod.ionsjc_tip3p
```

If you have a ligand, and that ligand is not contained in the original PDB file, then you will need to load in the coordinates and charges from the mol2 file you created earlier.

```
LIG=loadmol2 ligand_with_H.mol2
```

Here ‘LIG’ will be the residue name attributed to your ligand from now on. If you want to make this something more appropriate, go ahead. For example, if your ligand is dimethyl ketone, you may want to call it DMK instead of LIG. You also need to load in the frcmod file for your ligand.

```
loadamberparams ligand_with_H.frcmod
```

Now we will actually load in your biomolecule PDB. Make sure you have cleaned up the PDB and gotten it ready to be read before loading it into PDB. The PDB file should just contain the atomic coordinate lines. Go ahead and remove all the information added by the X-ray crystallographer at the beginning of the file. This will make it less likely for *tleap* to have problems reading the file format. Also, if there were any mutations made before the structure was crystallized, you will need to change those prior to this step. Amber can add missing atoms if necessary, but all the residue names must be correct prior to reading in the PDB.

```
Mol=loadpdb 1L3S.pdb
```

I have used 1L3S.pdb as an example, but obviously replace it with your PDB code or whatever you have named your PDB. In this case, ‘Mol’ is just a placeholder for *tleap* and will not be used by Amber after you close *tleap*, so once again you could call it something different if you prefer. By default, Amber does not cap the termini of a protein chain, so if the first amino acid of your protein PDB is a Lysine, the first residue will still be a Lysine, but with a positively charged –NH₃⁺ backbone group at the terminal end (similarly, your C-terminal residue would not change except the terminal end would be a –COO⁻ group). Sometimes, MD users prefer to “cap” the termini of their proteins to avoid the presence of charged groups at the ends of the protein sequence. Amber will not do this explicitly, but Amber does have two capped residues present in the Amber force fields that can be used for this situation. You can use other software (such as Maestro) or add these by hand (which is more complicated and only suggested for more advanced users) to cap your protein’s termini and resave your protein coordinates as a new PDB file with the capped residues that you load into Amber’s *tleap*. To be read successfully by *tleap*, the capped residue at the beginning (N-terminal end) of the protein should be called NME, while the capped residue at the end (C-terminal) should be called ACE. You can check these in your PDB file using `vi`, and change them if they are named incorrectly.

If the coordinates for your ligand were not in the biomolecule PDB, then you will need to combine the two within *tleap* before we can save any files.

```
com=combine{ Mol LIG }
```

This creates a new *tleap* object that first contains all the information for the biomolecule, followed by all the information for the ligand. I like to save a ‘dry’ topology file of my system prior to adding the ions and solvent. This can be very helpful later when running analysis, but also gives you a way to easily look at the solute without having to visualize any of the solvent later. To save a dry topology file of your ligand only type

```
saveamberparm LIG ligand_with_H.prmtop ligand_with_H.inpcrd
```

You can also save Amber files of just your biomolecule

```
saveamberparm Mol 1L3S.prmtop 1L3S.inpcrd
```

and your complex

```
saveamberparm com 1L3S_with_ligand.prmtop 1L3S_with_ligand.inpcrd
```

After you have saved these files, it is time to add the ions. Typically in MD simulations, unless we are doing a study to determine the effect of salt concentration on a system, we usually just neutralize the system with counterions instead of setting the system to a particular concentration. Using Amber it is not straightforward to set your system to a certain concentration because Amber will only let you add a certain number of ions directly. You must add ions and solvate to manually determine the salt concentration based on the number of ions you added and the total volume of the unit cell. Setting a system to a particular concentration using Amber is most easily done using the `addIonsRand` command after solvating (see below). You should have noticed when you saved the `prmtop/inpcrd` files earlier that you got a warning about the charge not being zero. For example:

```
WARNING: The unperturbed charge of the unit: -35.999999 is not zero.
```

This tells you the overall charge of the system currently (hopefully very close to an integer). In the above example, we would need to add 36 Na^+ ions to neutralize the system (`com`). This can be done by typing

```
addIons com Na+ 0
```

Placing the zero at the end tells *tleap* to neutralize the system, which hopefully means it will add 36 Na^+ ions. I have found that sometimes this doesn’t work if your overall charge is slightly off from being a whole integer, so alternatively you could have told it to add 36 Na^+ ions directly by typing

```
addIons com Na+ 36
```

If you also want a certain number of Cl^- ions, you could type

```
addIons com Cl- 14
```

to add 14 chloride ions to your complex. I also like to save a prmtop/inpcrd file of the system with these new ions for possible future use. To do this, type

```
saveamberparm com 1L3S_ligand_ions.prmtop 1L3S_ligand_ions.inpcrd
```

After adding counterions, we next want to solvate the system with a bunch of water molecules in a unit cell. To solvate the complex with TIP3 water molecules in a truncated octahedron with a minimum of 12 Å buffer region between the solute and the edge of the unit cell, type

```
solvateOct com TIP3PBOX 12.0
```

Amber can also use a cuboid box if instead you replaced `solvateOct` with `solvateBox`. The minimum buffer region should not be below 10.0 Å. I used 12.0 Å in this example because it is my preference. At this point, if you have not already added ions, you can do so using the `addIonsRand` command. The syntax for `addIonsRand` is the same as the `addIons` command, the difference is that the `addIonsRand` command will add ions randomly to your system and replace a water molecule for every ion that is added. So if you just wanted neutralize the system with Na^+ ions at this point, you could type

```
addIonsRand com Na+ 0
```

And this will neutralize a previously negatively charged system. However, this command is most ideal if you want to add a certain concentration of ions to your system. If you noticed, after you solvated (`solvateOct com TIP3PBOX 12.0`), there should have been several lines printed to the screen, one of which detailed the total volume of the cell in units of Å³. For example,

```
Center of solvent box is: -56.323047, -56.323047, -56.323047
Volume: 867992.836 Å^3 (oct)
Total mass 475497.521 amu, Density 0.910 g/cc
Added 22350 residues.
```

In this case, the volume is 867992.836 Å³. Let's say we want to set this simulation to run under conditions of 0.150 M Na^+ concentration (common among simulations trying to simulate *in vivo* or *in vitro* conditions), then we could do the following conversion

$$\frac{0.150 \text{ mol Na}^+}{1 \text{ L}} \times \frac{1 \text{ L}}{1 \times 10^{27} \text{ Å}^3} \times 867992.836 \text{ Å}^3 \times \frac{6.022 \times 10^{23} \text{ Na}^+ \text{ atoms}}{1 \text{ mol Na}^+} = 78.4 \text{ Na}^+ \text{ atoms}$$

Obviously, you are not likely to get exactly the concentration you desire, but in this case you could add 78 or 79 Na^+ atoms to obtain a concentration slightly below or above 150 mM Na^+ , respectively. To do this, use the `addIonsRand` command.

```
addIonsRand com Na+ 78
```

Of course, by doing this you still have a net overall charge for your unit cell that you will need to neutralize using an ion of opposite charge. In this case, that ion would likely be chloride, as shown below

```
addIonsRand com Cl- 0
```

Once we have neutralized and solvated our complex to our satisfaction, we can save the solvated topology (prmtop) and coordinate (inpcrd) files.

```
saveamberparm com 1L3S_ligand_solv.prmtop 1L3S_ligand_solv.inpcrd
```

This should have saved the files we wanted, so we can now exit out of *tleap* by typing

```
quit
```

While running *tleap* always keep an eye on the messages being printed to the screen. This is where any error messages or warnings will be displayed, and if you don't catch them then your prmtop/inpcrd files may not be created, or worse your files will be created incorrectly.

This process is very helpful for beginners or if you are making a system for the very first time, but for a more advanced approach it is helpful to know that all of these commands could have been put into a file (called, for example, *leap.in*) that looks like this:

```
source leaprc.protein.ff14SB
source leaprc.water.tip3p
source leaprc.gaff
loadamberparams frcmod.ionsjc_tip3p
LIG=loadmol2 ligand_with_H.mol2
loadamberparams ligand_with_H.frcmod
Mol=loadpdb 1L3S.pdb
com=combine{ Mol LIG }
saveamberparm LIG ligand_with_H.prmtop ligand_with_H.inpcrd
saveamberparm Mol 1L3S.prmtop 1L3S.inpcrd
saveamberparm com 1L3S_with_ligand.prmtop 1L3S_with_ligand.inpcrd
addIons com Na+ 0
addIons com Cl- 0
saveamberparm com 1L3S_ligand_ions.prmtop 1L3S_ligand_ions.inpcrd
solvateOct com TIP3PBOX 12.0
saveamberparm com 1L3S_ligand_solv.prmtop 1L3S_ligand_solv.inpcrd
```

And then you can have *tleap* directly read and execute all of the commands and exit once it is done using the following command

```
$AMBERHOME/bin/tleap -f leap.in
```

Where *leap.in* is the name of the file with all the *tleap* commands. Note that you do not need to have 'quit' in the *leap.in* file because *tleap* will automatically quit once it reaches the end of the file. This approach should be used with CAUTION as it becomes more difficult to observe any problems *tleap* might have if all these steps are done using command with a *tleap* input file.

After creating your Amber topology and coordinate files, it is important to visually check these files to ensure that there are no errors with your structure before beginning MD. These errors can range from an improperly placed ligand to incorrect atomic bonds. You can check for errors by using *cpptraj* to essentially reverse engineer a PDB file from the Amber topology and coordinate files you just created. *cpptraj* is part of the AmberTools software suite and is the primary way that MD data is analyzed after simulations have been completed. This program is covered more extensively beginning in the *cpptraj* section of this protocol. The PDB that *cpptraj* will create will be representative of the Amber topology and coordinate files you will be using for running MD.

The first task of running this *cpptraj* job is to load in your parameter file. In this case, your parameter file is your Amber topology file. You can load this file into *cpptraj* by using the following command.

```
parm 1L3S_ligand_solv.prmtop
```

Next, you will need to load in your trajectory file into *cpptraj*. In this case, your trajectory file is your Amber coordinate file. You can load this file into *cpptraj* by using the following command.

```
trajin 1L3S_ligand_solv.inpcrd
```

After loading in your parameter and trajectory file, you'll need to tell *cpptraj* what job it should perform. In this case, we want it to perform an *autoimage* job. The command *autoimage* tells *cpptraj* to create a new file from the Amber topology and coordinate files that have been previously loaded in. While it is likely self-explanatory, the *autoimage* command is as follows.

```
autoimage
```

Next, you have some options. If you want to view only the protein-ligand complex, you will want to add a command that strips both the ions and the water molecules from your input files. This makes viewing of just the protein-ligand complex easier and is generally the best option for making publication-quality images of your structure. If you want to strip the ions and water molecules, you can add the following command.

```
strip :WAT:Cl-:Na+
```

This command tells *cpptraj* to remove all atoms corresponding to the residue names of WAT, Cl-, and Na+ respectively. If you want to view your Amber topology and coordinate files exactly as they are produced from *tLeap*, do not include this line.

Next you'll want to create your PDB. This can be done by adding in the following command.

```
trajout 1L3S_ligand_0ns.pdb
```

For the sake of simplicity, all of these commands should be combined into a single input file in the same way this has been done with *tLeap*. This file will be called *cpptraj_inpcrd.in* for example.

```
parm 1L3S_ligand_solv.prmtop
```

```

trajin 1L3S_ligand_solv.inpcrd
autoimage
strip :WAT:Cl-:Na+
trajout 13LS_ligand_0ns.pdb
go

```

As with *tleap*, you can then have *cpptraj* directly read in your input file with the following command.

```
$AMBERHOME/bin/cpptraj -i cpptraj_inpcrd.in
```

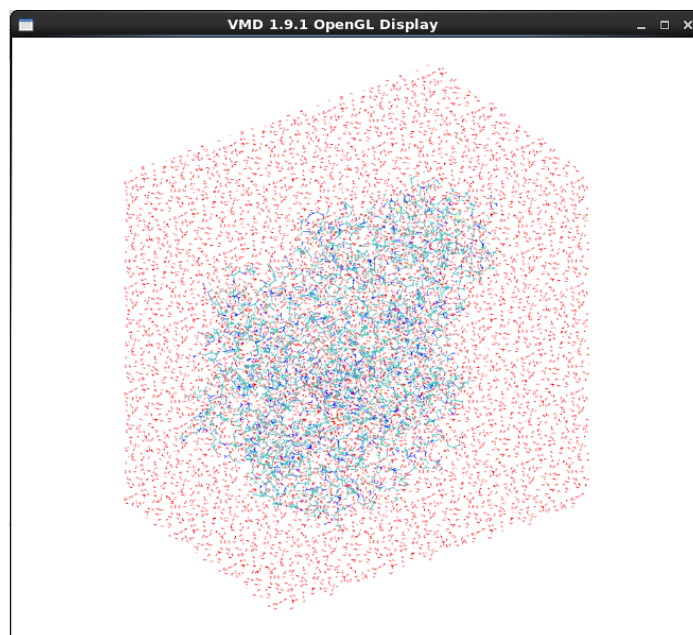
From here, you're ready to check your Amber topology and coordinate files for errors. Depending on which program you prefer, you can load the file into either Chimera or VMD (I prefer Chimera). This can be done easily with the following commands.

Chimera: `chimera 13LS_ligand_0ns.pdb`

VMD: `vmd 13LS_ligand_0ns.pdb`

(Note that if Chimera doesn't work, this is likely due to the fact that you are missing the proper lines to load Chimera in your workstation. This can be corrected by adding the line set `path=($path /usr/local/chimera/chimera-1/8/bin)` to your `.cshrc` file. Information on how to do this is located [here](#))

If you have decided to keep the ions and water molecules, your display should look something like this. While this image is from VMD, it will also look similar in Chimera.



This shows your solute (blue and green lines in the middle) surrounded by all the water molecules (red and white lines) added by *tleap* when you solvated.

Lastly, this entire process (from basic ligand preparation to checking your prmtop and inpcrd files in Chimera) can be streamlined significantly by combining everything into a single script. Scripts are useful as they are easy to copy and edit for applications to different systems. You can create a C-shell script as follows.

```
#!/bin/csh
module load amber/18
```

The first line tells Linux to run this script in a C-shell environment. The second line tells it to load in the program files for Amber18.

When using scripts, it is necessary to tell Linux to create your input files. This can be done by using the following commands.

```
rm > leap.in
cat > leap.in << EOF
...
EOF
```

The first line in tells Linux to remove any files named `leap.in`. These files should not exist prior to running the script. This line will also delete any old version of the desired input file should you change your script and run it another time later on. The second line uses a `cat` command. This command is highly versatile and has many functions. In this case, we are telling Linux to print all of the lines that follow this `cat` command into a file called `leap.in` until the flag `EOF` is reached. The “...” represents the lines of your input file, and `EOF` represents the flag telling Linux to stop printing lines into `leap.in`. This set of commands is very powerful and is essential for all scripts you will write in the future.

A full example script (called `ligprep.sh`) is as follows.

```
#!/bin/csh
module load amber/18

# If the ligand contains no Hatoms, run reduce to neutralize the ligand.
$AMBERHOME/bin/reduce ligand.pdb > ligand_w_H.pdb

# Run antechamber on the neutralized ligand to generate mol2 files.
$AMBERHOME/bin/antechamber -i ligand_w_H.pdb -fi pdb -o ligand_w_H.mol2 -fo
mol2 -c bcc -s 2 -nc 0 -m 1

# Run parmchk to generate a force field modification specific to your ligand.
$AMBERHOME/bin/parmchk2 -i ligand_w_H.mol2 -f mol2 -o ligand_w_H.frcmod

# Create leap.in file
rm leap.in
cat > leap.in << EOF
source leaprc.protein.ff14SB
source leaprc.water.tip3p
source leaprc.gaff
```

```

loadamberparams frcmod.ionsjc_tip3p
LIG=loadmol2 ligand_with_H.mol2
loadamberparams ligand_with_H.frcmod
Mol=loadpdb 1L3S.pdb
com=combine{ Mol LIG }
saveamberparm LIG ligand_with_H.prmtop ligand_with_H.inpcrd
saveamberparm Mol 1L3S.prmtop 1L3S.inpcrd
saveamberparm com 1L3S_with_ligand.prmtop 1L3S_with_ligand.inpcrd
addIons com Na+ 0
addIons com Cl- 0
saveamberparm com 1L3S_ligand_ions.prmtop 1L3S_ligand_ions.inpcrd
solvateOct com TIP3PBOX 12.0
saveamberparm com 1L3S_ligand_solv.prmtop 1L3S_ligand_solv.inpcrd
quit
EOF

# Run leap.in file
$AMBERHOME/bin/tleap -f leap.in

# Generate a 0ns PDB of the simulation you're about to run. This is necessary
to make sure tLeap ran properly. Always visualize your structure in Chimera
or VMD before running MD.
rm cpptraj_inpcrd.in
cat > cpptraj_inpcrd.in << EOF

parm 1L3S_ligand_solv.prmtop
trajin 1L3S_ligand_solv.inpcrd

autoimage
strip :WAT:Cl-:Na+
trajout 1L3S_ligand_0ns.pdb
go
EOF

# Run cpptraj_inpcrd.in file
$AMBERHOME/bin/cpptraj -i cpptraj_inpcrd.in

```

Good job if you have gotten this far. That was probably the toughest part of this whole process.

Other *tleap* Example Files

Note that the *tleap* input file example shown above is not the only way *tleap* is used. It is simply the most common use case where a simulation is run on a protein-ligand system. The following is what your *tleap* input file would look like in the event that you don't have a ligand.

```

source leaprc.protein.ff14SB
source leaprc.water.tip3p
loadamberparams frcmod.ionsjc_tip3p
Mol=loadpdb 1L3S.pdb
saveamberparm Mol 1L3S.prmtop 1L3S.inpcrd
addIons Mol Na+ 0
addIons Mol Cl- 0
saveamberparm Mol 1L3S_ligand_ions.prmtop 1L3S_ligand_ions.inpcrd

```

```
solvateOct Mol TIP3PBOX 12.0
saveamberparm Mol 1L3S_ligand_solv.prmtop 1L3S_ligand_solv.inpcrd
```

The *tleap* input files for apo systems are much more simple. Since there is no ligand, there is no need to load in a ligand mol2 file or a ligand frcmod file. You also don't need to source in the GAFF force field. There is also no need to combine the ligand and protein into a single system. Therefore, these lines are omitted.

This next example is for when you do have a ligand, but that ligand itself is a peptide. As mol2 files are only required for non-peptide molecules, there will be no need to load in a mol2 file for your ligand (and subsequently no need to run *reduce*, *antechamber*, or *parmchk*). You will instead load in a PDB file of your ligand.

```
source leaprc.protein.ff14SB
source leaprc.water.tip3p
loadamberparams frcmod.ionsjc_tip3p
PEP=loadpdb Peptide_5mer_fixed_pose88_reorient.pdb
Mol=loadpdb 3bcz_chainA.pdb
com=combine{ PEP Mol }
saveamberparm PEP Peptide_5mer_fixed_pose88_reorient.prmtop
Peptide_5mer_fixed_pose88_reorient.inpcrd
saveamberparm Mol 3bcz_chainA.prmtop 3bcz_chainA.inpcrd
saveamberparm com 3bcz_chainA_5mer_reoriented.prmtop
3bcz_chainA_5mer_reoriented.inpcrd
addIons com Na+ 0
addIons com Cl- 0
saveamberparm com 3bcz_chainA_5mer_reoriented_ions.prmtop
3bcz_chainA_5mer_reoriented_ions.inpcrd
solvateOct com TIP3PBOX 12.0
saveamberparm com 3bcz_chainA_5mer_reoriented_solv.prmtop
3bcz_chainA_5mer_reoriented_solv.inpcrd
```

As mentioned, in this case your ligand is a peptide. The line `PEP=loadpdb Peptide_5mer_fixed_pose88_reorient.pdb` loads in this peptide ligand. Note that PEP is just a flag name. You can call it whatever you want, as long as it is consistent. If you wanted to call it `LIG` like we did with our non-peptide ligands, you can do that as well, as long as the name is consistent throughout the rest of the input file.

In the event that you are running MD on a system with an atomType not normally present in GAFF (for example, let's say you are running MD on a system with a ligand that contains sp^3 Si atoms) you will need to manually specify that new atomType in your *tleap* input file. This can be done like in the following example.

```
source leaprc.protein.ff14SB
source leaprc.water.tip3p
source leaprc.gaff
loadamberparams frcmod.ionsjc_tip3p
loadamberparams frcmod.silicon2

addAtomTypes {
  { "Si"    "Si" "sp3" }
}
```

```

}

LIG=loadmol2 Ligand_w_Si.mol2
Mol=loadpdb Receptor.pdb
com=combine { Mol LIG }
saveamberparm LIG Ligand_w_Si.prmtop Ligand_w_Si.inpcrd
saveamberparm Mol Receptor.prmtop Receptor.inpcrd
saveamberparm com Receptor_Ligand_w_Si.prmtop Receptor_Ligand_w_Si.inpcrd
addIons com Na+ 0
addIons com Cl- 0
saveamberparm com Receptor_Ligand_w_Si_ions.prmtop
Receptor_Ligand_w_Si_ions.inpcrd
solvateOct com TIP3PBOX 12.0
saveamberparm com Receptor_Ligand_w_Si_solv.prmtop
Receptor_Ligand_w_Si_solv.inpcrd

```

Normally, sourcing in `leaprc.gaff` takes care of adding `atomTypes` for us. This is because the file `leaprc.gaff` consists of a long list of `atomTypes` for *tleap* to prepare, and then directs *tleap* to load in the file `gaff.dat` which contains the force field parameters characterizing all of these `atomTypes`.

In this case, we do not have a file that takes care of both of these tasks for us. In this example, first the parameter file for these Si atoms is loaded in using the line `loadamberparams frcmod.silicon2`. This file contains force field parameters characterizing sp^3 Si atoms. In order for *tleap* to load it properly, this file needs to be present in the folder you are running *tleap* in. Next, we tell *tleap* to prepare the `atomType` Si, which is a silicon atom that is sp^3 hybridized. The rest of this file is identical to normal *tleap* files.

A more common example of when you would need to manually add `atomTypes` to *tleap* is when dealing with phosphorylated amino acids. The following example does just that.

```

source leaprc.protein.ff14SB
source leaprc.water.tip3p
source leaprc.phosaa10

addAtomTypes {
  { "OP"  "O"  "sp2" }
  { "OQ"  "O"  "sp3" }
  { "OR"  "O"  "sp3" }
  { "OT"  "O"  "sp2" }
  { "OV"  "O"  "sp3" }
  { "OX"  "O"  "sp2" }
  { "OZ"  "O"  "sp3" }
}

phosaa = loadamberparams frcmod.phosaa10
loadoff phos_amino94.lib

loadamberparams frcmod.ionsjc_tip3p
PEP=loadpdb Peptide_5mer_fixed_pose88_reorient.pdb
Mol=loadpdb 3bcz_chainA.pdb
com=combine{ PEP MEM }
saveamberparm PEP Peptide_5mer_fixed_pose88_reorient.prmtop
Peptide_5mer_fixed_pose88_reorient.inpcrd

```

```

saveamberparm Mol 3bcz_chainA.prmtop 3bcz_chainA.inpcrd
saveamberparm com 3bcz_chainA_5mer_reoriented.prmtop
3bcz_chainA_5mer_reoriented.inpcrd
addIons com Na+ 0
addIons com Cl- 0
saveamberparm com 3bcz_chainA_5mer_reoriented_ions.prmtop
3bcz_chainA_5mer_reoriented_ions.inpcrd
solvateOct com TIP3PBOX 12.0
saveamberparm com 3bcz_chainA_5mer_reoriented_solv.prmtop
3bcz_chainA_5mer_reoriented_solv.inpcrd

```

This is a more advanced *tleap* input file example. Note that in this case, our ligand is a peptide.

Like in the example with Si, first we need to load in the parameters for our phosphorylated amino acids. First, we tell *tleap* to prepare several new oxygen atomTypes. For example, the first line tells *tleap* to add the atomType OP, which is an oxygen atom that is sp² hybridized. These new atomTypes correspond to the oxygen atoms on phosphate groups in phosphorylated amino acids. Next, the line `phosaa = loadamberparams frcmod.phosaa10` loads in the parameter data for these oxygen atoms. Finally, the line `loadoff phos_amino94.lib` tells *tleap* to load in a library of PREP files describing new phosphorylated amino acids.

There are three things to note here. First, the order in which atomTypes are added and parameter data is loaded is different from the example where Si atoms were added. Second, the line to load in the parameter data looks different than the line that was used in the Si example (specifically the `phosaa = part`). Third, the `loadoff phos_amino94.lib` is not present in the Si example.

First, it should be noted that the order in which atomTypes are added and parameter data is loaded in doesn't matter. It could be done either way in both cases, and it would still work correctly.

The second difference is because the parameters for phosphorylated ligands are actually built into Amber's program files, whereas the Si parameters are not (hence why we use the line `source leaprc.phosaa10` to load in the phosphorylated amino acid parameters as well). Whereas the Si parameter file `frcmod.silicon2` has to be located in the directory you're running *tleap* in, `frcmod.phosaa10` does not need to be because it is natively located within the Amber program files. Thus, the `phosaa = part` of the command `phosaa = frcmod.phosaa10` simply gives *tleap* a folder to look for this parameter file in.

The last difference is due to the fact that we are not only adding new atomTypes, but also new amino acid types. As such, we also need to load in the files describing these new phosphorylated amino acids. That is what the command `loadoff phos_amino94.lib` does. From here, the rest is a pretty standard *tleap* input file.

The following example is for a very complex system, intended to demonstrate all that is possible with *tleap*.

```

source leaprc.protein.ff14SB
source leaprc.water.tip3p
source leaprc.gaff
loadamberparams frcmod.ionsjc_tip3p
loadamberparams frcmod.magnesium

```

```

loadamberparams frcmod.phos_gaff

addAtomTypes {
    { "o3"  "O"  "sp3" }
}

loadamberprep magnesium.prep
loadamberprep ATP.prepi
ATP=loadpdb ATP.pdb
Mg2=loadpdb Mg.pdb
MYO=loadpdb MYO19_1w7j_WT.pdb
MAT=combine { MYO ATP }
MMG=combine { MAT Mg2 }
saveamberparm ATP ATP.prmtop ATP.inpcrd
saveamberparm MYO MYO19_1w7j_WT.prmtop MYO19_1w7j_WT.inpcrd
saveamberparm Mg2 Mg.prmtop Mg.inpcrd
saveamberparm MAT MYO19_1w7j_WT_ATP_Pose1.prmtop
MYO19_1w7j_WT_ATP_Pose1.inpcrd
saveamberparm MMG MYO19_1w7j_WT_Mg_ATP_Pose1.prmtop
MYO19_1w7j_WT_Mg_ATP_Pose1.inpcrd
addIons MMG Na+ 0
addIons MMG Cl- 0
saveamberparm MMG MYO19_1w7j_WT_Mg_ATP_Pose1_ions.prmtop
MYO19_1w7j_WT_Mg_ATP_Pose1_ions.inpcrd
solvateOct MMG TIP3PBOX 12.0
saveamberparm MMG MYO19_1w7j_WT_Mg_ATP_Pose1_solv.prmtop
MYO19_1w7j_WT_Mg_ATP_Pose1_solv.inpcrd

```

This input file includes the manual addition of new atomTypes, multiple combine terms, and makes use of PREP files. As mentioned previously, PREP files describe new amino acids to add into *tleap*. However, as you can tell from this, ATP and Mg²⁺ are very clearly not amino acids. This is because the technical function of the PREP file is to describe new amino acids to add into *tleap*, but in reality they can be used to describe the atoms, bonds, and charges of any molecule. This entirely negates the need to use a mol2 file at all. Note that none of these parameter files or PREP files are natively built into Amber, and as such these files need to be located in the directory in which you run *tleap*.

In this example, first parameters for Mg²⁺ ions are loaded into *tleap* using the line `loadamberparams frcmod.magnesium`. This is followed by loading GAFF modified polyphosphate parameters for ATP/ADP using the line `loadamberparams frcmod.phos_gaff`. Next, the atomType o3 is added. This is an sp³ hybridized oxygen atom with unique behavior due to its bond to the terminal phosphate in a molecule containing a polyphosphate chain (i.e. ATP/ADP). Note that the parameters describing this atomType are located in the file `frcmod.phos_gaff`. Next, the PREP file describing the charge of the Mg²⁺ ion is loaded in using the line `loadamberprep magnesium.prep`. This is followed by the PREPi file (just a newer version of the PREP file format) describing the partial charges and geometric construction of ATP using the line `loadamberprep ATP.prepi`. The PDB containing ATP is then loaded in using the line `ATP=loadpdb ATP.pdb` (as noted before ATP is just a flag, you can call it whatever you want as long as you use the name consistently). The PDB containing the Mg²⁺ ion is loaded in after this using the line `Mg2=loadpdb Mg.pdb`. From here, you load in your protein (the flag MYO is used in this example, and to reiterate again, this is just a flag and it doesn't matter whether you call this MYO, Mol, or bro; you can give it any three-letter name you want

as long as it is used consistently). Next the protein and ATP ligand are combined (this combination is named MAT). This combination is then further combined with the Mg^{2+} ion (this combination is named MMG).

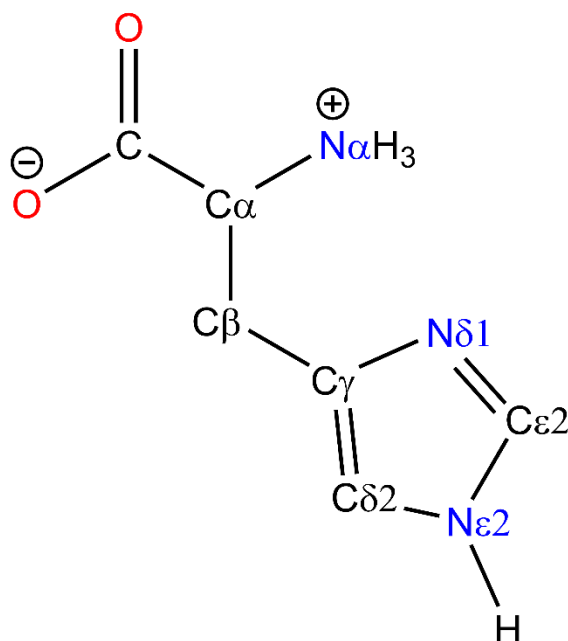
Note that in the event that you have an additional ion (for example Mg^{2+} or Ca^{2+}), the ion must always be combined last. This is to prevent future errors in MMPBSA/MMGBSA energetics calculations after your simulation has finished.

The rest is identical to a normal *tleap* file. Initial prmtop and inpcrd files are saved, then counterions and solvent are added and their respective prmtop and inpcrd files are also saved.

As mentioned before, this is a very complex example. In most cases, you'll never need to create a *tleap* file like this. This is mostly included in this manual to demonstrate the full range of what is possible with *tleap*. Note that all of the files not built into Amber natively (frcmod and PREP files) can be located at the following link. You can find parameters for many nonstandard ions, solvents, ligands, and amino acids here: <http://research.bmh.manchester.ac.uk/bryce/amber/>

A Note on Histidine Residues

The amino acid Histidine (3-letter code His) can exist in three different protonation states. The His side chain is an aromatic imidazole ring containing two nitrogen atoms. The protonation state of His refers to which of the nitrogen atoms in this imidazole ring happens to be bound to a hydrogen atom. These nitrogen atoms are named depending on their location in the ring. The nitrogen closest to carbon atom directly attached to the amino acid backbone is called the delta (δ) nitrogen, whereas the nitrogen furthest from the carbon atom directly attached to the backbone is called the epsilon (ϵ) nitrogen. The following diagram shows the amino acid His and the names of its atoms.

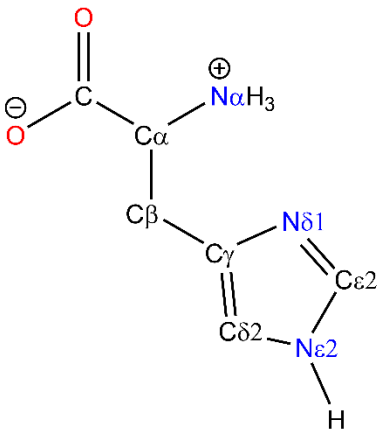
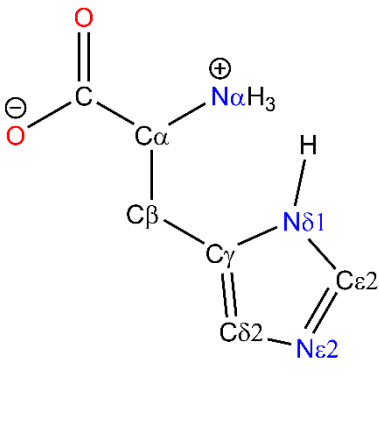
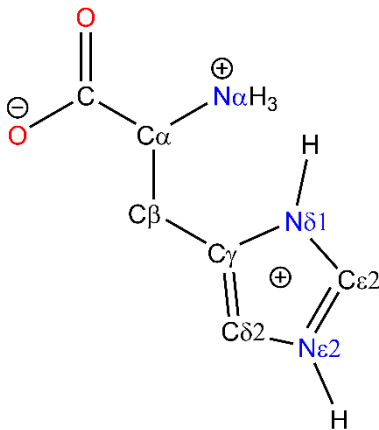


Note that the δ nitrogen is directly bound to the gamma (γ) carbon. This γ carbon is closest to the amino acid backbone. Also note that the ϵ nitrogen is not bound to this carbon. This is how one can differentiate between the two nitrogen atoms. The δ nitrogen is bound to the γ carbon and

thus closer to the backbone, whereas the ϵ nitrogen is not bound to the γ carbon and is thus further from the backbone.

The three protonation states are commonly referred to in the literature as the Tau (τ), Pi (π), and biprotonated states. The τ state occurs when only the ϵ nitrogen is protonated. The π state occurs when only the δ nitrogen is protonated. Lastly, the biprotonated state occurs when both nitrogens are protonated. In the biprotonated state, His is positively charged.

Amber does not use the standard HIS 3-letter code for His, and instead uses unique 3-letter codes for each of three protonation states. The τ state is abbreviated as HIE (because the ϵ nitrogen is protonated), the π state is abbreviated HID (because the δ nitrogen is protonated, and the biprotonated state is abbreviated HIP (because it has a positive charge). The following table displays the three protonation states.

τ Histidine (HIE)	π Histidine (HID)	Biprotonated Histidine (HIP)
		

Since Amber does not use the HIS 3-letter code, when *tleap* reads the HIS code in a PDB, it must convert the 3-letter code to an Amber accepted format. By default, *tleap* will convert all His residues to the HIE protonation state. This conversion process generally causes error messages. There are two approaches for solving errors related to HIS $\rightarrow$ HIE conversion.

The easiest solution is to simply open your PDB and manually delete every hydrogen atom bound to all His residues in your protein. This ensures you will not see any error messages when *tleap* converts HIS to HIE. However, this method should only be used when the protonation state of His residues is not important to the function of the protein. In the even that His protonation states are important, using this method will result in a simulation that is not true to reality.

If His protonation states are important, before running *tleap* it will be necessary for you to open your PDB and manually change the 3-letter code of all of your His residues to the Amber 3-letter code corresponding to the desired protonation state. As detailed above, these codes are HIE, HID, and HIP. Doing this ensures that *tleap* will accurately parameterize specific protonation states of His residues.

Running an Amber Job

CPU vs GPU

Amber has three main programs (executable files) for running MD: *sander*, *pmemd*, and *pmemd.cuda*. And each of these can be run in parallel across multiple processing units using their respective MPI versions: *sander.MPI*, *pmemd.MPI*, and *pmemd.cuda.MPI*. *sander* was the original MD engine written for Amber and it can do everything. Any type of MD you want to run (that Amber supports) can be run using *sander* (or *sander.MPI* if run on multiple CPUs). *pmemd* (specifically *pmemd.MPI*) was developed as a much faster version of *sander* but it had certain limitations because it cannot do everything *sander* can do. If you need to run MD on central processing units (CPUs) and *pmemd* can do it, always choose it over *sander*. But compared to other MD programs (such as NAMD or Desmond), Amber is slow (for several reasons we won't get into here) on CPUs for running regular unrestrained MD. As computational hardware and software developed, some nerds realized that graphical processing units (GPUs) that are used to run the graphics to our computer monitors are ideally suited for doing the calculations during molecular dynamics simulations. Thus, Amber developed and released *pmemd.cuda* that will run MD on GPUs instead of the traditional CPUs. Again, *pmemd.cuda* cannot do everything that *pmemd* can do, but it can do a lot of things, and for our purposes it can run regular unrestrained MD very quickly. Thus, *pmemd.cuda* and running Amber on GPUs will be the focus of this protocol. But you should be aware that *pmemd.cuda* does not support all possible options for MD simulations, and in those cases (such as mixed QM/MM) you should use either *pmemd.MPI* or *sander.MPI* on CPUs.

NOTE: AMBER16 cannot be run on workstations containing RTX 20-series GPUs. These GPUs are not supported by AMBER16. Likewise, you cannot run AMBER18 on the workstations Roald and Prota as the GPUs on these workstations are very old and not supported by AMBER18. If you started a project with AMBER16, you cannot run AMBER18 and expect the same results. Therefore, if your project started with AMBER16, finish it with AMBER16. The same goes with any other version of AMBER. This difference is illustrated in the following paragraphs (between AMBER14 and AMBER16).

Available GPUs

In the Parish Lab we have access to several GPU options. The workstations Prota and Roald both have a Nvidia GeForce GTX-580 GPU. The lab workstation Pauli has a Nvidia GeForce GTX-680 GPU. The Kollman workstations contains three GTX-780 GPUs. Additionally, we have access to eight GTX-780 GPUs on URPSDR and thirty-six GTX-980 GPUs on Vadur. Workstations Elion, Franklin, and Mayer all contain two GTX-980s GPUs each while Irene and Marie have three GTX-1080 GPUs each. Cooper, Sarah, and Evan all contain one GTX-1080-Ti each, and Adam contains three GTX-1080-Ti GPUs. Finally, there are two Tesla K20 GPUs on node22 of the MERCURY cluster (Marcy). There are details below on how to run simulations using these GPUs, but here is a table of the relative speeds for running a normal GPU *pmemd.cuda* simulation on each of those GPU types. The timings given below are for a system with a total of ~33,000 atoms.

GPU Type	Location	Simulation rate	Total # GPUs available
----------	----------	-----------------	------------------------

GTX-580	Roald, Prota	~28 ns/day	2
GTX-680	Pauli	~37 ns/day	1
GTX-780	Kollman	~51 ns/day	3
GTX-780	URPSDR	~51 ns/day	8
GTX-980	Elion, Franklin, Mayer, Hamilton	~55 ns/day	8
GTX-980	Vadur	~45 ns/day	36
GTX-1080	Marie and Irene	~75 ns/day	6
GTX-1080-Ti	Cooper, Sarah, Evan, Adam	~100 ns/day	6
RTX-2080	Thais	~120 ns/day	2
RTX-2080-Ti	Urarana	~140 ns/day	28
Tesla K20	Marcy	~48 ns/day	2

Selecting GPUs for use

On a local Parish lab computer

AMBER18 on all workstations (with the exception of Thais) is installed in the directory `/usr/local/amber/amber18`. All GPUs belonging to the Nvidia Geforce GTX 10-series and older require CUDA version 8.0 or newer to run. All GTX 10-series GPUs and older are capable of running AMBER16 (this excludes the RTX 20-series), whereas all 600-series GPUs and newer are capable of running AMBER18 (this excludes the GTX 500-series). The AMBER18 install located `/usr/local/amber/amber18` in makes use of CUDA 8.0.

In order to properly load in the AMBER18 program libraries, it is necessary to first load the AMBER18 module. Module files are located in `/usr/local/ur/modulefiles` on all workstations. The AMBER18 module looks like the following example.

```

#%Module 1.0

module load cuda/8.0
module load openmpi/1.6.5/gcc-i8

# AMBER stuff
set AMBER /usr/local/amber/amber18
setenv AMBERHOME $AMBER
prepend-path PYTHONPATH $AMBER/lib/python2.7/site-packages
prepend-path PATH $AMBER/bin
prepend-path LD_LIBRARY_PATH $AMBER/lib

```

This module file does several things. First, it loads in the CUDA 8.0 module file necessary for running *pmemd.cuda* Amber simulations on the GPUs. Next, it loads the correct OpenMPI module file. This is necessary for running parallelized CPU-bound workloads (for example, *MMPBSA.py.MPI*, which will be explained later). The remaining commands set up the AMBER18 environment necessary for running all AMBER18 jobs.

Note that in the event that a module file does not work for some reason, one can still load the AMBER18 program libraries by using the command `set AMBER`

/usr/local/amber/amber18. On Thais, where AMBER18 is installed in a different directory, this command would be set `AMBER /opt/amber/amber18`. The CUDA and OpenMPI module files would need to be loaded separately. Lastly, any AMBER command that takes the form of `$AMBERHOME/bin/...` will need to be changed to `/usr/local/amber/amber18/bin/...` (or `/opt/amber/amber18/bin/...` on Thais).

If you are logged in to a local workstation in the Parish lab that has a CUDA-enabled GPU (such as Pople, Pauli, Roald, or Protas), then you can run a *pmemd.cuda* Amber simulation. But before running your simulation, you should make sure there are ample resources for your calculation on the GPU you are attempting to use for your simulation. Unfortunately, on older workstations there is not an intuitive way to do this, but a good indicator can be determined by typing the command `'nvidia-smi'`. This will give you an output that looks similar to this:

```
Tue Mar  4 13:59:48 2014
+-----+
| NVIDIA-SMI 5.319.76   Driver Version: 319.76           |
+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
+-----+-----+
|   0   GeForce GTX 680      Off   | 0000:03:00.0    N/A |         N/A         |
| 58%   80C   N/A         N/A /  N/A |    1259MB / 2047MB |      N/A      Default |
+-----+-----+

+-----+
| Compute processes:                                     GPU Memory |
| GPU      PID  Process name                               Usage      |
+-----+-----+
|   0               Not Supported                          |
+-----+
```

If there is a substantial amount of RAM being used (in the above case 1259 MB or 1.259 GB), then that is a good indication that someone is already using the GPU for something substantial and you should not run your simulation there before checking with the persons that use that computer. However, if most of the RAM is not in use (0-300 MB), then you should be fine.

On newer GPUs (GTX-980s and above), the command `'nvidia-smi'` will give your more detailed and intuitive information. The following is the output on the workstation Adam, containing 3 GTX-1080-Ti GPUs.

```
Mon Aug 19 16:28:15 2019
+-----+
| NVIDIA-SMI 375.66           Driver Version: 375.66     |
+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
+-----+-----+
|   0   GeForce GTX 108...  Off   | 0000:17:00.0    Off |         N/A         |
| 71%   84C   P2     154W / 250W |    532MiB / 11172MiB |     97%      Default |
+-----+-----+
|   1   GeForce GTX 108...  Off   | 0000:65:00.0    On  |         N/A         |
| 36%   56C   P8      21W / 250W |    740MiB / 11172MiB |      0%      Default |
+-----+-----+
|   2   GeForce GTX 108...  Off   | 0000:B3:00.0    Off |         N/A         |
+-----+
```

21%	40C	P8	9W / 250W	153MiB / 11172MiB	0%	Default	
+-----+-----+-----+-----+-----+-----+-----+-----+							
+-----+-----+-----+-----+-----+-----+-----+-----+							
Processes:							GPU Memory
GPU	PID	Type	Process name			Usage	
=====							
0	62869	C	/usr/local/amber/amber16/bin/pmemd.cuda			379MiB	

The first table will now give you information on the GPU power-state (ranging from P0-P12, which P0 indicating the maximum power-state and P12 indicating the minimum power-state). It will also tell you the wattage each GPU is currently drawing and the maximum watts the GPU can draw. Lastly, it will also tell you the percent load each GPU is operating under. In this example, it is clearly evident that GPU 0 on Adam is running something. This is because it is in the P2 power-state, is drawing 154 watts, and is at 97% load. GPUs 1 and 2 for comparison are running at the P8 power-state, are drawing less than 20 watts, and are at 0% load.

Furthermore, one can look at the second table to see the exact processes running on each GPU. In this example, GPU 0 is currently running an Amber16 MD job with the process ID 62869. If you want to kill a specific GPU job, you can use the command “`kill -9 PID`”, with PID representing the process ID of the job. For obvious reasons, you can only kill your own jobs.

If you are trying to use a workstation with multiple GPUs (such as Adam), then you will point your job to a specific GPU. You would need to setup the working environment to point *pmemd.cuda* to only the available GPU you want to run on so it knows not to use the GPU that is currently running a job. To do that, you would need to set the `CUDA_VISIBLE_DEVICES` variable. In this example for Adam, GPU 0 is being used and GPUs 1 and 2 are free. If you wanted to run a *pmemd.cuda* job on GPU 1 on Adam, the following command would direct the job to GPU 1.

```
export CUDA_VISIBLE_DEVICES=1
```

Now I know all that was very confusing, so we have developed an easier way of determining which GPUs are in use through a script. In order to use this script, you must first set up a keygen, which will authorize you to log onto any workstation without a password.

To activate keygen, use command

```
ssh-keygen
```

This will generate a `id_rsa.pub` file. To get to this file, you must change directories into your `.ssh` folder which is located in your home directory.

```
cd /home/netid/.ssh
```

The `id_rsa.pub` file holds the `rsa-id` for the workstation. The `rsa-id` is a unique identifier code for the workstation. This code varies for every person and for every workstation.

Next, you will need to create a file called `authorized_keys` within the `.ssh` folder. To do this, type

```
vi authorized_keys
```

The main script that outlines which GPUs are available or not available is located on Pople. Thus, any workstation that you intend on running a simulation on would need the rsa-id from Pople within its `authorized_keys` file.

The rsa-id for Pople is located within the `id_rsa.pub` file on Pople in the `/home/netid/.ssh` directory. If you do not see this file within that directory on Pople, you must first generate it using the command:

```
ssh-keygen
```

When you `vi` the `id_rsa.pub` file on Pople, it will look something like this

```
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEAovqXzqGqYndbsS9XKNeL6tKcw6/ndlgVQN3gna25n3BM6tdLx
Nd2r3x2rTn6SUszBpV+VopqRJhr1cb8+JbQTKGpbO6sn7E61Mijqa0+0RglymwSXslN5XMQtMDUet
QKhD0RIdMrz/EiJM9hFisocl3btAyuLmtutd9plMPBXtxvjuG8j1wg0H1RfOzi+ZrKB8i9AaTndGh
og1ltnoP+EkoYG9ApoxTxus3XZ2WWA7NfpKNlvDLjFAToC0SIMcVZgvUYLwmc6D7y9syirRylJ5bk
KUKj4fe4xhlq+GdMjiX1Oh0PMqDSdByCq5ICsz//NuR3TbHQ+L1oTDoGDgndWw==
em4uv@pople.richmond.edu
```

Now you will need to copy and paste this into the `authorized_keys` file located on the workstation that you intend on running the simulation on.

As an example, my `authorized_keys` file on Elion looks like this

```
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEAovqXzqGqYndbsS9XKNeL6tKcw6/ndlgVQN3gna25n3BM6tdLx
Nd2r3x2rTn6SUszBpV+VopqRJhr1cb8+JbQTKGpbO6sn7E61Mijqa0+0RglymwSXslN5XMQtMDUet
QKhD0RIdMrz/EiJM9hFisocl3btAyuLmtutd9plMPBXtxvjuG8j1wg0H1RfOzi+ZrKB8i9AaTndGh
og1ltnoP+EkoYG9ApoxTxus3XZ2WWA7NfpKNlvDLjFAToC0SIMcVZgvUYLwmc6D7y9syirRylJ5bk
KUKj4fe4xhlq+GdMjiX1Oh0PMqDSdByCq5ICsz//NuR3TbHQ+L1oTDoGDgndWw==
em4uv@pople.richmond.edu
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEA2fpf8rjZywt035SRDRg7sic1eNerX/T4VeHEGleihqAgn8Ywl
+X8z+RYRfl5tqjnkooF0ecCDrTSIXFFYtcNbunUMZ5A49oqte0Us8L/zrHmv+DTsfRjBrrd/pM85U
pphxucna0OFpyhF/kKD7awfkQavMEMt4x0oxhLRdBI/FdgtoZ8S83p1BNrIFtma6XfaMo8osptOmE
zpdAp1uo/yWiKq0wfScVqbVmlJ9s5RtRQfNOaBfHPZiY6Us8cd+oxjN45uY8SrH3LiFbkaNRSAjmw
DvjHl2TOWPu4V6Mtd9NZZwpfuSpVclha9/e3oPdc1+p3x5ktnGO60ygAPHqNMw==
em4uv@head.cluster
```

I have the rsa-id of both Pople and one of the clusters within my `authorized_keys` file.

So far we have authorized the workstation in which you are running your simulation on to access Pople without sanctioning the need for a password. Now, we have to do the same process over again for Pople so that Pople can access the workstation that you are running on without a

password. Thus, now you must copy the rsa-id of the workstation you are running from to the `authorized_keys` file in Pople.

After doing this for all the workstations that have GPUs, your `authorized_keys` file on Pople may look something like this.

```
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEAu+Os1fapTuDb2CqCuAT7cBBgYrRaVaoQnmO+AUXmJ11hfnjZw
D+iC8gTED7ES/AV0igXj5AxGelaZvv445KrIrTXI01v3pB9BiawE4fRhNNxY8sGupendT6RkpF8y3
IKplgxyBnErY5ohdodRvjKA0/f0bEeWmskGLOgD2m43xQo3Zi8zhVKfBgCa61Bh1rEp/Ck+hV5PWB
WYhl4cJgx5zUu+H8wZBosGA4Qgpt/dVhpW1SN+C+JasjAt5V9arMfRXmOspBXTWvA8KQtFOmu8F+F
iuXKMYAs/g0rTuBU52CiB5hErGOMxr4SMP+dQx5dg63HNpRdjajEKnMBbYD9iQ==
em4uv@kollman.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEASk5EKrzuY/mJQFq0EwrqfdHbMo4ML1XkkTZWG479mYn8JFhs5
bxPy5XfmRmquiaMcT2zACXuCsYbkXKisSK6WY2swgKQSFMDaAX24Dp40Ait6T4FaIVkPxeHOTcV2m
L7WmCPb5Dbk8va4zQ6UBZM7LLAFY2wgSNV2u0FzYCQCUUvXYkXdaoSDDpfKENr6F90pjgat65uE74
uYSHOB8dWXZje7C+X9AOaxsM62M86wm4N/M5TVG0lRtP98quDIw1h2pDPDZ8NJr6Nauh/kKgJCul
bBGqu0+7AfRJ2WTrPx1QSEMtkhh1YoDrnpRrH6NetuyqZpXclTJMeckmbftZw==
em4uv@prota.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEAlCRX8eRLmCZ4Xbni fmfO3j0FfP69sxTtvXNguGCGmGUSWB8wN
phXcYULZIOfslKq1Iav2E3AOF+dOtk2opcHbRW0gRu0405KL6/B+PhDzXyKc9ECmTLABODvpiWJq5
QjhUfKxCuB+da7eDWC1+op6JonmoGBwAO9kOlzWEAiQvTFR51Kvznt33VArVDbc0vlgOmzutF7OS
hp7YSGa1NVJhjM8mam0YRhP+XEgTL6IdwpX+PkC5xhIHMSLxNU74UZrdxsXXnKtRea+7BKJvuGggm
fruTMhsaTTTj8WgMz8cr+qCCp9k9of15kSKCmUxYygWWteACb7jy2k2gWfiyQ==
em4uv@roald.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEAOvqXzqGqYndbsS9XKNeL6tKcw6/ndlgVQN3gna25n3BM6tdLx
Nd2r3x2rTn6SUszBpV+VopqRJhrlcb8+JbQTKGpb06sn7E61Mijqa0+0RglymwsXslN5XMQtMDUet
QKhD0RIdMrz/EiJM9hFisocl3btAyuLmtutd9pLMPBxtxvjuG8j1wg0H1RfOzi+ZrKB8i9AaTndGh
og1ltnoP+EkoYG9ApoxTxus3XZ2WWA7NfpKNlvDLjFAToC0SIMcVZgvUYLwmc6D7y9syirRylJ5bk
KUKj4fe4xh1q+GdMjiX1Oh0PMqDSdByCq5ICsz//NuR3TbHQ+Ll0TDoGDgndWw==
em4uv@popple.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEArAVTeUf5eNzUD0zNSGXmfsR21uGNpVPRCfNUCI9LvSgzvzwyv
FMlmmUu+bnrOSkLOV6yZXwDigKKLLaMK86qZNLHcqtHAPjZdGHOp117DHUEIYAQY9X1pbRIoU915f
jKKwDeAQoGPbv+8RMkVEeVDQHq991TsxCR2JC5Wjp0jYz7i/1+7ySzEwZNDS6K0/T0wu1KAfIuq6h
bGGYCoKmtO4xGxX51DZ0SplDxO3SyXFh2eigzX/kzzT5StnQVhOX8MxcHEAXyxtCtkAy35cU2Lf3t
4USK+Y0GDq0x1jtT2W4/4XjybBpZJUpsb+Qe+iRsdziTqiVdUjOrjnt4HldTGQ==
em4uv@pauli.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEAzax8EVosHWHN7gH5kEUx+eqm3ezNtBwmbRx1uGo/5Etd5Q1Iq
wdyew4+9uF7V2HSZOX/pMEiMXQyWRMF9OFXGu/OSVTrek1fA7eInAl8VmNfvrGASw1RAeh/7NJ4uv
SFJfBnPgQfSvFgJrDEEw7JX/XcDlDld8GG3RxXAUgKKmPdbEIKu7dTWUh7M+Jvn14ipgqkoysGX1E
go9hc215bGijkeRIVIXu67NzoVTKsT8hP7Z3JZQq6W8YmLZebi4f4C5/yslBPLprillJh11vRBcfq
StABAmyCkNLCZneZFsOXmty11NodgEhPvKOCgaeO50Jv67xPJqAXt9zt6sXolQ==
em4uv@franklin.richmond.edu

ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAQEAAuqqjXVeV4dQ5nC7IWCIXUFWeHzCLLk13x/mXEBv7+s5nla5pX
9bdIgN2+V/D3n5ABjbvqABUb7SsN2lcq6AbLgza17Y75r/xJju9AyUPSxWsaMd+mNW0/MFXZseO6s
ACR+2Ql62AHeBDveVCBEZHhaNeRG2xhmb63eZi1X0Vx1DdzK7eI/IO6Fw19Mnrd75XdUsPnLQKBw
O8cUOM2P2SyJ3d3Wr+cfK558HYzX0WbuWMOLQ4eYI8tiXS3+U3ostIeq5rqOERA7EwjAqKbC+qX5Z
/G7uas4805aPkbduF8Y0FE15BcsdwiDtLlnN6jxt1yjj41SE93qhCMV+RWcdoQ==
em4uv@mayer.richmond.edu
```

```
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEA9UZg+vF1WtCADzLTcOnDAjTglSA8dd+uoEeGkRrg5/REVsZ4I
kxUu/zYYKRTQug5VlZWzsJhoynNdHsLMu3CkIgqRNFJlrVwyQOqP6hkhTVW7c0954gvskcabRDsxl
h/m5w9PP+x+H+GykZNIQrzQsKSDC4Ukx4yOJp3iG4XwJcYcN9LYgf7U3wVfhDN8KsHnjPg27fcXV
kzCKJiwwJxVtjx49gCctdhXBe0waUehi2fVdazAjnrsHqVEAxu8R+4lkhGAAq/JeFoIy1YCZQTnm0
T7HKy2hZNpF4vqPSow5oV6ZT3yAgIbI95uADaXg/IX5PhU9bZkbLtZObRBWb9Q==
em4uv@hamilton.richmond.edu
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEA1QSQ5j60kc0OriwbfRebSd/56WYamRqPo4MwvVRg6+giRxY1d
RHuYPtCuken0OywcH+700wrl60kwuQmAl+yy1nFJ+bgaNI8DMbKli9gZOXDU6cRUP0mrLr7NDcyMg
wyHJRKFYsXOzWp1A98GSFSSJrL15po6L41ayP6h+FcExcMFQb911yJQS1f9brv1CZhSqclzyi5kGN
EuCw7EAYJbja4ELuXWMAMkL+qfIm9PfpL9BKxg+odIByDVn86VUckKg1YWY3wwuKoLKKmp35C+aj
duF/G82rZ9Xbh2/690+bt/rQNv7uHYKjqmMUff4PijsFEhrNdpBvS08no48zNQ==
em4uv@elion.richmond.edu
```

Notice how I have the rsa-id of every workstation with a GPU in my `authorized_keys` file on Pople. Pople has to have access to every workstation with a GPU so that the script can get edited from any workstation that you choose to run on.

Now that you have all of your `authorized_keys` files set up, let's check the script to ensure that the workstation is indeed available. First, you must locate the script and paste it in your home directory on your workstation. The script is located on Pople in Gus' home folder. To access this directory from Pople, type the following

```
cd /home/abeane
```

Once there

```
ls -a
```

to view all the contents of the directory. The gpu script is named `.gpu.sh*`. `rsync` this file over to the home directory on your workstation, which is Elion in my case.

```
rsync -azvp -progress .gpu.sh* elion:/home/em4uv
```

Now, go back to your workstation and execute the script from your home directory.

```
cd /home/netid
```

```
./ .gpu.sh*
```

When you do this, your terminal will look something like this.

Hostname	GPU	ID	Availability	User
thais	RTX-2080	0	Available	
thais	RTX-2080	1	Available	
adam	GTX-1080-Ti	0	Unavailable	mr7rn

adam	GTX-1080-Ti	1	Unavailable	ja9ia
adam	GTX-1080-Ti	2	Available	
sarah	GTX-1080-Ti	0	Unavailable	mr7rn
cooper	GTX-1080-Ti	0	Unavailable	mr7rn
evan	GTX-1080-Ti	0	Available	
irene	GTX-1080	0	Available	
irene	GTX-1080	1	Available	
irene	GTX-1080	2	Available	
marie	GTX-1080	0	Available	
marie	GTX-1080	1	Available	
marie	GTX-1080	2	Available	
hamilton	GTX-980	0	Available	
hamilton	GTX-980	1	Available	
franklin	GTX-980	0	Available	
franklin	GTX-980	1	Available	
mayer	GTX-980	0	Available	
mayer	GTX-980	1	Available	
elion	GTX-980	0	Unavailable	lm9av
elion	GTX-980	1	Unavailable	lm9av
pople	GTX-780	0	Not Working	
pople	GTX-650	1	Available	
pople	GTX-780	2	Not Working	
kollman	GTX-780	0	Available	
kollman	GTX-780	1	Available	
kollman	GTX-780	2	Available	
kollman	GTX-780	3	Not Working	
roald	GTX-580	0	Available	

Once you know the GPU you want to use is available, then you are ready to run MD with *pmemd.cuda*. You can run *pmemd.cuda* straight from the command line prompt in Terminal, or you could choose to run Amber as I do using a C-shell script. Below is an example script (maybe called `md.sh`) that can be used to run Amber on a local workstation in the Parish lab:

```
#!/bin/csh -f
$AMBERHOME/bin/pmemd.cuda ...
```

The first line says that we are running in a C-shell environment (as all computers in the Parish Lab are), and the second line is the Amber *pmemd.cuda* command line that we will get into more details about later in this protocol. Once you make this file, `md.sh`, you will need to change the permissions to make it executable.


```
chmod u+x md.sh
```

Now that you are ready to run your simulation, you want to make sure you edit the GPU script so that it indicates to everyone that you are using a GPU. Luckily, we have a script that already does that for you. You have to go back to Gus' home folder on Pople.

```
cd /home/abeane
```

In his home directory there is a directory called GPU. Inside this directory, there is another directory called Workstation_scripts.

```
cd GPU/Workstation_scripts
```

This directory has a script for every GPU on every workstation. rsync over the script for the GPU that you wish to use to the directory on the workstation in which you will be running the simulation from. For example, I am going to run a simulation on GPU 0 on Franklin in a directory called test.

```
rsync -azvp --progress franklin_0.sh* franklin:/em4uv/scratch/test
```

Now go back to the directory where your simulation files are located and vi the script that you just copied over. It will look something like this. You will need to replace the netid em4uv with your own netid.

```
#!/bin/bash

export CUDA_VISIBLE_DEVICES=0

ssh pople bash -c "'
sed -i -e 's/franklin          GTX-980          0
Available/franklin          GTX-980          0          Unavailable
em4uv/g' /home/abeane/GPU/GPUstatus.txt
'"

#things that need to be run
./md.sh

ssh pople bash -c "'
sed -i -e 's/franklin          GTX-980          0
Unavailable          em4uv/franklin          GTX-980          0
Available/g' /home/abeane/GPU/GPUstatus.txt
'"
```

This script was written in bash so the first line lets the workstation know that the script was written in bash. The second line which says export CUDA_VISIBLE_DEVICES=0 tells the workstation to send the simulation to GPU 0.

```
ssh pople bash -c "'
```

```
sed -i -e 's/franklin GTX-980 0
Available/franklin GTX-980 0 Unavailable
em4uv/g' /home/abeane/GPU/GPUstatus.txt
''
```

This is the part that edits the script located on Pople. It basically switches the word Available to Unavailable next to the franklin 0.

```
#things that need to be run
./md.sh
```

Underneath this heading, you will need to execute the script that has all your MD executables within it. This file is likely called `md.sh`.

```
ssh pople bash -c ''
sed -i -e 's/franklin GTX-980 0
Unavailable em4uv/franklin GTX-980 0
Available/g' /home/abeane/GPU/GPUstatus.txt
''
```

Finally, this last part edits the script to say Available instead of Unavailable once your simulation has completed.

The final step is to execute the workstation script to begin your simulation. Since MD simulations can usually take a long time to run, I prefer to execute this script using the `nohup` command. `nohup` allows for you to start a command or program and then tells the computer to continue running that program even after you log out of the computer. This allows you to log off the computer so that someone else can use it, but still keeps your MD simulation running on that computer. To execute with `nohup`, type

```
nohup ./franklin_0.sh &
```

The ampersand tells the computer to run the program in the background, still allowing you to use that Terminal window to do more work. 😊

Now when you execute the command `./gpu.sh*` your terminal should look like this.

Hostname	GPU	ID	Availability	User
thais	RTX-2080	0	Available	
thais	RTX-2080	1	Available	
adam	GTX-1080-Ti	0	Unavailable	mr7rn
adam	GTX-1080-Ti	1	Unavailable	ja9ia
adam	GTX-1080-Ti	2	Available	
sarah	GTX-1080-Ti	0	Unavailable	mr7rn

cooper	GTX-1080-Ti	0	Unavailable	mr7rn
evan	GTX-1080-Ti	0	Available	
irene	GTX-1080	0	Available	
irene	GTX-1080	1	Available	
irene	GTX-1080	2	Available	
marie	GTX-1080	0	Available	
marie	GTX-1080	1	Available	
marie	GTX-1080	2	Available	
hamilton	GTX-980	0	Available	
hamilton	GTX-980	1	Available	
franklin	GTX-980	0	Unavailable	em4uv
franklin	GTX-980	1	Available	
mayer	GTX-980	0	Available	
mayer	GTX-980	1	Available	
elion	GTX-980	0	Unavailable	lm9av
elion	GTX-980	1	Unavailable	lm9av
pople	GTX-780	0	Not Working	
pople	GTX-650	1	Available	
pople	GTX-780	2	Not Working	
kollman	GTX-780	0	Available	
kollman	GTX-780	1	Available	
kollman	GTX-780	2	Available	
kollman	GTX-780	3	Not Working	
roald	GTX-580	0	Available	

Notice how it says Unavailable and my netid next to franklin 0.

Templates for all of the GPU scripts can be found in the directory
 /home/scr/ja9ia/Workstation_scripts on Prota.

Note that an alias both can and should be set up to make this script easier to run. An alias is simply a custom command that can be created to shorten a longer command. To do this, type `cd` and hit the enter button to navigate to your home directory. From here, use `vi` to edit the hidden file `.cshrc`. This file contains all of the aliases available to you in the Linux C-shell environment. If you have not created any aliases, your `.cshrc` file should look like the following.

```
if (-e /usr/local/etc/csh.cshrc) then
    source /usr/local/etc/csh.cshrc
endif
```

Next, we are going to want to create the alias command “gpu”. This command will be a shortcut command for running the command `/home/netid/./gpu.sh*` from any location on your workstation. To do this, add the following line to your `.cshrc` file (with special care taken to change `netid` to your actual `netid`).

```
alias gpu '/home/netid/./gpu.sh'
```

Your `.cshrc` file should look like the following example.

```
if (-e /usr/local/etc/csh.cshrc) then
    source /usr/local/etc/csh.cshrc
endif
alias gpu '/home/ja9ia/./gpu.sh'
```

From here, save this file and type the following command to activate your alias.

```
source ~/.cshrc
```

This command will make it so that all new terminals will have your new alias available. From here on, you will be able to check the GPU availability script as simply as typing the command “gpu”. Note that you can use this method of alias creation for any command you might want to shorten.

Thais and AMBER18

As mentioned previously, the GPUs on Thais (Nvidia GeForce RTX-2080) belong to the newest (as of 1/2020) family of Nvidia GPUs (RTX 20-series). As a result, these GPUs require a different version of the CUDA code in order to run MD correctly using AMBER18. The GTX 10-series GPUs require CUDA 8.0 or newer for AMBER16 and AMBER18. This version of CUDA also satisfies the requirements for all earlier GPUs (with the exception of the GTX 500-series, which is not compatible with AMBER18). The RTX 20-series GPUs, however, require CUDA 9.2 or newer. This leads to some complications with how AMBER18 is installed on Thais.

For all programs, the golden image (or original program install) is located on Adam. When a new program is installed, it is first installed on Adam in `/usr/local/` and then copied over to the same directory on every other workstation. Due to the fact that the golden image for AMBER18 on Adam is compiled to run on CUDA 8.0, anytime a new program is installed on Adam in `/usr/local/` and copied to the other workstations, all programs in this same directory on the other workstations will be overridden. Since Thais requires CUDA 9.2 or newer, this means that anytime a new program is installed, AMBER18 on Thais will be overridden and replaced with a version that is incompatible with the RTX 20-series GPUs. This is of course a major problem.

In order to correct this issue, AMBER18 on Thais is installed in a different directory. While AMBER18 on all other workstations is located in `/usr/local/amber/amber18`, the program files on Thais can be found in `/opt/amber/amber18`. Program files located in `/opt/` are not overridden whenever new programs are installed.

This has some ramifications. First and foremost, the module file to load AMBER18 is different than on all of the other workstations. To invoke AMBER18 correctly, the following module file is used.

```
#%Module 1.0

module load cuda/10.1
module load openmpi/1.6.5/gcc-i8

# AMBER stuff
set AMBER /opt/amber/amber18
setenv AMBERHOME $AMBER
prepend-path PYTHONPATH $AMBER/lib/python2.7/site-packages
prepend-path PATH $AMBER/bin
prepend-path LD_LIBRARY_PATH $AMBER/lib
```

Note that CUDA version 10.1 is loaded instead of the usual version 8.0. Also note that the AMBER environment is set from the directory `/opt/amber/amber18` instead of the usual `/usr/local/amber/amber18`. Lastly, it should be noted that this module should be located in `/opt/modulefiles` rather than `/usr/local/ur/modulefiles` to prevent the module file itself from being overridden each time files are pushed from Adam.

From here, everything should be in order for AMBER18 to function normally on Thais.

On URSPDR

In the winter of 2013, URSPDR received two additional nodes (node12 and node13). Each node had 16 CPUs and more importantly for this topic 4 GPUs (Nvidia GeForce GTX-780). These GPUs can be used to run Amber's *pmemd.cuda* on the URPSDR cluster. Login to URPSDR using

```
ssh urspdr
```

Create a folder in your work directory

```
mkdir /work/bmiller4/Amber_simulation
```

Change directories into that directory

```
cd /work/bmiller4/Amber_simulation
```

From a Terminal window on your local computer with your Amber input files, you can move your Amber files to URSPDR using rsync

```
rsync -azvp --progress list_of_Amber_files urspdr:/work/bmiller4/Amber_simulation/
```

Unlike on the Parish lab computers, the cluster already organizes which GPUs are available for simulations, thus you do not have to worry about setting the `CUDA_VISIBLE_DEVICES` variable. This is done through the PBS queueing system. All that you have to do is submit a `jobfile` script according to the PBS format with information to setup the

environment and run Amber. An example `jobfile` script is shown below, where the actual line to run Amber is not fully explicit, but we will get to that later.

```
#!/bin/csh

#$ -S /bin/csh
#$ -N jobname
#$ -cwd
#$ -q gpu.q
#$ -o amber.out
#$ -j y
#$ -cwd
#$ -pe mpi 1

# print the date and time so we know when the simulation starts
date

# Change directories to the folder with all the files
cd /work/bmiller4/1LV5_mismatch

# Setup the environment to run Amber properly
module load amber/18

# Print the node we are running on
echo "nodefile="
hostname
echo "=end nodefile"

# run Amber
$AMBERHOME/bin/pmemd.cuda ...

# Print the date so we know when the simulation ended
date
```

This script is much more involved than the simple C-shell script used on a local workstations in the lab. The first line is essentially the same as the `md.sh` script described previously to run MD on local workstations. The next several lines beginning with `#$` are read by the PBS queue manager when assigning all the jobs on the URSPDR cluster. This job is being submitted only to the nodes that have GPUs (by specifying `gpu.q`) You should change the name of the job (`jobname`). You can also change the name of the standard output file (which is currently called `amber.out`); this is the file where the information that we want printed from the script will be printed. Finally, most jobs will only need one GPU (using `pmemd.cuda`), but if you do run Amber in parallel using `pmemd.cuda.MPI` then you will need to change the `#$ -pe mpi 1` to `#$ -pe mpi 2`. The rest of the script is commented (lines starting with a pound sign (`#`) are comments and are not read by the computer) to explain the purpose of the following lines. You should make sure to change the directory (currently set to `/work/bmiller4/1LV5_mismatch`) to the directory with all your MD files. And obviously the `pmemd.cuda` command has not been explicitly defined in this example, but it that will be explained later. To submit this `jobfile` to the queue so that it will run, type

```
qsub jobfile
```

And then you should see the `jobname` in the queue when you type `qstat`. In this particular example, two jobs are running.

job-ID	prior	name	user	state	submit/start at	queue	slots	ja-task-ID
26163	0.50500	zoo	cc7na	r	12/02/2019 21:33:14	gpu.q@node12.cluster	1	
26164	0.50500	A18_TEST	ja9ia	r	12/02/2019 21:36:29	gpu.q@node12.cluster	1	

If you want to kill a job for any reason, use the command `qdel job-ID`. For example, if I wanted to kill my job in the above example, I would use the command `qdel 26164`

Once your simulation is finished, you'll need to move your files onto your local workstation. This prevents the cluster's storage space from overfilling. In a terminal logged into your local workstation, enter the following `rsync` command:

```
rsync -avz urspdr://work/bmiller4/Amber_simulation/'list_of_Amber_files' .
```

On Vadur

In February 2016, we received 9 additional GPU nodes (gpu01-gpu09). Each node had 12 CPUs and more importantly for this topic 4 GPUs (Nvidia GeForce GTX-980). These GPUs can be used to run Amber's *pmemd.cuda* on Vader. Login to the new cluster using

```
ssh vadur
```

Create a folder in your work directory

```
mkdir /work/bmiller4/Amber_simulation
```

Change directories into that directory

```
cd /work/bmiller4/Amber_simulation
```

From a Terminal window on your local computer with your Amber input files, you can move your Amber files to Vadur using `rsync`

```
rsync -azvp --progress list_of_Amber_files vadur:/work/bmiller4/Amber_simulation/
```

Unlike on the Parish lab computers, the cluster already organizes which GPUs are available for simulations, thus you do not have to worry about setting the `CUDA_VISIBLE_DEVICES` variable. This is done through the PBS queueing system. All that you have to do is submit a `jobfile` script according to the PBS format with information to setup the environment and run Amber. An example `jobfile` script is shown below, where the actual line to run Amber is not fully explicit, but we will get to that later.

```
#!/bin/csh
#$ -S /bin/csh
#$ -N jobname
#$ -cwd
#$ -q gpu.q
#$ -o amber.out
#$ -j y
```

```

#$ -cwd
#$ -pe mpi 1

# print the date and time so we know when the simulation starts
date

# Change directories to the folder with all the files
cd /work/bmiller4/1LV5_mismatch

# Setup the environment to run Amber properly
module load amber/18

# Print the node we are running on
echo "nodefile="
hostname
echo "=end nodefile"

# run Amber
$AMBERHOME/bin/pmemd.cuda ...

# Print the date so we know when the simulation ended
date

```

This script is much more involved than the simple C-shell script used on a local workstations in the lab. The first line is essentially the same as the `md.sh` script described previously to run MD on local workstations. The next several lines beginning with `#$` are read by the PBS queue manager when assigning all the jobs on the Vadur cluster. This job is being submitted only to the nodes that have GPUs (by specifying `gpu.q`) You should change the name of the job (`jobname`). You can also change the name of the standard output file (which is currently called `amber.out`); this is the file where the information that we want printed from the script will be printed. Finally, most jobs will only need one GPU (using `pmemd.cuda`), but if you do run Amber in parallel using `pmemd.cuda.MPI` then you will need to change the `#$ -pe mpi 1` to `#$ -pe mpi 2`. The rest of the script is commented (lines starting with a pound sign (`#`) are comments and are not read by the computer) to explain the purpose of the following lines. You should make sure to change the directory (currently set to `/work/bmiller4/1LV5_mismatch`) to the directory with all your MD files. And obviously the `pmemd.cuda` command has not been explicitly defined in this example, but it that will be explained later. To submit this `jobfile` to the queue so that it will run, type

```
qsub jobfile
```

And then you should see the jobname in the queue when you type `qstat`. In this particular example, two jobs are running.

job-ID	prior	name	user	state	submit/start at	queue	slots	ja-task-ID
26163	0.50500	zoo	cc7na	r	12/02/2019 21:33:14	gpu.q@gpu04.cluster	1	
26164	0.50500	A18_TEST	ja9ia	r	12/02/2019 21:36:29	gpu.q@gpu06.cluster	1	

If you want to kill a job for any reason, use the command `qdel job-ID`. For example, if I wanted to kill my job in the above example, I would use the command `qdel 26164`

Once your simulation is finished, you'll need to move your files onto your local workstation. This prevents the cluster's storage space from overfilling. In a terminal logged into your local workstation, enter the following rsync command:

```
rsync -avz vadur://work/bmiller4/Amber_simulation/'list_of_Amber_files' .
```

On Urarana

In December 2019, we received a new cluster with seven GPU nodes (gpu01-gpu07). Each node contains 48 CPUs and 4 GPUs (Nvidia GeForce RTX 2080-Ti). As with URSPDR and Vadur, these GPUs can be used to run Amber's *pmemd.cuda* on Urarana. Login to the new cluster using

```
ssh urarana
```

Create a folder in your work directory

```
mkdir /work/ja9ia/AMBER18_Job
```

Change directories into that directory

```
cd /work/ja9ia/AMBER18_Job
```

From a Terminal window on your local computer with your Amber input files, you can move your Amber files to Vadur using rsync

```
rsync -azvp --progress list_of_Amber_files urarana:/work/ja9ia/AMBER18_Job
```

Unlike URSPDR and Vadur however, Urarana uses the SLURM queueing system. As such, your `jobfile` will look slightly different to accommodate the SLURM format. The `jobfile` should contain all of the information necessary to setup the environment and run Amber. The following is an example `jobfile` for running MD on Urarana.

```
#!/bin/csh

#SBATCH --job-name=AMBER_Job
#SBATCH --output=AMBER_Job.txt
#SBATCH --partition=gpu
#SBATCH --gres=gpu:rtx2080ti:1
#SBATCH --ntasks=1
#SBATCH --time=00:00:00
#SBATCH --mail-user=justin.airas@richmond.edu
#SBATCH --mail-type=ALL

# Print the simulation start date/time
date

# Print the GPU node the simulation is running on
echo "I ran on:"
cd $SLURM_SUBMIT_DIR
echo $SLURM_NODELIST

# Load the necessary program libraries
module load cuda/10.1
```

```

module load openmpi-3.0.1/gcc
source /usr/local/amber/amber18/amber.csh
setenv CUDA_HOME /opt/cuda-10.1

# Set the output file directory
cd /work/ja9ia/AMBER18_Job

# Run Amber Jobs
$AMBERHOME/bin/pmemd.cuda ...

# Print the simulation end date/time
date

```

As with the PBS `jobfile` for URSPDR and Vadur, this script is more involved than a standard C-shell script. The first line is simply setting up the appropriate C-shell environment.

The next lines beginning with `#SBATCH` are read by SLURM to appropriately manage GPU jobs on Urarana. The first of these lines sets the jobname (`AMBER_Job` in this example, you should change it to match your job). The next line outputs a text document detailing information about your job (more on this later). In this example, this file is named `AMBER_Job.txt`. The next line informs SLURM which partition (or node) to run the job on. This should always be set to `gpu` to select for GPU-containing nodes configured under the `gpu` partition name. Next, the GPU type and the number of GPUs of that type that you want to run on is specified. In this example, we indicate we want to run on one Nvidia GeForce RTX 2080-Ti. Note that if in the future GPUs are added to Urarana, you'll need to change the GPU name to the new type of GPU you want to run on. The next line simply sets the number of tasks to be run (always just 1). Next, we set the amount of time the job is allowed to run for. Setting this value to `00:00:00` means the job is allowed to run until completion. The last two lines optionally allow you to send yourself an email indicating that a job has been started. If you're running many jobs at a time, you'll likely get many, many emails. If you do decide to include these lines, be sure to change the email to your address (I'd prefer to not be spammed).

From here, everything is set up for SLURM to run manage the job. The remaining lines will execute commands and print the results of those commands to the `AMBER_Job.txt` output file. Assuming no errors occur, it would be expected that this file will indicate the job start date/time, the node the job is running on, and the end date/time. Any errors messages will be written to this file as well, which allows for an easier time diagnosing them.

The command `sinfo` will detail currently available resources. The output will look something like the following. In this example, node `gpu02` is in use.

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
gpu*	up	infinite	1	mix	gpu02
gpu*	up	infinite	6	idle	gpu[01,03-07]

To submit your `jobfile` to the queue, type the following command:

```

sbatch jobfile

```

To check if your job (and all other submitted jobs) are running, use the command `squeue`. The output will look something like the following. In this example, two GPUs on node `gpu02` are in use (one by Carol and the other by me).

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST (REASON)
	45	gpu	A18_SLUR	cparish	R	29:19	1 gpu02
	46	gpu	A18_SLUR	ja9ia	R	9:29	1 gpu02

If you want to kill a job for any reason, use the command `scancel JOBID`. For example, if I wanted to kill my job in the above example, I would use the command `scancel 46`

Once your simulation is finished, you'll need to move your files onto your local workstation. This prevents the cluster's storage space from overflowing. In a terminal logged into your local workstation, enter the following `rsync` command:

```
rsync -avz urarana://work/netid/Amber_simulation/'list_of_Amber_files' .
```

On MERCURY (Marcy)

NOTE: The following section has not been updated. Tesla K20 GPUs are extremely slow and not supported by new versions of Amber.

Marcy has two Tesla K20 GPUs present on node22 that can be used to run MD simulations using *pmemd.cuda*. You can access Marcy using `ssh`

```
ssh <username>@marcy.bucknell.edu
```

On Marcy, your working directory should be a folder in your home directory (`/home/<username>`). So make a folder to run your simulations and transfer your files over there using `rsync` (described in the previous section). On Marcy, your Amber `jobfile` script should look similar to the following:

```
#!/bin/tcsh
#PBS -N jobname
#PBS -l mem=1GB
#PBS -q gpu
#PBS -j oe
#PBS -l walltime=60:00:00
#PBS -r n
#PBS -l ncpus=1
#
# Change directories to the folder that you submitted the job from
cd $PBS_O_WORKDIR
# Setup the environment to run Amber 12
module load amber12/intel
# Setup the environment to run Amber 14
```

```
# setenv AMBERHOME /home/bmiller/amber14
# MD command
$AMBERHOME/bin/pmemd.cuda ...
```

As with URSPDR, the first few lines of this script are for the queue manager. You should change the `jobname` so that you can recognize the simulation when you are looking the queue. This script will only submit the job to the `gpu` queue so that it doesn't try to run on any nodes without GPUs. You will need to set a `wall-time` sufficient to run the entire simulation. To get this accurate, you may want to run a short test job to see how fast your simulation will run on the GPUs before you run for a longer period of time. And please, be respectful of other users. Don't set a job to run for more than a 2-3 days so that other users will also be able to use the GPUs without having to wait weeks for your job to get done. This jobfile asks for only a single GPU (`ncpus=1`), but you can also ask for both GPUs (`ncpus=2`) if you want to run in parallel using *pmemd.cuda.MPI*. You can also use Amber 14 instead of the native Amber 12 on marcy by adding a comment (#) in front of the `module load amber12/intel` line, and remove the comment (#) from the `setenv AMBERHOME /home/bmiller/amber14` line. Once again, after you make these modifications to the script and correctly type the *pmemd.cuda* command (see below), then you can submit this `jobfile` script by typing

```
qsub jobfile
```

On NERSC

NOTE: The following section has not been updated. Tesla C2050 GPUs are extremely slow and not supported by new versions of Amber.

To access the NERSC computers, you will need to `ssh` to Carver

```
ssh -Y username@carver.nersc.gov
```

Notice that when you `ssh` you are actually logging into Carver, which is the main NERSC login supercomputer. The supercomputer we actually want to use is though is called Dirac. Dirac is where all the GPUs are that we will use for our MD simulations. Dirac has a lot of different nodes with different GPUs on them. Below is a breakdown of GPU type and number for each node on Dirac.

Node	GPU type	Number of GPUs
dirac01	N/A	N/A
dirac02	Tesla C2050	1
dirac03	Tesla C2050	1
dirac04	Tesla C2050	1
dirac05	Tesla C2050	1
dirac06	Tesla C2050	1
dirac07	Tesla C2050	1

dirac08	Tesla C2050	1
dirac09	Tesla C2050	1
dirac10	Tesla C2050	1
dirac11	Tesla C2050	1
dirac12	Tesla C2050	1
dirac13	Tesla C2050	1
dirac14	Tesla C2050	1
dirac15	Tesla C2050	1
dirac16	Tesla C2050	1
dirac17	Tesla C2050	1
dirac18	Tesla C2050	1
dirac19	Tesla C2050	1
dirac20	Tesla C2050	1
dirac21	Tesla C2050	1
dirac22	Tesla C2050	1
dirac23	Tesla C2050	1
dirac24	Tesla C2050	1
dirac25	Tesla C2050	1
dirac26	Tesla C2050	1
dirac27	Tesla C2050	1
dirac28	Tesla C2050	1
dirac29	Tesla C2050	1
dirac30	Tesla C2050	1
dirac31	Tesla C2050	1
dirac32	Tesla C2050	1
dirac33	Tesla C2050	1
dirac34	Tesla C2050	1
dirac35	Tesla C2050	1
dirac36	N/A	N/A
dirac37	Tesla C2050	1
dirac38	Tesla C2050	1
dirac39	Tesla C2050	1
dirac40	Tesla C2050	1
dirac41	Tesla C2050	1
dirac42	Tesla C2050	1
dirac43	Tesla C2050	1
dirac44	Tesla C2050	1
dirac45	Tesla C1060	1
dirac46	Tesla K20Xm	1
dirac47	Tesla K20c	1
dirac48	Tesla K40c	1
dirac49	Tesla C2050	4
dirac50	Tesla C1060	4

Isn't that more information than you really wanted? No? Okay, then we shall continue.

There are essentially four types of GPUs available on NERSC (Tesla C1060, Tesla C2050, Tesla K20, and Tesla K40), so the obvious question becomes how fast does Amber run on each of these GPU architectures? For starters, Amber does not support MD on the very old (in computer hardware years) Tesla C1060 GPUs (dirac45 and dirac50). So don't even try to run on them. As far as the other three GPU types go, below is a table of their recorded simulation speeds compared to a GTX-780 GPU found on Pople or URSPDR. These timings were all performed on the same system that contained a total of 81,689 atoms.

GPU type	Speed (ns/day)
GTX-780 (on Pople or URSPDR)	19.95
Tesla C2050	7.98
Tesla K20Xm	21.5
Tesla K40	22.0

As you can tell, the Tesla K20 and Tesla K40 GPUs are slightly faster than our local GTX-780 GPUs, while the Tesla C2050 is *significantly* slower, which is unfortunate since Dirac consists mostly of nodes with Tesla C2050 GPUs. But... running MD slowly is better than not running it at all. And if you are simulating a much smaller system (compared to 81,689 atoms) and not running for many hundreds of nanoseconds, then the Tesla C2050 GPUs will likely be sufficient for your needs. Finally, I should also note that running Amber in parallel as been tested on NERSC, and the results are not good. The same system mentioned above (81,689 total atoms) was run in parallel (using `pmemd.cuda.MPI`) on 2 GPUs (Tesla K40 and Tesla K20X) and the result was a speed of only 16.9 ns/day. In other words, running parallel using 2 GPUs on different nodes is actually slower than running the same simulations using only one of those GPUs. This is due to the interconnect between the two nodes being slow making communication between the two GPUs the bottleneck and consequently slows down the simulations. If you do want to run Amber in parallel on Dirac, I strongly urge you to use dirac49, which has 4 of the Tesla C2050 GPUs all on the same node, so it will be at least faster than just running Amber on a single C2050 GPU. For the above-mentioned system (81,689 total atoms), I was able to reach 12.1 ns/day using four C2050 GPUs, which is up substantially from 7.98 ns/day with only one C2050 GPU.

The NERSC computers all use the C-shell (`csh`) environment. To load the proper environment variables for running Amber (and setup some nice aliases), edit your `.cshrc.ext` file in your home directory.

```
vi ~/.cshrc.ext
```

Then add the following lines just *before* the last line (`# end .cshrc.ext`) in your `.cshrc.ext` file.

```
# Aliases
alias scr 'cd $SCRATCH'
alias qu 'qstat -u username'
alias lt 'ls -lthr'
alias la 'ls -lthra'
```

```
alias smi 'nvidia-smi'
alias qdirac 'qstat | grep dirac'

# Setting up the environment
setenv AMBERHOME /global/scratch2/sd/username/amber14
setenv CUDA_HOME /usr/common/usg/cuda/5.5
setenv LD_LIBRARY_PATH "/usr/common/usg/cuda/5.5/lib:${LD_LIBRARY_PATH}"
test -f /global/scratch2/sd/username/amber14/amber.csh && source
/global/scratch2/sd/username/amber14/amber.csh

# Loading the proper modules
module unload pgc
module unload openmpi
module unload cuda
module load gcc-sl6
module load openmpi-gcc-sl6
module load cuda
```

Notice that in several places above, you should replace `username` with your actual username (such as `bmiller`) for NERSC. After doing this, you should source your `.cshrc` file so the changes you just made take effect.

```
source ~/.cshrc
```

Unfortunately, Amber is not already compiled on NERSC, and we are unable to view each other's files. So you will need to put Amber in your scratch directory. Fortunately (for you), Amber has already been compiled on NERSC by our group. And since the computer is the same, you should be able to avoid recompiling Amber by simply copying an already compiled version of Amber to NERSC and unpacking it in your scratch directory. How do you do that? I'm glad you asked...

The NERSC compilation of Amber can be found as a tarball on Pople. So `ssh` there first.

```
ssh pople
```

Then `cd` to the directory containing the tarball

```
cd /scratch/bmiller4
```

Then `rsync` the tarball to your scratch directory on NERSC.

```
rsync -azvp --progress NERSC_amber_compiled.tgz username@carver.nersc.gov:
/global/scratch2/sd/username/
```

And make sure to insert your actual NERSC username instead of `username` in the above command. Once this transfer has completed, you should be able to `ssh` to carver

```
ssh username@carver.nersc.gov
```

Move to your scratch directory

```
cd $SCRATCH
```

Then unpack the Amber tarball

```
tar -zxvf NERSC_amber_compiled.tgz
```

This will create an `amber14` folder in your scratch directory and will print *every single file* in the Amber folder to the screen, so don't be alarmed when your screen is overwhelmed with text. To make the Amber completely usable for you, edit the `amber.csh` file in your Amber directory.

```
cd $AMBERHOME
```

and then edit the file

```
vi amber.csh
```

In the first line of the file

```
setenv AMBERHOME "/global/scratch2/sd/bmiller/amber14"
```

Change `bmiller` to your NERSC username and re-save the file. After doing this, you should have a compiled version of Amber that you can use.

As with most supercomputers, you should place all your files for simulations/calculations within folders in your scratch directory. You can get directly to your scratch directory at any time by typing

```
cd $SCRATCH
```

or just by using the alias

```
scr
```

if you took my advice and added the above lines to your `.cshrc.ext` file in your home directory. To move a directory on your local Parish workstation containing all your Amber files for your simulation to NERSC (carver), use the `rsync` command

```
rsync -azvp --progress MD_files username@carver.nersc.gov:/global/scratch2/sd/username/
```

Make sure to replace `username` with your actual NERSC username. And keep in mind that in this instance `MD_files` is a folder name containing all the necessary Amber files (`prmtop`, `inpcrd/rst7`, `mdin` files, etc). Now once you are logged into Carver, you should `cd` to the directory containing all your Amber files (the folder called `MD_files` in the example above), and then make a new `jobfile` that looks similar to the following


```

#!/bin/csh -f
#PBS -N jobname
#PBS -j oe
#PBS -l nodes=1:ppn=1
#PBS -q dirac_reg
#PBS -l walltime=00:30:00

```

```
cd $PBS_O_WORKDIR
```

```

# 25 ns of unrestrained MD at 335 K
$AMBERHOME/bin/pmemd.cuda ...

```

This should look similar to previous jobfiles mentioned earlier in this protocol. Note that this is a PBS queue, and all the regular commands for submitting/deleting jobs using this queue can be found in the Advanced Unix Manual.

You should change `jobname` to anything you want that distinguishes your job from any others. If you happen to want to use more than one node for some reason, you can change that by increasing `nodes=1` to `nodes=2`. Additionally, on this same line you can select what type of GPU you want to use if you prefer. For example, if you want to only use the Tesla K40c GPU (why not, it's the fastest, right?), then you can change this line in your jobfile to read

```
#PBS -l nodes=1:ppn=1:k40
```

You can also tell the queue manager that you want to use the Tesla K20c by specifying

```
#PBS -l nodes=1:ppn=1:k20
```

Or the Tesla K20x by specifying

```
#PBS -l nodes=1:ppn=1:k20x
```

You can also specify to use any of the Tesla C2050 GPUs by changing the line to read

```
#PBS -l nodes=1:ppn=1:fermi
```

And finally, if `dirac49` is not being utilized, you could run Amber in parallel on four of the Tesla C2050 GPUs by changing the line in your jobfile to read

```
#PBS -l nodes=1:ppn=8:mfermi
```

And if you do choose this route, make sure you modify your

```
$AMBERHOME/bin/pmemd.cuda ...
```

command to read

```
mpirun -np 4 $AMBERHOME/bin/pmemd.cuda.MPI...
```

This will ensure that you are actually utilizing all four of the C2050 GPUs for your simulation instead of just one. As mentioned previously, using four C2050 GPUs will improve your performance by an extra 50% compared to using only a single C2050 GPU.

Dirac has four different queues available to all users. In the above example, the `dirac_reg` queue is being utilized, which will be standard for most users in our lab. Here is a breakdown of the important details about each queue.

Submit Queue	Nodes	Max Wallclock	Relative Priority	Max Run Limit
<code>dirac_int</code>	1	30 mins	1	1
<code>dirac_reg</code>	1-12	6 hrs	2	2
<code>dirac_small</code>	1	6 hrs	2	4
<code>dirac_special</code>	1-48	Contact consult@nersc.gov to arrange		

You will notice a few things about the above table. First, you can *not* run any job on any of the queues for longer than six hours. This is a major difference from running on URSPDR or a local workstation that has no max wall-clock. The relative priority also changes depending on the queue you choose. For the most part, `dirac_int` is primarily for testing, while `dirac_special` is primarily for long production jobs that need access to a *lot* of nodes for a long period of time. You likely won't need to use either `dirac_int` or `dirac_special` for any reason. If you are planning on running more than two jobs, then you may want to consider using `dirac_small` instead of `dirac_reg` since you can run four jobs instead of just two.

You must also pay special attention to the max wall-clock time. If you are running on `dirac_reg` or `dirac_small`, you will not be able to run any calculations over 6 hours in length at a time. So take that into account when you are considering how many nanoseconds you plan on simulating for each job.

The line

```
cd $PBS_O_WORKDIR
```

simply tells the queue to change directories to the folder that you submitted the job, which should be where all your files are located. And the final line is clearly the command for running Amber (`pmemd.cuda`). Notice that the “...” after `pmemd.cuda` is something you should fill in with the information needed to run your simulation (see below for more details).

Once you have successfully finished modifying your `jobfile`, you are ready to submit your simulation. Unfortunately, a caveat of running on Dirac is that we cannot submit our `jobfile` from the login node (Carver). Instead, you will need to `ssh` to one of Dirac's nodes in order to submit. I often just `ssh` to `dirac46` out of habit, but you can actually `ssh` to any of them (listed previously in a table).

```
ssh dirac46
```

You will need to type your NERSC password when prompted. Once you have successfully `ssh`'d into one of Dirac's compute nodes, you should `cd` into the folder in your scratch directory with your `jobfile` and corresponding Amber files. Once there, you should be able to type

```
qsub jobfile
```

And the job should submit, printing a message similar to the following

```
[bmiller@dirac46 3HT3_ternary_10mM_Mg]$ qsub jobfile
9692089.cvrsvc09-ib
```

where 9692089 is the job ID for the submitted calculation. If necessary, you should be able to delete jobs and submit held jobs just as you would for any PBS queue (see Advanced Unix Manual for more details).

Invoking Amber (*pmemd.cuda*)

In this section, we will explain how to invoke Amber in serial and parallel using *pmemd.cuda* to run MD simulations. It is important to note, although all of the examples presented use *pmemd.cuda*, the same commands will work if you choose to run Amber on CPUs using either *sander* or *pmemd*. For both versions, it will first be necessary to load the Amber module (discussed [in-depth](#) earlier). This can be done using the command `module load amber/18`

In serial

Running Amber in serial simply refers to invoking *pmemd.cuda* (or *sander* or *pmemd*) using only a single GPU (or CPU). Although *pmemd.cuda* performs many calculations on the GPU, a single CPU is still used for all calculations in order to perform normal setup and writing to files. The full usage of *pmemd.cuda* can be found by typing

```
$AMBERHOME/bin/pmemd.cuda --help
```

Entering this command will print the following full usage of *pmemd.cuda* to the Terminal window.

```
usage: pmemd [-O] -i mdin -o mdout -p prmtop -c inpcrd -r restrt
           [-ref refc -x mdcrd -v mdvel -e mden -inf mdinfo -l logfile]
           [-amd amdlog_name -suffix output_files_suffix]
```

We will not use most of the possible options in typical MD simulations. In fact, a typical Amber *pmemd.cuda* simulations command is shown below.

```
$AMBERHOME/bin/pmemd.cuda -O -i mdin_file -o mdout_file -p solvated_complex.prmtop -c
solvated_complex.inpcrd -ref reference_coordinates.inpcrd -r output_restart_file.rst7
-x output_trajectory.nc
```

The `-o` flag tells Amber that if one of the output files already exists overwrite it anyway. The `-i` flag is followed by the Amber MD input file name (*mdin_file*) that contains all the simulations parameters/details (how long to run, how often to write the coordinates, temperature, pressure, etc) used by *pmemd.cuda*. We will get into more detail about the *mdin* file below. The `-o` flag is followed by the MD output file name (*mdout_file*). The *mdout_file* contains all of the energies for the system at each step that is printed, as well as a bunch of other information including all of the parameters specified in the *mdin_file*. The `-p` is followed by the name of the solvated complex topology file (*solvated_complex.prmtop*) that you previously created using *tleap*. Next, the `-c` flag precedes the name of the solvated complex coordinate file (*solvated_complex.inpcrd*) that was also created using *tleap* at the same time as the topology file. In the event that you are restarting/extending a simulation, the `-c` flag will instead represent the coordinate file that was written by a previous *pmemd.cuda* calculation (and thus will have a *.rst7* extension to denote that it is a restart coordinate file instead of an initial input coordinate file (*.inpcrd*)). The `-ref` flag is used if you are imposing any restraints on the system (typically done during minimization, heating, and equilibration). This (*reference_coordinates.inpcrd*) is a set of coordinates (usually identical to the coordinates specified using the `-c` flag) that Amber will use as a reference to keep the structure nearly fixed to during the simulation. The `-r` flag begins the list of output file names and is followed by the name of the output restart file (*output_restart_file.rst7*) that is printed at the end of each simulation. This file contains all of the updated coordinates and velocities that are necessary to continue the simulation from its current time instead of at the beginning. This file would serve as the input coordinate file (`-c`) for a future *pmemd.cuda* if you need to extend the simulation further. Finally, the `-x` flag is followed by the name of the trajectory file (*output_trajectory.nc*) that will be written that contains all of the coordinates for every frame of the simulation you want to save. These files can be quite larger (many GB) for long simulations depending on how often you choose to write the coordinates to this file (which is managed in the MD in file explained later). The Amber default is to write these in ASCII format (plain text) usually with a *.mdcrd* extension, but these types of files can be easily corrupted and become large and difficult to transfer very quickly. For that reason, I strongly encourage the use of binary NETCDF file formats for trajectory files with the *.nc* extension (as shown above). You can control the use of ASCII versus NETCDF format in the MD input file, which will be described in more detail below. Finally, it should be noted that trajectory files are only written when MD is actually performed. Thus, no trajectory files are written during the initial minimization process, but will be written during all other stages of the MD process.

Below is an example of a serial *pmemd.cuda* command.

```
$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 4KTQ_solv.prmtop -c 4KTQ_solv_md016.rst7 -
ref 4KTQ_solv_md016.rst7 -o 4KTQ_solv_md017.mdout -r 4KTQ_solv_md017.rst7 -x 4KTQ
_solv_md017.nc
```

This example is of a simulation on PDB 4KTQ and is the 17th time that the simulation has been extended longer than the initial simulation time. Note that it reads in the restart file from the 16th simulation (4KTQ_solv_md016.rst7) and all outputs are for the 17th MD simulation.

In parallel

pmemd.cuda can be run in parallel across multiple GPUs when available (such as Pople, URSPDR, or Marcy), BUT (as of Amber 12) running across multiple GPUs is highly inefficient and you do not receive nearly the speed-up you might expect or want. For example, on Marcy I can run a simulation on one GPU that gets ~19 ns/day, but when I use both GPUs I only increase to ~23 ns/day. This is expected to be significantly improved upon in future releases of Amber, but these are the results of our current software availability. Thus, you can imagine that it is not very efficient to run *pmemd.cuda* in parallel, so I would generally advise against it unless the GPUs are in low demand for all users.

That being said, the usage for *pmemd.cuda.MPI* (the parallel version of *pmemd.cuda*) is exactly the same. The difference is that you will need to specify the number of GPUs to be used on the command line (as well as in the jobfile script if you are trying to run on Marcy or URSPDR, explained previously) and you will need to specify an MPI wrapper. The generic command looks something like this:

```
mpirun -np # $AMBERHOME/bin/pmemd.cuda.MPI -O -i mdin_file -o mdout_file -p
solvated_complex.prmtop -c solvated_complex.inpcrd -ref reference_coordinates.inpcrd -
r output_restart_file.rst7 -x output_trajectory.nc
```

Notice that this example is exactly the same as the serial command, except using *pmemd.cuda.MPI* instead of *pmemd.cuda* and at the beginning you must specify an MPI wrapper (in this case, `mpirun`) followed by `-np` the flag and a number (#) corresponding to the number of GPUs you want to use. For example,

```
mpirun -np 2 $AMBERHOME/bin/pmemd.cuda.MPI -O -i md.mdin -p 3HPO_solv.prmtop
-c 3HPO_solv_md024.rst7 -ref 3HPO_solv_md024.rst7 -o 3HPO_solv_md025.mdout -r
3HPO_solv_md025.rst7 -x 3HPO_solv_md025.nc
```

This is running an MD simulation extension (the 25th extension) on PDB 3HPO across 2 GPUs using `mpirun`. I should also point out that another common MPI wrapper is `mpiexec`, which can be used the same as `mpirun` except that the number of GPUs does not need to be specified on the command line, but this can be more complicated if you are not wanting to use all of the available GPUs.

Example mdin files

Now that we have covered how to run MD using *pmemd.cuda* and *pmemd.cuda.MPI*, we will discuss some details about the MD input files (AKA mdin files) for the various stages of MD simulations, and provide examples. If you are new to running MD simulations, I remind you that the general process of an MD simulation involves four main steps in this order: minimization, heating, equilibration, unrestrained MD.

In the following section I show example mdin files for *pmemd.cuda*, but I am NOT going to cover every single option in each file. I will cover the highlights of each mdin file that are important on a regular basis and for other users to modify them. If you want an extensive list of all options available in Amber, please consult that Amber User Manual PDF in the Amber directory (`$AMBERHOME/doc/`).

Minimization

Unlike Desmond, there is no default way of performing minimization (or MD for that matter) in Amber. Personally, I use a seven-step minimization at the beginning of each new MD simulation that I will explain in detail here. You are welcome to modify this process any way that you deem necessary, but I am simply providing you with a step-wise explanation of the minimization technique that I have found successful for past systems of mine.

Since PDB files do not contain hydrogen atoms and the solvent molecules are not initially in optimized positions after being added by *tleap*, the minimization is primarily used to help reduce any clashes created by adding hydrogen atoms, ions, and/or water molecules. The mdin file for the first step (which I typically call `min1.mdin`) of minimization looks like this

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H=',
/
```

Amber does not read the first line of the mdin file, so that is a line typically used to describe the type of MD that will be run. The `&cntrl` denotes the start of a name ‘namelist’ of variables and options. There are different namelists for different purposes (for example, QM/MM simulations also require a `&qmmm` namelist), but the `&cntrl` is in every Amber mdin file. The end of the namelist is denoted by the ‘/’, which you see at the end of the file here showing the end of the `&cntrl` namelist. Setting `imin=1` tells Amber we want to run a minimization using 1000 steps of a common minimization method called steepest descent minimization (`ncyc=1000`), followed by another common minimization method called conjugate gradient for the remainder of the 5000 total minimization steps (`maxcyc=5000`). We print the energies to the mdout file (the file name following the `-o` flag in the command line) every 50 steps (`ntp=50`) for the entire minimization (5000 steps). We define an 8.0 Å cutoff for calculating all non-bonded energies (`cut=8`, which is Amber’s default/suggested value). This means that only non-bonded atoms within an 8.0 Å radius contribute to the non-bonded energy term of a particular atom. Our explicit solvent simulations are performed using periodic boundary conditions (i.e. the unit cell repeats in all directions for infinity), which allows the biomolecule to leave the unit cell at times as it naturally drifts during the simulation, but setting `iwrap=1` means that when any molecule leaves the unit cell, it re-enters the unit cell on the opposite side of the cell. This is simply an imaging artifact to make it easier for us to visualize the simulations and in no way effects the physics of the simulations. As I mentioned earlier, we want to use restraints during the minimization process (`ntr=1`), so we impose a restraint weight of 10.0 kcal/mol/Å² (`restraint_wt=10.0`) on all non-hydrogen atoms in our system (`restraintmask='!@H=`) since we really only want to ligand the hydrogen atoms at this point. Obviously, there are other MD options not presented here that

could be specified, but for my purposes this is enough and all other variables are given their default values by Amber.

The second stage of minimization is exactly like the first except I have lowered the restraint weight to only 5.0 kcal/mol/Å² (`restraint_wt=5.0`), as you see in the next minimization mdin file (`min2.mdin`).

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=5.0,
  restraintmask='!@H=',
/
```

Same with the third stage where I have lowered the restraint weight even further to 2.0 kcal/mol/Å² (`restraint_wt=2.0`) for `min3.mdin`.

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=2.0,
  restraintmask='!@H=',
/
```

I lower the restraint weight even further to 1.0 kcal/mol/Å² (`restraint_wt=1.0`) for `min4.mdin`.

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=1.0,
  restraintmask='!@H=',
/
```

The restraint weight is dropped to 0.5 kcal/mol/Å² (`restraint_wt=0.5`) for `min5.mdin`.

```

Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=0.5,
  restraintmask='!@H=',
/

```

And dropped to 0.1 kcal/mol/Å² (restraint_wt=0.1) for min6.mdin.

```

Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=0.1,
  restraintmask='!@H=',
/

```

Finally, in the last stage of minimization (min7.mdin) I do not impose any restraints and allow all atoms to minimize freely.

```

Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncyc=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
/

```

Notice how `ntr=1` has been removed along with the restraint weight and restraint mask options for this last stage. Each minimization step is very quick, so I typically combine all seven of these steps into a single jobfile script. An example is shown below that could be run on a local Parish lab workstation with a cuda-enabled GPU.

```

#!/bin/csh -f

$AMBERHOME/bin/pmemd.cuda -O -i min1.mdin -p 1L3S_solv.prmtop -c 1L3S_solv.inpcrd -ref
1L3S_solv.inpcrd -o 1L3S_solv_min1.mdout -r 1L3S_solv_min1.rst7 -x 1L3S_solv_min1.nc

```



```

$AMBERHOME/bin/pmemd.cuda -O -i min2.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min1.rst7 -ref
1L3S_solv_min1.rst7 -o 1L3S_solv_min2.mdout -r 1L3S_solv_min2.rst7 -x 1L3S_solv_min2.nc

$AMBERHOME/bin/pmemd.cuda -O -i min3.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min2.rst7 -ref
1L3S_solv_min2.rst7 -o 1L3S_solv_min3.mdout -r 1L3S_solv_min3.rst7 -x 1L3S_solv_min3.nc

$AMBERHOME/bin/pmemd.cuda -O -i min4.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min3.rst7 -ref
1L3S_solv_min3.rst7 -o 1L3S_solv_min4.mdout -r 1L3S_solv_min4.rst7 -x 1L3S_solv_min4.nc

$AMBERHOME/bin/pmemd.cuda -O -i min5.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min4.rst7 -ref
1L3S_solv_min4.rst7 -o 1L3S_solv_min5.mdout -r 1L3S_solv_min5.rst7 -x 1L3S_solv_min5.nc

$AMBERHOME/bin/pmemd.cuda -O -i min6.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min5.rst7 -ref
1L3S_solv_min5.rst7 -o 1L3S_solv_min6.mdout -r 1L3S_solv_min6.rst7 -x 1L3S_solv_min6.nc

$AMBERHOME/bin/pmemd.cuda -O -i min7.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min6.rst7 -ref
1L3S_solv_min6.rst7 -o 1L3S_solv_min7.mdout -r 1L3S_solv_min7.rst7 -x 1L3S_solv_min7.nc

```

Notice how each *pmemd.cuda* command represents a different stage of the minimization, made clear by the use of *min1.mdin* on the first *pmemd.cuda* line, *min2.mdin* on the second line, and so on.

Heating

PDB files have no intrinsic velocities contained within them, thus no kinetic energy meaning the atoms have no initial velocities. During the next stage of MD, heating, we assign initial velocities (at random using the starting time as the initial seed value since *ig=-1*) and add thermal energy to the system. The heating *mdin* file (*heat.mdin*) that I use for this stage looks something like this

```

Explicit solvent initial heating mdin
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=50000, ntwx=50000, ntwr=50000, nstlim=1000000,
  dt=0.002, ntt=3, gamma_ln=5.0, ig=-1,
  ntc=2, ntf=2, cut=8, ntb=2, ntp=1,
  iwrap=1, ioutfm=1, nmropt=1,
  ntr=1, restraint_wt=10, restraintmask=':1-603'
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=100000,
  VALUE1=10.0, VALUE2=100.0,
/
&wt
  TYPE='TEMP0', ISTEP1=100001, ISTEP2=500000,
  VALUE1=100.0, VALUE2=300.0,
/
&wt TYPE='END' /

```

This looks substantially different and more complicated than the minimization input files, but we will examine all the important details here. First, note that there still a *&cntrl* namelist, in addition to three *&wt* namelists near the end of the file. These namelists arise because we set *nmropt=1* to allow for NMR-style restraints that we use to increase the temperature. Let us start by looking at those *&wt* namelists. In the first *&wt* namelist, we are changing the temperature

(TYPE='TEMPO') starting at step 0 (ISTEP1=0) at a temperature of 10.0 K (VALUE1=10.0), and increasing linearly until we reach 100.0 K (VALUE2=100.0) at 100,000 steps (ISTEP2=100000). In the second &wt namelist, we finish our temperature change (TYPE='TEMPO'), by increasing from 100.0 K (VALUE1=100.0) at step 100,001 (ISTEP1=100001) linearly to a final temperature of 300.0 K (VALUE2=300.0) at step 500,000 (ISTEP2=500000). The third and final &wt namelist just tells Amber that is the end of the NMR-style restraints.

Outside of the heating portion of the mdin file, the rest of the file is mostly just related to molecule dynamics. This is not a minimization, so we set `imin=0`. `irest=0` says we are beginning a new simulation and we should not read in velocities `ntx=1`, especially since there are none in the minimization restart file (*min7.rst7) anyway. The mdin file heats the system over a total of 2.0 ns (1,000,000 total steps (`nstlim=1000000`)) using a 2 fs (`dt=0.002`) time-step) and we every 50,000 steps we are printing energies to the mdout file (`ntpr=50000`), writing coordinates to the trajectory file (`ntwx=50000`), and printing out an updated restart file that includes the current coordinates and velocities (`ntwr=50000`). Our temperature is controlled using a common Langevin thermostat (`ntt=3`) with an atomic collision frequency of 5.0 ps⁻¹ (`gamma_ln=5.0`). We apply the SHAKE algorithm (`ntc=2`) that fixes all bond lengths involving a hydrogen atom that allows us to use a 2.0 fs time-step and allows us to ignore all bond interactions involving H-atoms (`ntf=2`). We also use periodic boundary conditions applying constant pressure conditions (`ntb=2`) and wrap any molecules back into the unit cell that try to leave (`iwrap=1`). As I mentioned earlier, I strongly encourage the use of binary NETCDF trajectory files to be written because they are more precise, safer, and smaller than ASCII formatted trajectories. The variable `ioutfm=1` tells Amber to write the trajectory in NETCDF format. Finally, during this MD stage we are mostly heating the solvent (and will slowly allow this kinetic energy to be transferred to the solute in the equilibration step coming up next) because we restrain (`ntr=1`) all atom of the solute (in this example, that is residues 1-603, `restraintmask=':1-603'`) with a 10 kcal/mol/Å² force (`restraint_wt=10`). For any simulation that you run will you need to modify the `restraintmask` residues to match your system. If you aren't sure how many residues are in your solute, then just take a look at the top of your 'dry' topology file (.prmtop) that you saved with *tleap* that has no ions or water molecules in it. The top of the topology file should look something like this

```
%VERSION  VERSION_STAMP = V0001.000  DATE = 09/06/13  14:17:44
%FLAG TITLE
%FORMAT(20a4)
default_name
%FLAG POINTERS
%FORMAT(10I8)
  10093      17      4973      5260      11359      7206      22090      17407          0          0
  55841      603      5260      7206      17407      68        155        67        36          0
    0          0          0          0          0          0          0          0        34          0
    0
```

The second column of the second row of numbers after `%FORMAT(10I8)` is the residue number (it is 603 here, just like in our example heating mdin file). Once you change the residue restraint mask to the correct number of residues in your 'dry' prmtop file, and modify the final temperature (if you are going to run at some temperature other than 300 K), then this heating mdin file should be ready for any system you need to heat with Amber's *pmemd.cuda*. An example `heat.sh` script to run the heating is shown below.

```
#!/bin/csh -f
$AMBERHOME/bin/pmemd.cuda -O -i heat.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_min7.rst7 -ref
1L3S_solv_min7.rst7 -o 1L3S_solv_heat.mdout -r 1L3S_solv_heat.rst7 -x 1L3S_solv_heat.nc
```

Equilibration

After we have heated our solvent to our target temperature (typically 300 K), we need to equilibrate our solute at that temperature as well since we had previously been restraining the solute during the previous heating process. Once this, this process can be done many, many different ways, but the equilibration protocol that I typically use (because it has historically worked for me in the past) involves seven stages of equilibration that slowly reduces the restraint every 500 ps over a total of 2.5 ns simulation time. The mdin file for the first stage of heating (which I typically call eq1.mdin) looks something like this

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
  ntr=1, restraint_wt=10, restraintmask=':1-603'
/
```

This file should (hopefully!) look eerily similar to `heat.mdin`. The main difference in the `&cntrl` namelist is that we are now only running this simulation for 500 ps (250,000 steps (`nstlim=250000`) at 2.0 fs per step (`dt=0.002`)). We have also specified for our initial (`tempi=300`) and reference temperature (i.e. where the temperature should stay during the simulation, `temp0=300`) to be set at 300.0 K. We have also reduced the collision frequency to 1.0 ps^{-1} (`gamma_ln=1.0`) and increased `cut` to `cut=9` instead of `cut=8` in `heat.mdin`. Notice that just like the heating, we have put a 10 kcal/mol/ $\text{\AA}^2$ restraint (`restraint_wt=10`) on residue 1-603 of the solute (`restraintmask=':1-603'`). So basically, this first stage of equilibration is nearly the same as the heating, *except* that we are no longer changing the temperature. However, the remaining steps will be intended to lower this restraint weight enough that we remove it as we prepare for running MD without restraints in our production run (i.e. unrestrained MD).

The second stage of equilibration is exactly like the first stage, except the restraint weight in the `eq2.mdin` file has been halved to 5.0 kcal/mol/ $\text{\AA}^2$ (`restraint_wt=5.0`).

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
```

```
ntr=1, restraint_wt=5.0, restraintmask=':1-603'
/
```

The third step (eq3.mdin) lowers the restraint down to 2.0 kcal/mol/A² (restraint_wt=2.0).

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
  ntr=1, restraint_wt=2.0, restraintmask=':1-603'
/
```

The mdin file for the fourth stage of equilibration (eq4.mdin) is shown below, where the restraint weight has been halved again to 1.0 kcal/mol/A² (restraint_wt=1.0).

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
  ntr=1, restraint_wt=1.0, restraintmask=':1-603'
/
```

eq5.mdin further reduces the restraint down to 0.5 kcal/mol/A² (restraint_wt=0.5).

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
  ntr=1, restraint_wt=0.5, restraintmask=':1-603'
/
```

The sixth and final stage of equilibration that imposes a restraint (eq6.mdin) contains a small 0.1 kcal/mol/A² (restraint_wt=0.1) on the solute.

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
```

```
temp0=300, gamma_ln=1.0, ig=-1,
ntp=1, ntc=2, ntf=2, cut=9,
ntb=2, iwrap=1, ioutfm=1,
ntr=1, restraint_wt=0.1, restraintmask=':1-603'
/
```

The final stage of equilibration (eq7.mdin) involves no restraints on any atoms on the system; thus it is fully unrestrained molecular dynamics.

```
Explicit solvent molecular dynamics constant pressure MD
&cntrl
  imin=0, irest=0, ntx=1,
  ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
/
```

You can see in the above mdin file that all lines of are identical to the previous equilibration mdin files *except* that the line referring to the restraint has been removed (i.e. there is no `ntr=1, restraint_wt=0.1, restraintmask=':1-603'` in eq7.mdin). This allows for 500 ps of equilibration for the solute before we begin more unrestrained MD that we will use for our analysis.

These equilibration stages also do not take very long (the time for each stage is ~2 hours, although that can vary with system size), so I typically run all of these stages using a single jobfile script. An example script that could be used to run all of these commands on a Parish lab workstation can be found below.

```
#!/bin/csh -f

$AMBERHOME/bin/pmemd.cuda -O -i eq1.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_heat.rst7 -ref
1L3S_solv_heat.rst7 -o 1L3S_solv_eq1.mdout -r 1L3S_solv_eq1.rst7 -x 1L3S_solv_eq1.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq2.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq1.rst7 -ref
1L3S_solv_eq1.rst7 -o 1L3S_solv_eq2.mdout -r 1L3S_solv_eq2.rst7 -x 1L3S_solv_eq2.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq3.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq2.rst7 -ref
1L3S_solv_eq2.rst7 -o 1L3S_solv_eq3.mdout -r 1L3S_solv_eq3.rst7 -x 1L3S_solv_eq3.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq4.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq3.rst7 -ref
1L3S_solv_eq3.rst7 -o 1L3S_solv_eq4.mdout -r 1L3S_solv_eq4.rst7 -x 1L3S_solv_eq4.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq5.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq4.rst7 -ref
1L3S_solv_eq4.rst7 -o 1L3S_solv_eq5.mdout -r 1L3S_solv_eq5.rst7 -x 1L3S_solv_eq5.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq6.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq5.rst7 -ref
1L3S_solv_eq5.rst7 -o 1L3S_solv_eq6.mdout -r 1L3S_solv_eq6.rst7 -x 1L3S_solv_eq6.nc

$AMBERHOME/bin/pmemd.cuda -O -i eq7.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq6.rst7 -ref
1L3S_solv_eq6.rst7 -o 1L3S_solv_eq7.mdout -r 1L3S_solv_eq7.rst7 -x 1L3S_solv_eq7.nc
```

Just like when we ran the seven stage minimization all at once, notice that each *pmemd.cuda* command uses a different mdin file (eq1.mdin, eq2.mdin, and so on). Also, the initial coordinates for the first *pmemd.cuda* simulation is the restart file from the heating MD stage

(1L3S_solv_heat.rst7), and each subsequent command reads in the coordinates and velocities from the previous equilibration stage (1L3S_solv_eq1.rst7, 1L3S_solv_eq2.rst7, and so on). The final restart file (1L3S_solv_eq7.rst7) will be used as the input coordinates and velocities for our unrestrained MD simulation below.

Unrestrained MD

After we have finished our equilibration steps, we begin the final (and probably most important) stage of MD. We have been careful and cautious with each of the previous steps to minimize, heat, and equilibrate slowly so that our system does not become unstable during our unrestrained MD stage (or “production run”). The unrestrained MD will take the longest of all the MD stages and it will be the sole focus of all future analyses. The amount of simulation time needed to complete the unrestrained MD stage is different from system to system, and also varies depending on what questions you want to answer about the system. Sometimes 10 ns is long enough to find out what you want to know; other times it may require 5 μ s! This is something you should think about beforehand and potentially ask more advanced MD users in the group. And even the most advanced MD users may not be able to narrow it down exactly because all simulations are different and have their own caveats.

When I simulate my production run, I typically break the simulation into chunks. So if I want to run a 1 μ s simulation, I won’t submit a job that runs all of that at once. Maybe I split it up into 50 or 100 ns chunks. This is often done for several reasons. First, it gives you the opportunity to analyze each chunk as they are completed. Second, in the event of a computer issue and the simulation dies unexpectedly, it is easy just to restart the simulation from the previous stage instead of restarting somewhere in the middle. So it allows you to keep uniform chunks of your simulation. Third, it is also more practical because our trajectories (especially for long simulations) can get quite large, and when you are transferring or trying to save/store a very large file, there is a lot that could be wrong to corrupt your data, and want to minimize our chances at corrupting our trajectories. The specific chunk of time I choose to simulate for depends on where I am running the simulation. For some places (like a Parish lab workstation) it doesn’t matter too much, but for some supercomputer systems, there is a wall-clock that you must abide by and you so need to make sure that each chunk of simulation will complete in less than the maximum wall-clock time. A typical unrestrained MD input file (md.mdin) is shown below.

```
Explicit solvent molecular dynamics constant pressure 25 ns MD
&cntrl
  imin=0, irest=1, ntx=5,
  ntp=50000, ntwx=50000, ntwr=-50000, nstlim=12500000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1,
/
```

Again, this mdin file should look very familiar to previous files. The main differences are that we are now restarting the simulation (irest=1) by reading in the previous velocities *and* coordinates from the restart file (ntx=5). This chunk of unrestrained MD is being performed for 25 ns

(12,500,000 total steps ($\text{nstlim}=12500000$) using a 2.0-fs time step ($\text{dt}=0.002$)). We are now printing the energies every 50,000 steps (or 100 ps, $\text{ntpr}=50000$), writing to the trajectory file every 50,000 steps ($\text{ntwx}=50000$), and writing the restart file every 50,000 steps ($\text{ntwr}=-50000$). The negative value used for $\text{ntwr}=-50000$ is a flag that tells *pmemd.cuda* to write out every restart file with a unique name and keep that restart file instead of it being overwritten every 50,000 steps. This isn't wanted or necessary in all cases, but this gives you more flexibility at times. If you observe something interesting might have occurred between two trajectory frames, you can go back to the previous frame and restart the simulation from there and print more often to see what you might have missed by not writing often enough. The restart file is actually written twice using this mdin file. There will be a normal restart file (defined used the $-r$ flag in the *pmemd.cuda* command) that will be overwritten every 50,000 steps with the current coordinates and velocities. An example restart file name would look similar to this: 1L3S_solv_md001.rst7. Using a negative value here, will also give you a bunch of restart files that look something like this: 1L3S_solv_md001.rst7_50000 at the 50,000th step, 1L3S_solv_md001.rst7_100000 at the 100,000th step, and so on.

An example jobfile that could be used to run a single 25ns Amber job on one of the Parish lab workstations for unrestrained MD is shown below.

```
#!/bin/csh -f

# 25 ns of unrestrained MD at 300 K
$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq7.rst7 -ref
1L3S_solv_eq7.rst7 -o 1L3S_solv_md001.mdout -r 1L3S_solv_md001.rst7 -x 1L3S_solv_md001.nc
```

The line # 25 ns of unrestrained MD at 300 K is just a comment and is skipped by the computer when the script is executed. It is simply a helpful message to let you know what the *pmemd.cuda* job will be running. Once this first simulation has completed, if we wanted to extend the simulation, we could alter the jobfile to look like this.

```
#!/bin/csh -f

# 25 ns more of unrestrained MD at 300 K
$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_md001.rst7 -ref
1L3S_solv_md001.rst7 -o 1L3S_solv_md002.mdout -r 1L3S_solv_md002.rst7 -x 1L3S_solv_md002.nc
```

Notice that the input and reference coordinates are now from the md001 simulation (1L3S_solv_md001.rst7), and all the output files have the md002 designation. You MUST to change the name of the output files or else the previous output files will be overwritten since we specified to use the $-o$ flag. Also, the mdin file has not changed (md.mdin) since we are running MD with exactly the same parameters as the previous simulation. And this process would continue for as long as you want to run your production simulation.

For simulations of any length, it is best practice to run every job in 25ns chunks. For example, if you intend to run a 100ns simulation, you should do so by running 4 successive jobs of 25ns each. This ensures that if an error occurs during the simulation, you can restart from one of the 25ns chunks rather than from the beginning of the simulation. Your script for this example 100ns job should look as follows:

```
#!/bin/csh -f
```

```
# 100ns of unrestrained MD at 300K (4 25ns chunks)
$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_eq7.rst7 -ref
1L3S_solv_eq7.rst7 -o 1L3S_solv_md001.mdout -r 1L3S_solv_md001.rst7 -x 1L3S_solv_md001.nc

$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_md001.rst7 -ref
1L3S_solv_md001.rst7 -o 1L3S_solv_md002.mdout -r 1L3S_solv_md002.rst7 -x 1L3S_solv_md002.nc

$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_md002.rst7 -ref
1L3S_solv_md002.rst7 -o 1L3S_solv_md003.mdout -r 1L3S_solv_md003.rst7 -x 1L3S_solv_md003.nc

$AMBERHOME/bin/pmemd.cuda -O -i md.mdin -p 1L3S_solv.prmtop -c 1L3S_solv_md003.rst7 -ref
1L3S_solv_md003.rst7 -o 1L3S_solv_md004.mdout -r 1L3S_solv_md004.rst7 -x 1L3S_solv_md004.nc
```

Analysis of Unrestrained MD Simulations

The AmberTools package contains a plethora of analysis tools of Amber trajectories. Feel free to peruse the AmberTools user manual ([\\$AMBERHOME/doc/Amber18.pdf](#)). In this section I am going to highlight two of the better and more important analysis tools. I won't cover everything about each one, but I want to give you a glimpse into how to use them and some of the things each one can do. I should also point out that you should not run these analysis tools interactively (i.e. not through the queue) on any compute cluster (such as URSPDR, Marcy, etc). Instead, I suggest you run these analysis tools on your local workstation because they can be quite computationally intensive and you do not want to be responsible for locking up a compute cluster.

cpptraj

The main trajectory analysis tool for Amber is called *cpptraj* (trajectory analysis tool written in C++). This is a very powerful program that can do a LOT of different analyses on Amber trajectories. I strongly encourage you to look through the *cpptraj* section of the user manual just to get an idea of all the things it can do for you.

Concatenating and imaging trajectories

Often when we are running molecular dynamics simulations our solute will generally drift and move throughout the unit cell, and sometimes even leave one side of the unit cell and enter the opposite side. This makes it difficult to visualize or analyze our simulations at times, so *cpptraj* gives us the ability to concatenate our MD simulations (that are likely divided into different chunks anyway), image the solute back into the middle of the unit cell, and remove all translational and rotational movement during the trajectory for easier visualization and analysis. As a reminder, this does not change the results of the simulations in any way. All the motions of the protein are still exactly the same, but the different frames will all now be in the same frame of reference so we can obtain accurate results and easily visualize the entire trajectory without being forced to look at the drift of the biomolecule (unless that is something you are interested in specifically). A typical *cpptraj* input file (that I usually call *cpptraj.in*) to concatenate and image a trajectory in this way would look similar to this.


```

trajin 1L3S_solv_md001.nc
trajin 1L3S_solv_md002.nc
trajin 1L3S_solv_md003.nc
trajin 1L3S_solv_md004.nc
trajin 1L3S_solv_md005.nc

autoimage
#strip :WAT:Cl-:Na+

rmsd protein :1-603@CA

trajout 1L3S_imaged_solv.nc netcdf

```

This input file will read in your first five trajectories (`trajin 1L3S_solv_md001.nc`, `trajin 1L3S_solv_md002.nc`, ..., `trajin 1L3S_solv_md005.nc`) and then image those trajectories (`autoimage`) by placing the solute in the middle of the unit cell for all frames. The line after `autoimage` is commented out (`#`), but sometimes it is easier and more practical not to visualize all of the water molecules if we are not interested in them. This command (`strip :WAT:Cl-:Na+`), if uncommented (i.e. remove the `#` at the beginning), will remove (i.e. strip) all of the water, chloride, and sodium residues from the trajectory. We perform a RMSD (root-mean-square deviation) alignment of just the α -carbons for all residues of the solute (residues 1-603 in this example) to remove any of the rotational movement from the solute (`rmsd protein :1-603@CA`). The final line of the input file (`()`) tells *cpptraj* to write out a new trajectory file in NETCDF format that is a concatenation of all the original Amber trajectories that were read in at the beginning of the file. You can invoke *cpptraj* using the following command in Terminal.

```
$AMBERHOME/bin/cpptraj -p solvated_complex.prmtop -i cpptraj_file.in
```

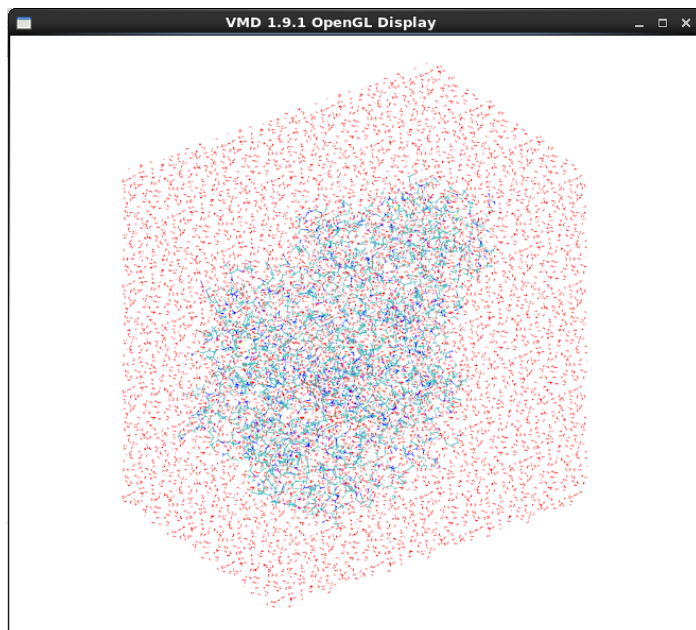
It is important that the topology (`.prmtop`) file you provide matches exactly with the trajectories in your *cpptraj* input file (i.e. do not accidentally give *cpptraj* the ‘dry’ topology file when all of the trajectories contain ions and water molecules). Also, you need to make sure you run this command in the directory that contains the necessary `prmtop`, *cpptraj* input file, and any trajectories being read by *cpptraj* (or you specify the exact path of each trajectory file if they are not in your current directory).

Once you have made this new trajectory you can visualize it using either Chimera or VMD. To open the your new trajectory in Chimera, first open Chimera using the command “`chimera`” in the terminal. Next, in the top menu, navigate to Tools → MD/Ensemble Analysis → MD Movie. From here, click on ‘Browse’ next to the box titled `Prmtop`, navigate to the directory containing your `prmtop`, and select that `prmtop` file. Next, click on the ‘Add...’ button next to the `Trajectory` box, and select your new, imaged NETCDF trajectory file. Click OK, and your trajectory file will open after a few sections.

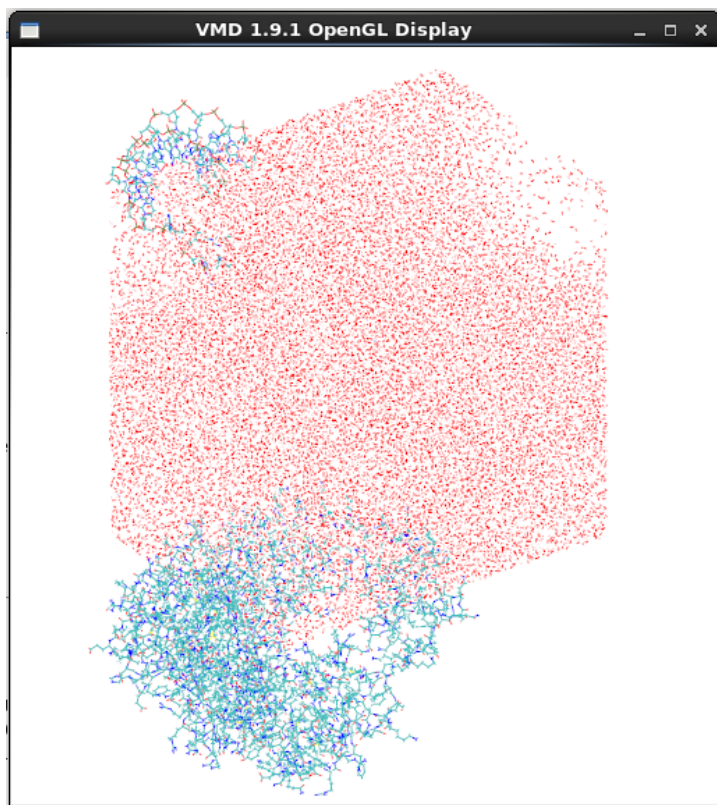
To open VMD and the files straight from the command line, type the following in a Terminal window in the folder that contains your `prmtop` and new, imaged NETCDF trajectory file.

```
vmd -parm7 solvated_complex.prmtop -netcdf new_imaged_trajectory.nc
```

This will load the topology file and NETCDF trajectory all at once. Alternatively, you can load them individually after you open VMD (by just typing `vmd`) by going to File → 'New Molecule...' in the VMD Main window. Click 'Browse...' and locate the topology file (`.prmtop`). The 'Determine file type:' drop-down menu should be AMBER7 Parm, and then click Load. At this point you have only added the topology information (i.e. what atoms are listed in what order). To load the new trajectory, click on 'Browse...' again and load the new, imaged trajectory file (`.nc`) this time. Set 'Determine file type:' to 'NETCDF (AMBER, MMTK)' (if it does not automatically choose it) and click Load. Once you have loaded the trajectory, the VMD Display should look something like this.



Notice that it looks very similar to when we first opened the original `prmtop` and `inpcrd` files in VMD. If we had not centered and imaged the trajectory already, we might have ended up with something like this from the original Amber NETCDF trajectory.



As you can imagine, this is difficult to look at over a trajectory since the two biomolecules are really closely interacting according to the simulation (and the physics), but because of the placement of the unit cell, it appears as though the system complex is not together. This is why we center and image our trajectories prior to analysis and visualization.

Obtaining RMSD, RMSF, Distances and angles

A very easy and common use of *cpptraj* is to obtain RMSD values, as well as specific distance and angle measurements for a trajectory. A *cpptraj* input file to do this sort of analysis (that I typically call *cpptraj_analysis.in*) could look something like this

```
trajin 1L3S_solv_imaged.nc 1 1000 5

rmsd all :1-603 out rmsd_all.dat
rmsd protein :1-603@CA out rmsd_protein.dat
atomicfluct out rmsf_protein.dat @CA,C,N byres

distance 699_629 :426@CA :356@CA out P699_R629_dist.dat
angle ohelix :356@CA :438@CA :427@CA out R629_G711_N700_angle.dat
```

The initial line in *cpptraj_analysis.in* tells *cpptraj* to read in the *imaged* trajectory file (*trajin 1L3S_solv_imaged.nc*). It is important to read in the imaged trajectory file to prevent crazy large distances and RMSD and RMSF values if/when your biomolecule jumps from one side of the unit cell to the other during the trajectory. After the trajectory file name, you can

specify a subset of the trajectory for *cpptraj* to analyze. In this case (1 1000 5), *cpptraj* will read in every 5th frame starting at frame 1 and ending at frame 1000. If you do not put any numbers after the file name, then *cpptraj* will read all frames in the trajectory file. The next two lines deal with calculating the RMSD, but with different reference structures. The first line (`rmsd all :1-603 out rmsd_all.dat`) aligns the solute (residues 1-603 in this example) using all atoms and outputs the results to a file called `rmsd_all.dat`. Meanwhile, the second line (`rmsd protein :1-603@CA out rmsd_protein.dat`) aligns the solute (residues 1-603) using only the α -carbon atoms and outputs the results to a file called `rmsd_protein.dat`. These RMSD output files are two-column text files where the first column is the frame number and the second column is the RMSD value. For example, the first five lines will look similar to what is shown below.

```
1 0.0000
2 1.6384
3 1.8207
4 1.7979
5 1.7674
... ..
```

Frame 1 obviously has a RMSD of 0.0000 because it does not deviate relative to itself. All of the other RMSD values represent their deviation with respect to the coordinates of frame 1. The `rmsd` command in *cpptraj* does allow you to change the reference coordinates to be any frame of the trajectory (but the default is frame 1) or even a structure not in the trajectory (such as another crystal structure or frame from a different simulation). See the user manual for more details on how to do this.

The third line calculates the RMSF per residue, or the root-mean squared fluctuation of each amino acid in the protein. This value is essentially the average RMSD of each individual amino acid over the course of the entire simulation. This analysis is useful as it gives you an idea of which regions of your protein are experiencing the most fluctuation over the course of your simulation. The first command of this line (`atomicfluct out rmsf_protein.dat`) tells *cpptraj* to calculate the RMSF and output these values to the file `rmsf_protein.dat`. The next section tells it to calculate these RMSF values using only the alpha carbons (CA), backbone nitrogens (N), and carbonyl carbon (C) of every amino acid in the protein. Note that the line in the previous RMSD step (`:1-603`) is read by the `atomicfluct` command and is used to indicate which amino acids to calculate the RMSF value for. The last section (`byres`) simply tells it to print the data by each residue. For example, your output file will look like this.

#Res	AtomicFlx
1.000	1.6064
2.000	1.6348
3.000	1.8687
4.000	2.1037
5.000	2.8202
...	...

The first column corresponds to the amino acid number, and the second to the average atomic fluctuation of that residue (aka the RMSF value). As you might have put together from the previous paragraph, this RMSF value is calculated for each amino acid using the average atomic

fluctuation of each alpha carbon, nitrogen, and carbonyl carbon in that particular amino acid across each frame of the simulation.

The last two lines in `cpptraj_analysis.in` are used to calculate a distance and an angle measurement. The first of the two lines measures the distance between the α -carbon of residue 429 (`:426@CA`) and the α -carbon of residue 356 (`:356@CA`) and outputs the results to a file called `P699_R629_dist.dat`. You should note that these residue numbers come straight from the Amber topology file and thus are from the Amber numbering and not the numbering from the original PDB. In fact, the output file is aptly named `P699_R629_dist.dat` because residue 356 in the Amber numbering is actually Arg629 (an arginine at the 629 position) in the original PDB structure, and residue 426 for Amber is actually Pro699 in the original numbering order from the PDB. The re-numbering occurs because Amber always numbers starting at 1, while PDB coordinates are often only portions of a full biomolecule, so the numbering may begin later than one. The `699_629` after `distance` in the distance command is actually just a name for the distance that *cpptraj* uses internally in case you want to refer back to that same distance later in the *cpptraj* input file for some reason. Since it is just a name it doesn't really matter what you call it as long as it helps you determine what distance is being measured.

The distance command can also be used to calculate the center of mass distance between two residues. The difference is subtle. The above command will measure the distance between the two specified atoms. However, if you were to use the masks `:356` and `:426` without specifying an atom, *cpptraj* would instead calculate the mass distribution of each atom in residues 356 and 426, and then measure the distance between the central points of the mass distribution for both residues. Furthermore, if you wanted, you could measure the distance between one specific atom of one residue and the center of mass of the other residue. For example, if you used the masks `:356` and `:426@CA`, *cpptraj* would measure the distance between the central point of the mass distribution of residue 356 and the α -carbon of residue 426.

The format of the `angle` command is clearly similar to the `distance` command. Example shown in `cpptraj_analysis.in` determines the angle between the α -carbon atoms of residue 356 (`:356@CA`), residue 438 (`:438@CA`), and residue 427 (`:427@CA`) and outputs the frame number and angle to a file called `R629_G711_N700_angle.dat`. `R629_G711_N700_angle.dat` is formatted into two columns with column one being the frame number and column two being the measured angle value, just like the output distance file `P699_R629_dist.dat` shown above. The internal *cpptraj* name associated with this angle is `ohelix` and once again has no bearing on the measurement or output file. The command to run *cpptraj* to for this analysis is the same as described for concatenating trajectories, except using this input file instead.

```
$AMBERHOME/bin/cpptraj -p solvated_complex.prmtop -i cpptraj_analysis.in
```

Make sure to run this command in the folder/directory with all of the necessary files (`prmtop`, imaged NETCDF trajectory, and `cpptraj_analysis.in`) and the result would be (in this case) four output files (two RMSD text files, a distance text file, and an angle text file).

Calculating RMSD from a Specific Reference Structure

A standard RMSD calculation as described above will calculate the root-mean-squared deviation of each frame of the inputted trajectory. This deviation is measured by using the starting frame as

a reference structure. However, there are instances where one might instead want to calculate the RMSD of a trajectory from a different reference structure. For example, you might wish to calculate the RMSD of your trajectory in reference to a PDB downloaded from the Protein Data Bank. Alternatively, you might want to compare your full trajectory to a conformer or cluster structure that appears within that trajectory. This process requires a deceptively more involved input file.

```
trajin JIP1_all_imaged_stripped.nc
parm JIP1_Conf1.pdb
reference JIP1_Conf1.pdb parm JIP1_Conf1.pdb [JP1]
rmsd Conf1 ref [JP1] :1-10@CA :1-10@CA out RMSD_JIP1_to_Conf1.dat
```

The first line will simply load in the trajectory file named `JIP1_all_imaged_stripped.nc`. The next two lines work together and will seem slightly counter-intuitive at first. As such, I will explain what each command does individually, and then combine everything afterwards.

The `parm` command will load in your PDB file named `JIP1_Conf1.pdb` and use the residue and atom names in it to set up a parameter topology for that file. In the next line, the `reference` command will load in the atomic coordinates contained within the PDB file you just created a parameter topology for. Within this same line, you'll next use the `parm` command again to apply the parameter topology to the atomic coordinates. At the very end of this line, the term `[JP1]` is simply a flag that tells *cpptraj* to run the previous commands in this line when the flag `[JP1]` is read next. Essentially, this means *cpptraj* will load in the reference PDB when it reads the flag `[JP1]`. Note that this flag is just an arbitrary name. You can give it any name that makes sense for your script.

As I said, that might seem a bit counter-intuitive. If you recall running *tLeap*, you will note that two different file types are outputted. First is a parameter topology file (`prmtop`) containing the all the atomic parameters necessary to properly simulate your protein or protein-ligand system. The second is an input coordinate file (`inpcrd`) that contains the starting XYZ coordinates of each atom within your protein or protein-ligand system. The `parm` and `reference` commands work in similar ways. The `parm` command will load in your PDB and use it to create atomic parameters necessary to properly simulate the structure in that PDB file, whereas the `reference` command will load in your PDB and read the XYZ coordinates of each atom within the structure in that PDB file. The first time you use the `parm` command you are creating a parameter topology. After using the `reference` command to load in the atomic coordinates from your PDB, the `parm` command is used a second time to tell *cpptraj* that the parameter topology created from the specified PDB file should be applied to the coordinates you just loaded with the `reference` command.

Hopefully that makes a bit more sense of things. The last line is where we calculate the RMSD of the trajectory from the desired reference PDB. In this line, the `rmsd` command will calculate the RMSD of each frame of the trajectory inputted with the `trajin` command and internally save the RMSD data into a dataset named `Conf1`. As discussed in the last two paragraphs, the previous line set the necessary commands to load in the reference structure to the flag `[JP1]`. As such, the `rmsd` command is told to calculate the RMSD of the trajectory from the desired reference structure using the command `ref [JP1]`.

We have now successfully told *cpptraj* to calculate the RMSD of the trajectory using our specified reference structure. However, we still need to tell the command **how** to calculate the

RMSD. You will note the mask `:1-10@CA :1-10@CA`. This tells the `rmsd` command to measure the RMSD by looking at only the deviation of the alpha carbons in residues 1-10 of the inputted trajectory in comparison to the alpha carbons in residues 1-10 of the specified reference structure (note that the first instance of `:1-10@CA` points to your inputted trajectory, whereas the second instance points to your reference structure). Finally, the command `out` in this line will save the calculated RMSD into a file we can read named `RMSD_JIP1_to_Conf1.dat`. Finally, all that is left is for you to run the input file using a command like the following example.

```
$AMBERHOME/bin/cpptraj -p JIP1.prmtop -i cpptraj_reference.in
```

With that, you've successfully calculated the RMSD of your trajectory in reference to a specified structure. Note that the `prmtop` file (`JIP1.prmtop`) we load in is the `prmtop` corresponding to the trajectory we load in with the `trajin` command.

The `parm` and `reference` commands can also load in file formats other than PDB files (notably `mol2` files). Furthermore, your reference structure does not necessarily need to have the same number of amino acids as your trajectory file. For example, say the 10 residues of your reference structure are simply a small fragment of a 600-residue protein that you ran simulations on. In this example, let's say these 10 residues actually correspond to residues 158-167 of the protein you ran MD on. You could calculate the alpha carbon RMSD of residues 158-167 in your MD trajectory in reference to residues 1-10 in your reference structure by using the mask `:158-167@CA :1-10@CA` where the first mask is your trajectory and the second is your reference structure. Furthermore, you could also calculate the RMSD using more atoms than just the alpha carbons. If you wanted to also include the backbone nitrogen and carbonyl carbon into your RMSD calculation, you could use mask `:158-167@CA,C,N :1-10@CA,C,N` to do this. The input file for such a calculation might look as follows.

```
trajin iTasser1_25ns_imaged.nc
parm JIP1_Conf1.pdb
reference JIP1_Conf1.pdb parm JIP1_Conf1.pdb [JP1]
rmsd Conf1 ref [JP1] :158-167@CA,C,N :1-10@CA,C,N out RMSD_to_iTas1_Conf1.dat
```

Sorting Frames of a Trajectory by Specific Criteria

In certain cases, it is desired to separate a single trajectory into multiple smaller trajectories meeting some criteria. For example, one might desire to split a trajectory into several smaller trajectories containing only the frames where the system RMSD falls within the range of 0-2 Å, 2-4 Å, 4-6 Å, etc. Sorting a trajectory in such a way can be conducted using the command `outtraj`. This command functions similarly to `trajout` in that it will output a new trajectory from the inputted trajectories. However, no matter where `trajout` appears in your input file, the command will only be run after all other lines in the input file have been run. In other words, `trajout` will always be the last command run, and as such all commands that either calculate data or modify the trajectory will be run before `trajout` is run.

The command `outtraj` differs in that it will always run in the order that it is placed in the input file. Furthermore, it can be used in conjunction with the command `maxmin`. When used in conjunction with `outtraj`, the command `maxmin` is capable of writing a new trajectory using

only the frames of the original trajectory where the values for a certain dataset fall within a defined minimum and maximum range. The following example input file will write a new trajectory using only frames of the original that demonstrate an RMSD in the range of 0 to 2 Å.

```
trajin COVID_6lu7_Darunavir_md001.nc
rmsd COV :1-307@CA out RMSD_COVID_6lu7_ChainA_Dar_COV_Only.dat
outtraj COVID_6lu7_ChainA_Dar_RMSD_max2_min0.nc maxmin COV min 0 max 2.0
```

In the above example, the RMSD analysis is conducted normally. The only difference is in the final line. As discussed, the `outtraj` command will write a new trajectory from the inputted original trajectory (in this case, the file `COVID_6lu7_Darunavir_md001.nc`). The file name `COVID_6lu7_Dar_RMSD_max2_min0.nc` is simply what we are naming this new trajectory. The command `maxmin` tells `outtraj` to include only frames where the RMSD we wrote to the dataset `COV` falls within the range of 0 to 2 Å (`min 0 max 2.0`).

This is generally a simple enough task. As usual, you can run this input file using a command akin to the following example, where `COVID_6lu7_Darunavir_solv.prmtop` is just the `prmtop` file corresponding to the trajectory we load in with the `trajin` command.

```
$AMBERHOME/bin/cpptraj -p COVID_6lu7_Darunavir_solv.prmtop -i cpptraj_sort.in
```

It should be noted that the `outtraj` and `maxmin` commands are not limited to sorting a trajectory by RMSD. As long as an analysis produces a by-frame or over-time dataset, one can sort and create a new trajectory using that data. For example, if I were interested in only the frames where a certain bond angle falls within the range of 120 to 140°, I would use the following input file.

```
trajin 1L3S_solv_imaged.nc
angle ohelix :356@CA :438@CA :427@CA out R629_G711_N700_angle.dat
outtraj 1L3S_356_438_427_ang_max140_min120.nc maxmin ohelix min 120 max 140
```

RMSD and angle ranges are not all that one can sort a trajectory by. As stated before, any by-frame or over-time dataset can be used to output a new trajectory containing only the frames that fall within a certain numerical range. The only restriction is that the analysis must be performed using a `cpptraj` command. For example, it would not be possible to create a new trajectory using only frames within a certain MMGBSA $\Delta\Delta G$ energy range, as MMGBSA energy is calculated using `mmpbsa.py` and not `cpptraj`.

Calculating the Surface Accessible Surface Area (SASA) of a Complex

To calculate the contact surface area, we can utilize the `molsurf` command in `cpptraj`. To do this you would have to first calculate the total surface area of each monomer. In my case, one monomer would be DNA polymerase and the other would be the DNA. Then, you would calculate the surface area of the complex (DNA polymerase + DNA). We can then use the following relationship to calculate the contact surface area: DNA poly + DNA - (DNA poly+DNA).

The `molsurf` command in your `cpptraj_analysis.in` file would look like the following:


```

trajin DNApoly_DNA_complex.pdb #solvent and ions are removed beforehand

molsurf DNApoly :28-566 out DNApoly_sasa.dat
molsurf DNA :1-27 out DNA_sasa.dat
molsurf complex :1-566 out complex_sasa.dat

```

Note: Make sure that the file that you `trajin` is free of solvent and ions.

An additional explanation of this is described
here: <http://archive.ambermd.org/201508/0390.html>

If you would like to find the SASA of the hydrophilic and hydrophobic residues, you would have to use VMD. Those instructions can be found in the VMD manual.

Hydrogen Bonding Analysis

An understanding of the hydrogen bonds occurring in a protein can aid in the understanding of protein folding as well as the interactions of the protein with a bound ligand. *Cpptraj* can perform a detailed analysis of all hydrogen bonding occurring in the system where either F, O, or N atoms are the H-bond acceptor. Four types of hydrogen bonds can be analyzed. The first type are all solute-solute hydrogen bonds. More specifically, this means only the hydrogen bonds that are occurring between the atoms in the amino acids in your protein or the atoms in your ligand. The second type are the hydrogen bonds occurring between only the amino acid backbones of the protein or protein-ligand system. These are referred to as solute-solute backbone hydrogen bonds. The third type are solute-solvent hydrogen bonds, or all hydrogen bonds occurring between your protein and solvent atoms. In this case, an atom in your protein will be the donor and an atom in the solvent will be the acceptor, or vice versa. The fourth type are solute-solvent-solute bridge hydrogen bonds. These are hydrogen bonds occurring in your system where one region of the protein is hydrogen bonding to a water molecule, and that same water molecule is hydrogen bonding to another region of your protein.

In order to conduct proper H-bond analysis, *cpptraj* will need to calculate all four types of hydrogen bonds occurring in your system. In your input file, you will first want to use the `trajin` command to load in the solvated trajectory files you wish to conduct H-bond analysis on. Next, you will need to input your H-bonding command. It should look something like the following.

```

hbond all :1-12 out hbvtime.dat solventdonor :WAT solventacceptor :WAT@O
avgout All_UU_avg.dat solvout All_UV.dat bridgeout All_bridge_avg.dat

```

Some key terms need to be discussed here. UU indicates all solute-solute (protein-protein) hydrogen bonds, UV indicates a solute-solvent (protein-water) hydrogen bond, and Bridge indicates solute-solvent-solute (protein-water-protein) hydrogen bonds. In all systems, your water molecules will be your solvent. This makes your protein or protein-ligand system your solute. So, the output file `All_UU_avg.dat` includes all of the hydrogen bonds occurring in only your protein or protein ligand system. `All_UV_avg.dat` includes all of the hydrogen bonds

occurring between only protein and solvent molecules. Lastly, `All_bridge_avg.dat` includes all of the hydrogen bonds occurring where one region of your protein is hydrogen bound to a water molecule, and that same water molecule is hydrogen bound to another region of the protein.

To break down the command, the flag `hbond` tells *cpptraj* to run hydrogen bonding analysis. The keyword `all` tells this command to print this data into a dataset named `all`. The mask `:1-12` tells *cpptraj* to analyze all hydrogen bonds occurring between all of your residues (in this example, the protein only has 12 amino acids). The next command `out hbvtime.dat` prints out a general data file containing the total number of hydrogen bonds occurring at each individual frame of the simulation. This file specifies the number of each type of hydrogen bond occurring, and is printed in the following format.

#Frame	all[UU]	all[UV]	all[Bridge]	all[ID]
1	0	22	1	1273 (9+11+),
2	1	17	1	2188 (6+8+),
3	0	15	0	None
4	1	21	0	None
5	2	21	1	1563 (9+10+),
...	...	...	...	...

In the first column, we see the frame number (to convert to time units, remember that there are 10 frames per nanosecond). The second column is the number of hydrogen bonds occurring between all of the atoms in the protein at that frame. The third column is the number of hydrogen bonds occurring between protein atoms and solvent atoms at that frame, where a protein atom is the acceptor and a solute atom is the donor, or vice versa. The fourth column is the number of hydrogen bonds occurring at that frame where a protein atom is hydrogen bound to a water molecule that is hydrogen bound to another protein atom. Lastly, the final column is unimportant to us. This simply is an ID number used by *cpptraj* to describe the H-bonds that are occurring.

The command `soventdonor :WAT solventacceptor :WAT@O` are pretty intuitive. First, it tells *cpptraj* that all water molecules in the system will donate hydrogen bonds. Next, it tells *cpptraj* that specifically the oxygen atoms in the water molecules will be hydrogen bond acceptors. Note that in the first command, it isn't necessary to tell *cpptraj* that the hydrogen bond donors in the water molecules are the hydrogen atoms. For obvious reasons, hydrogen atoms will always be the hydrogen bond donor. However, as mentioned earlier, the hydrogen bond donor could be either F, O, or N atoms, and as such it is necessary to tell *cpptraj* that the hydrogen bond acceptor in the water molecules is the oxygen atom.

Next, the command `avgout All_UU_avg.dat solvout All_UV.dat bridgeout All_bridge_avg.dat` produces all of the individual data files describing each type of hydrogen bond in the system. As mentioned above, the output file `All_UU_avg.dat` includes all of the hydrogen bonds occurring in only your protein or protein ligand system, the file `All_UV_avg.dat` includes all of the hydrogen bonds occurring between only your solvent molecules, and the file `All_bridge_avg.dat` includes all of the hydrogen bonds occurring between your protein and the solvent molecules. These output files all follow the same general format.

#Acceptor	DonorH	Donor	Count	Frac	AvgDist	AvgAng
LYS_2@O	SolventH	SolventDnr	18034	0.9017	2.7626	159.6981

LEU_9@O	SolventH	SolventDnr	16716	0.8358	2.7622	158.7865
THR_6@O	SolventH	SolventDnr	16547	0.8274	2.7707	159.0046
SolventAcc	ARG_3@HH11	ARG_3@NH1	9568	0.4784	2.8623	155.9379
SolventAcc	LYS_2@HZ3	LYS_2@NZ	9370	0.4685	2.8503	155.2951
SolventAcc	LYS_2@HZ2	LYS_2@NZ	9253	0.4627	2.8499	155.0804
...	...	...	...	...	...	...

In the first column, we see the atom that is accepting a hydrogen bond (either a F, O, or N atom). In the event that this atom is part of the protein, it will give us the 3-letter code, residue number in the protein, and PDB atom name of the atom in that amino acid that is accepting the hydrogen bond. If a solvent molecule is accepting a hydrogen bond, it will simply state that it is a solvent acceptor. The next column tells us which hydrogen atoms is the hydrogen bond donor. Like with the acceptor column, if the hydrogen atom donating the hydrogen bond is part of the protein, it will give us the 3-letter code, residue number, and PDB atom name of the atom in that amino acid that is accepting the hydrogen bond. If the atom donating the hydrogen bond is part of the solvent molecule, it will simply state that it is a solvent hydrogen. The next column tells us the polarizing atom the donor hydrogen atom is covalently bound to (in the example R-O1-H---O2-R, O1 is the polarizing atom the donor hydrogen atom is bound to). Next, the count column tells us how many frames of the simulation this particular hydrogen bond persists for and the fraction column tells us what fraction of the simulation this accounts for. Lastly, the final two columns detail characteristics of the hydrogen bond itself, including the average distance between the donating hydrogen and the accepting atom as well as the angle between the polarizing atom the donating hydrogen is bound to, the donating hydrogen itself, and the hydrogen bond acceptor.

After this command has successfully run, we will need to run another command in order to analyze the hydrogen bonds occurring between only the atoms in the amino acid backbones of our protein. This command should be put in the same input file, directly below the previous `hbond` command. It should look something like the following.

```
hbond Backbone :1-12@C,O,N,H avgout BB_hbond_avg.dat series uuseries
BB_hbond.gnu
```

As before, the first command `hbond` tells *cpptraj* to run a hydrogen bond analysis calculation, whereas `Backbone` tells it to write this data to a dataset named `Backbone`. The command `:1-12@C,O,N,H` tells this analysis to look at the hydrogen bonds forming between all 12 residues of this protein (as stated before, the protein used in this example has 12 amino acids in total), focusing on only the carbon, oxygen, nitrogen, and hydrogen atoms (the PDB names of the atoms in the amino acid backbones).

Also identical to the previous command, `avgout BB_hbond_avg.dat` writes out the file `BB_hbond_avg.dat` containing all of the information on the hydrogen bonds occurring between the amino acid backbones. This file is in the same format of the previous three files. Lastly, the line `uuseries BB_hbond.gnu` simply tells *cpptraj* to print out a file detailing the number of backbone hydrogen bonds per frame in a format graphable by *gnuplot*.

From here, four more commands will need to be added into your input file to successfully get all of the hydrogen bonding data. This next set of commands creates 4 files detailing the raw number of each type of hydrogen bonds occurring per frame of the simulation. These are highly

useful for graphical analysis of the hydrogen bonding behavior of the system. The commands that create this data are as follows.

```
create All_UU_nhbvtime.dat all[UU]
create BB_UU_nhbvtime.dat Backbone[UU]
create All_UV_nhbvtime.dat all[UV]
create All_Bridge_nhbvtime.dat all[Bridge]
```

As you can see, these all use the same `create` command. The only difference between them is which data is used to create the files. For example. The first command uses the UU data (solute-solute or protein-protein hydrogen bonds) from the dataset named `all`. The second command uses the UU data (solute-solute or protein-protein hydrogen bonds) from the dataset named `Backbone`. The third command uses the UV data (solute-solvent or protein-water hydrogen bonds) from the dataset named `all`. The final command uses the Bridge data (solute-solvent-solute or protein-water-protein hydrogen bonds) from the dataset named `all`.

Note that the dataset names were created in the first two commands. For example, the command `hbond all` wrote all data to the dataset `all`, whereas `hbond Backbone` wrote all data to the dataset `Backbone`.

From here, there is only one more command that needs to be added to your input file. This calculates the RMSD of your amino acid backbones, using the first frame of your simulation as the reference point. This is not a part of hydrogen bond analysis and is the same exact command as normal RMSD. It is included simply as a reference.

```
Rms BBrmsd :1-12@C,CA,N out BBrmsd.dat
```

This covers everything you need for running hydrogen bonding analysis. Your completed input file should look something like the following.

```
trajin Protein_ligand_solv_md001.nc
trajin Protein_ligand_solv_md002.nc
trajin ...

hbond all :1-12 out hbvtime.dat solventdonor :WAT solventacceptor :WAT@O
avgout All_UU_avg.dat solvout All_UV.dat bridgeout All_bridge_avg.dat

hbond Backbone :1-12@C,O,N,H avgout BB_hbond_avg.dat series uuseries
BB_hbond.gnu

create All_UU_nhbvtime.dat all[UU]
create BB_UU_nhbvtime.dat Backbone[UU]
create All_UV_nhbvtime.dat all[UV]
create All_Bridge_nhbvtime.dat all[Bridge]

rms BBrmsd :1-12@C,CA,N out BBrmsd.dat
```

From here, the starting the job is as simple as running the following command in your terminal.

```
$AMBERHOME/bin/cpptraj -p Protein_ligand_solv.prmtop -i cpptraj_Hbond.in
```

If your simulations contain a ligand, you can create another data file containing only ligand hydrogen bonding interactions using the following command.

```
sed -n '/LIG/p' All_UU_avg.dat > Ligand_Only_UU_avg.dat
```

This `sed` command will open the file `All_UU_avg.dat` containing all the hydrogen bonds occurring in only your protein or protein ligand system and print all lines containing the residue `LIG` to a new file named `Ligand_Only_UU_avg.dat`. This allows for easier analysis of hydrogen bonds occurring with the ligand.

It should be noted that the term `LIG` is applicable as long as you used the flag `LIG` to load in your `ligand_w_H.mol2` file in *tLeap*. If you used a different flag, just substitute `LIG` with that flag. This is because *tLeap* assigns your ligand an amino acid code identical to this flag when creating your `prmtop` and `inpcrd` files.

If you do not have a ligand and are instead interested in the hydrogen bonding of a specific amino acid, you can simply substitute `LIG` with the amino acid of interest. For example, if I were interested in a serine at position 137, I would substitute `LIG` with `SER_137`.

Other Analyses

The previous list was not an exhaustive list of all analyses *cpptraj* is capable of, but simply a description of the most commonly used ones. There are many, many other possible analyses available, ranging from clustering jobs that group together frames belonging to a common average structure to calculations of the grid water density around a protein. The full list can be found here: <https://amber-md.github.io/cpptraj/CPPTRAJ.xhtml>

Plotting data using gnuplot

Many beginning MD users attempt to use Microsoft Excel for graphing their results. This can be done for MD research, BUT Excel has many limitations and I advise against it. Excel has difficulties with large amounts of data (which we often generate with our simulations) and has its own ideas about what our graphs should look like, requiring countless hours fixing our graphs to make them look appropriate for scientific research/journals. There are many other graphing programs available, but my preference is to use *gnuplot*. Unlike Excel or Xmgrace or similar graphing programs, *gnuplot* has no graphical user interface (GUI). Instead, *gnuplot* is used from the command line and can read input files to make plots and save images of these plots.

Since I have already introduced how to obtain RMSD values for a trajectory using *cpptraj* (see above), I will use that data as an example of results that can be plotted using *gnuplot*. As a reminder, the data file (for later reference I will call `rmsd.dat`) looks like this

```
1 0.0000
2 1.6384
3 1.8207
4 1.7979
5 1.7674
... ..
```

And this continues to end of the simulation. Personally (and for any publication-quality figures), you should plot the RMSD as a function of the simulation time not as a function of the frame number. This can be done in one step on the command line using two commands called *cat* and *awk*. For a file called `rmsd.dat` that contains all of your RMSD values (or distance measurements or whatever data you want plotted), type the following

```
cat rmsd.dat | awk '{print 0.1*$1, $2}' > new_rmsd.dat
```

The *cat* command simply prints out every line in `rmsd.dat` one-by-one. The solid vertical line (`|`) before *awk* is called a ‘pipe’. Pipes are used when you want the output of one command to immediately be used as input for the following command. So as each line from `rmsd.dat` is printed, *awk* reads in that line and prints the value in column one multiplied by 0.1 (`0.1*$1`), and prints the value in column two without modifying it (`$2`). In this example, the frames are printed out every 100 ps (or 0.1 ns), so to plot the results as a function of the simulation time in units of ns, we multiplied by 0.1. Finally, the greater-than symbol (`>`) prints the output from *awk* into a new file called `new_rmsd.dat`. So the first five lines of `new_rmsd.dat` now looks like this

```
0.1 0.0000
0.2 1.6384
0.3 1.8207
0.4 1.7979
0.5 1.7674
```

Notice how column one has been multiplied by 0.1 while column two is unchanged from the original file. Now we are ready to prepare the *gnuplot* input file that will tell *gnuplot* to plot this data and make an image of it. To do this, the *gnuplot* input file (which I will call `gnuplot.in`) would look like this

```
set term png size 2000, 1200 font '/usr/share/fonts/liberation/LiberationSans-
Regular.ttf, 40'

set output "RMSD_solute.png"
set title "RMSD\nSolute"
set xlabel "Time (ns)"
set ylabel "RMSD (Å)"
set nokey
plot 'new_rmsd.dat' w l lw 3
```

The first line sets the *gnuplot* terminal to ‘png’ (`set term png size 2000, 1200`), setting the stage for us to save an image of the plot in png format. The end of the first line gives the path on your computer to a specific font that we want to use (`LiberationSans-Regular.ttf`) in size 40 font for all of the text on the plot. You can exclude this and *gnuplot* will just use its default font (which isn’t very nice), or you can change the font to something else you would prefer. The second line defines the file that we want to save the image to using *gnuplot*. In this case, when we run *gnuplot* it will save the plot to a file called `RMSD_solute.png`. The third line tells *gnuplot* the title to put at the top of the plot. In this case, we want the top line to read ‘RMSD’ and underneath that on a new line (`\n`) will be the word ‘Solute’. The next two lines define the x- and y-axes labels as ‘Time (ns)’ and ‘RMSD (Å)’, respectively. In this case since we are only plotting one set of data, we have chosen to not place a legend/key on the plot (`set nokey`), but if

you want the key to be printed on the figure simply tell *gnuplot* where you want it to be placed in the plot (for example, if you want the key to be in the bottom right-hand corner of the plot, instead of `set nokey` you would have `set key bottom right`). Next, we actually tell *gnuplot* to plot a data file (`plot 'new_rmsd.dat'`) with lines (`w l`) using a line width of 3 (`lw 3`). You can also change the line color by adding `lc 2` to the end of that line to change the data to line color 2 (different numbers correspond to different colors in *gnuplot*; you may have to play with the different colors to find one you prefer, but I find that in general the colors are chosen fairly well by default). If you wanted to add another data set to the same plot (for example, placing the RMSD values for both the entire solute and just the protein), then you would simply place a comma after specifying the end of the first file and then add the details for another file, as demonstrated below.

```
plot 'new_rmsd.dat' w l lw 2 ti 'Solute', 'new_rmsd_protein.dat' w l lw 2 ti 'Protein'
```

Notice that a line-width of 2 (`lw 2`) was chosen here for both data sets. There is a comma separating the details for both `new_rmsd.dat` and `new_rmsd_protein.dat`. The other change is we have added a title (for example, `ti 'Solute'`) for each data set. We called the first data set `Solute` and the second data set `Protein` accordingly. This is how the data sets would be labeled in the key/legend if we decided to place the key on the plot.

If you have a file that contains many columns of data (for example, an MMGBSA output file), it is also possible to tell *gnuplot* to graph only certain columns of data within that file. Let's say that the data we're interested in is located in columns 1 and 8 of this data file. Simply adding in the term `using 1:8` to the `plot` line would tell *gnuplot* to graph only the data in these columns. A full line of this sort would look like the following example.

```
plot 'MMGBSA_edit.csv' using 1:8 w l lw 2 title 'MMGBSA'
```

Another helpful thing to know is that you can also control the scale for the x and y axes. *gnuplot* will automatically set the scale for the x and y axes, but sometimes it is necessary to define this yourself. In that case, you can simply add a line (or two) for the x- and/or y-axis to set the range. For example,

```
set xrange [0:1000]
set yrange [0:30]
```

Would set the range of the x-axes from 0 to 1000 and the range of the y-axis from 0 to 30, but make sure to add these lines to the *gnuplot* input file BEFORE you plot the data or it will not be incorporated into the saved image.

To invoke *gnuplot* to read in and perform all of the tasks we described in the `gnuplot.in` file, simply type in a Terminal window

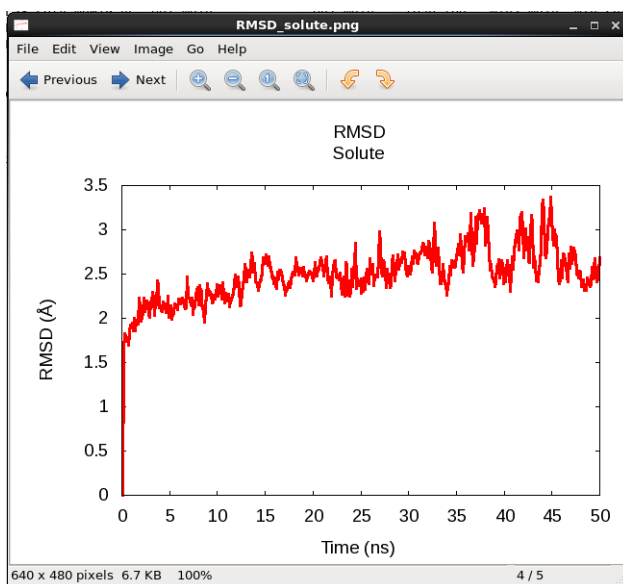
```
gnuplot gnuplot.in
```

Obviously, make sure that the `gnuplot.in` and data file that you are plotting are in the folder/directory you are running this command. If all goes well, unix will print nothing to the screen (and you should celebrate). Although nothing was printed to the screen, *gnuplot* was executed and you should have now a `RMSD_solute.png` image file in your directory now. You can

go to Places and open up a finder window and browse to the location on your computer where that new png image file exists, or you can just use the *eog* command in your Terminal window.

```
eog RMSD_solute.png
```

This should bring up a window showing your RMSD plot.

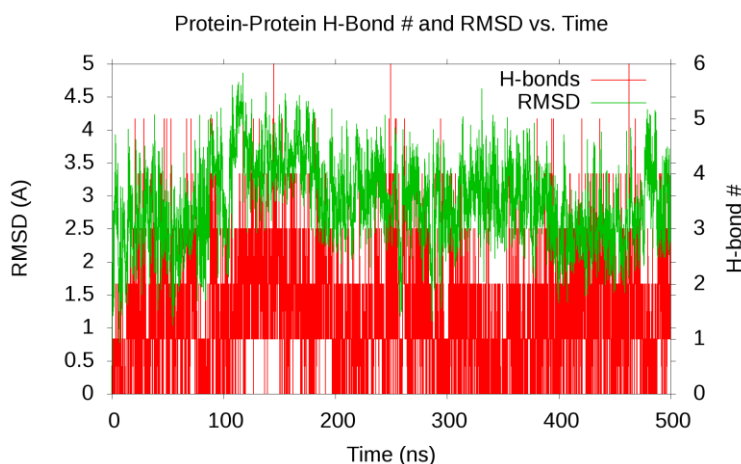


In the example above, the original trajectory had a total of 500 frames (or 50 ns worth of simulation time). There are some more advanced things you can do with *gnuplot* as well. For example, say you want to make a graph of RMSD and hydrogen bonds over time. This type of graph would require two different y-axes. You can do that pretty easily using an input file in the following format.

```
set term png size 2000, 1200 font
'/usr/share/fonts/liberation/LiberationSans-Regular.ttf, 40'
set output "UU_Hbond_RMSD_JIP1_500ns.png"
set title "Protein-Protein H-Bond # and RMSD vs. Time"
set xlabel "Time (ns)"
set ylabel "RMSD (Å)"
set y2label "H-bond #"
set ytics
set y2tics
set tics out
set key top right
plot 'All_UU_EDIT.dat' w l lw 2 axes xly2 ti "H-bonds",
'JIP1_Capped_500ns_BBrrmsd_EDIT.dat' w l lw 2 axes xly1 ti "RMSD"
```

In this input file, we specify that there are two y-axes by labeling y-axis 1 using the command `set ylabel` and y-axis two by using the command `set y2label`. Then, you need some additional lines to set the individual tic marks on the two different y-axes (these being `set ytics`, `set y2tics`, and `set tics out`). Lastly, all you have to do is specify in the plot

command which dataset goes on which axis using the flags `axes x1y1` and `axes x1y2`. Then you can produce a graph that looks like the following.



It's also possible to use *gnuplot* to create bar graphs. Bar graphs are commonly used to display the percent occurrence of hydrogen bonds from *cptraj* analysis or to display the per residue free energy decomposition data from MMGBSA calculations. This is a simple matter of adding in an additional line to set the width of the bars in the graph. Typically, the bar width is set to 0.5.

```
set boxwidth 0.5
```

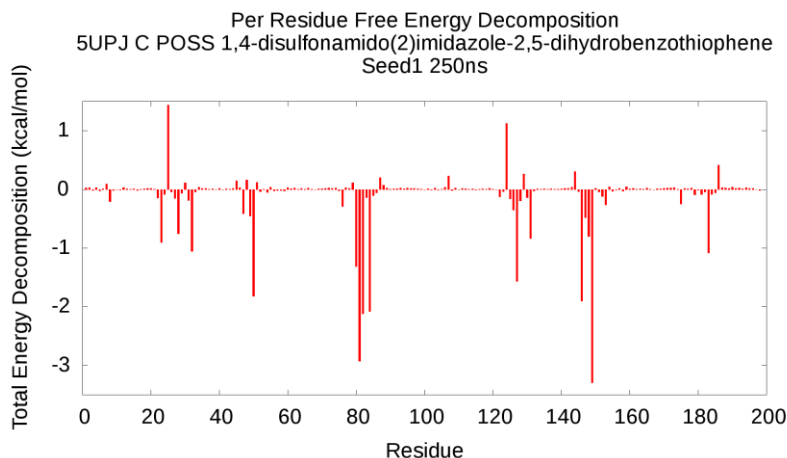
Next, an additional line should be added to set the fill style of the bars. Typically, a solid fill style is used.

```
set style fill solid
```

Lastly, in the `plot` line you would specify that you are plotting the data as a bar graph by using the term `with boxes`. This full line would look like the following.

```
plot "decomp.dat" with boxes
```

This would result in a graph that looks like the following example.



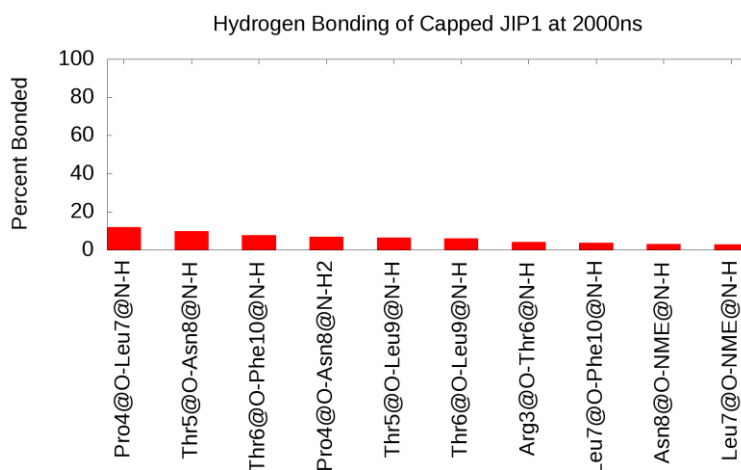
Lastly, there are two other commands related to bar graphs that I want to make note of briefly. If you had wanted to give non-numerical labels to your x-axis data, you could use the command `set xtics` to give each bar the label you want. That command would be formatted like the following.

```
set xtics("Label 1"    1.000, "Label 2"    2.000, "Label 3"    3.000)
```

From there, you could add in another line to rotate the x-axis labels to ensure they don't write over each. This line would look as follows

```
set xtic rotate by 90
```

A successfully created graph of this sort would look as follows.



I have only given you a brief synopsis of how to use *gnuplot* and an even briefer list of the many, many things *gnuplot* can do for you. Since *gnuplot* is free and open source, there are many online resources to help you learn the different aspects of *gnuplot*. A quick 'google' search should help you figure out how to do anything that *gnuplot* can do that I have not already explained.

MMPBSA.py & MMPBSA.py.MPI

Another highly used analysis tool available with AmberTools is a python program called *MMPBSA.py* that is used to calculate the free energies from MD trajectories (you can read the *MMPBSA.py* publication in JCTC here: <http://pubs.acs.org/doi/abs/10.1021/ct300418h>). *MMPBSA.py* is most often used to calculate the binding free energies of protein-ligand complexes, which is especially useful for drug design. But the analysis package also has many other calculations it can do, including residue free energy decomposition, alanine scanning, and entropy calculations. This package runs in serial. This means the calculation will run on only one CPU core. In order to improve calculation runtime, you should always run *MMPBSA.py.MPI*, a parallelized version of *MMPBSA.py* that runs across multiple CPU cores instead of just one. The online tutorial for using *MMPBSA.py* and *MMPBSA.py.MPI* can be accessed using the link below

http://ambermd.org/tutorials/advanced/tutorial3/py_script/

Typically, *MMPBSA.py.MPI* is run according using a GB calculation instead of a PB calculation. GB calculations tend to produce slightly lower delta binding energy values, but also are significantly faster than PB. In this case, we refer to it as an MMGBSA calculation. The following script shows how a typical MMGBSA calculation is conducted on a system with a ligand. This script details both a GB delta binding energy calculation ($\Delta\Delta G$ energy of the interactions between the ligand and the protein) and a per res decomp calculation (the per amino acid breakdown of the $\Delta\Delta G$ energy).

```
#!/bin/csh
module load amber/18

# Create MMGBSA Input file
rm mmgbsa.in
cat > mmgbsa.in << EOF
Input file for running GB
&general
    verbose=1, interval=1, startframe=1
/
&gb
    igb=5, saltcon=0.150
/
&decomp
idecomp=1, print_res="1-759"
dec_verbose=1,
/
EOF

# Run MMGBSA
mpirun -np 20 python $AMBERHOME/bin/MMPBSA.py.MPI -O -i mmgbsa.in -o
MMGBSA_DBind.dat -do MMGBSA-Decomp.dat -sp Protein_Ligand_solv.prmtop -cp
Protein_Ligand.prmtop -rp Protein.prmtop -lp Ligand.prmtop -y *md0*nc -eo
MMGBSA_DBind.csv
```

```
# Create delta binding energy file
cp MMGBSA_DBind.csv EDIT_MMGBSA_DBind.csv

# Create per res decomp energy file
cp MMGBSA_Decomp.dat MMGBSA_Decomp_EDIT.dat

# Modify per res decomp energy file
sed -i '1,8d' MMGBSA_Decomp_EDIT.dat #Remove first 8 rows
sed -i '760,+8000000d' MMGBSA_Decomp_EDIT.dat #Remove rows 760 and onwards
sed -i 's/,/ /g' MMGBSA_Decomp_EDIT.dat #Replace all commas with spaces
cat MMGBSA_Decomp_EDIT.dat | awk '{print $1 $2, $2, $18}' > MMGBSA_Decomp_
_EDIT_GRAPH.dat #Create new .dat file with desired columns

# Modify delta binding csv file
sed -i '1d' EDIT_MMGBSA_DBind.csv # Remove first row
sed -i '1d' EDIT_MMGBSA_DBind.csv # Remove second row
sed -i 's/,/ /g' EDIT_MMGBSA_DBind.csv #Replace all commas with spaces
cat EDIT_MMGBSA_DBind.csv | awk '{print 0.1*$1, $2, $3, $4, $5, $6, $7, $8}
' > Delta_MMGBSA_DBind.csv
```

A second thing to note is that this script assumes you have a ligand. The command to run an MMGBSA calculation looks slightly different without a ligand (as will be noted soon).

Like with our other scripts, the first step is to create the `mmgsa.in` file. The first `&general` section indicates the general settings of the calculation. The flag `verbose=1` simply indicates the amount of data outputted (I always use 1, as this contains everything we need). The second is the interval at which the frames are read (you may be able to infer that `interval=1` means that every frame is read). The last flag quite obviously indicates which frame number to start at. You could also specify an end frame if you wanted to using the flag `endframe=`.

The next `&gb` section indicates the settings for the generalized-Born calculation. This is the job that outputs the delta binding energy data. The flag `igb=5` indicates which version of the generalized-Born calculation is used (always use 5). The second flag `saltcon=0.150` indicates the expected experimental salt concentration of the solvent. In many tutorials you'll find people using either 0.100 or 0.150. I typically use 0.150.

The last `&decomp` section indicates the settings for the per-res decomp calculation. The first flag `idecomp=1` indicates which calculation type to use (always use 1). The second `print_res="1-759"` indicates which amino acids to calculate the individual energy contribution for. In this example, my protein has 759 amino acids, and as such I tell it to calculate for all 759 amino acids. The last `dec_verbose=1` flag is the verbosity of the decomp calculation. Like with the GB calculation, I also use a value of 1 for this.

Now that we've discussed the input file, we can talk about the script. The first section of this script is the command line that runs the MMGBSA calculation. The first flag `mpirun -np x` tells the job to run on multiple CPUs, and to use `x` number of processors. The table below shows how many processors are available on each workstation and how fast they are. Running with more processors will always increase your calculation speed. Generally, a newer CPU will always be faster than an older one, regardless of the speed the CPU runs at (listed in GHz). However, when the CPUs are from the same hardware generation, a higher GHz CPU clock will always run faster than a lower clock. Regardless of CPU clock speed however, it is always best to run on

more processors rather than faster ones. For example, 10 CPUs on Thais will be slower than 20 on Cooper regardless of the CPU clock speed.

Workstation	CPU Name	# of CPUs	CPU Clock Speed
Thais	Intel Xeon W-2133	12	3.6 GHz
Adam	Intel Xeon W-2155	20	3.3 GHz
Anna	Intel Xeon W-2155	20	3.3 GHz
Cooper	Intel Xeon E5-2630v4	20	2.2 GHz
Evan	Intel Xeon E5-2630v4	20	2.2 GHz
Sarah	Intel Xeon E5-2630v4	20	2.2 GHz
Marie	Intel Xeon E5-2630 v4	20	2.2 GHz
Irene	Intel Xeon E5-2630 v4	20	2.2 GHz
Franklin	Intel Xeon E5-2620 v3	24	2.4 GHz
Elion	Intel Xeon E5-2620 v3	24	2.4 GHz
Hamilton	Intel Xeon E5-2620 v3	24	2.4 GHz
Mayer	Intel Xeon E5-2620 v3	24	2.4 GHz
Newton	Intel Xeon E5-2620 v3	12	2.1 GHz
Kollman	Intel Xeon E5-2620 v2	12	2.1 GHz
Pople	Intel Xeon E5-2620 v2	24	2.1 GHz
Pauli	Intel Xeon E5-2630	12	2.3 GHz
Dirac	Intel Xeon E5-2630	12	2.3 GHz
Prota	Intel Xeon X5650	12	2.67 GHz
Roald	Intel Xeon X5650	12	2.67 GHz
Bohr5	Intel Xeon E5620	8	2.4 GHz

The second line `$AMBERHOME/bin/MMGBSA.py.MPI` is the program file location for running an MMGBSA calculation on multiple CPUs. The next section `-O -i mmgbsa.in -o MMGBSA_Dbind.dat -do MMGBSA_Decomp.dat` tells the program to override all files with identical names, read in the `mmgbsa.in` file, and output a general GB calculation output file called `MMGBSA_Dbind.dat` and a specific pre res decomp output file called `MMGBSA_Decomp.dat`.

The next section is very sensitive. The next flag `-sp` tells MMGBSA to load in your solvated complex prmtop file. This is the `solv.prmtop` file created at the end of *tleap* after counterions and water molecules have been added. This is the same prmtop file used for initiating an MD job. The next flag `-cp` tells MMGBSA to load in your unsolvated and ion-free complex prmtop. This prmtop is made by *tleap* when the `protein.pdb` and `ligand.mol2` files have been combined, and before ions and water molecules are added. The next flag `-rp` tells MMGBSA to load in your receptor (or protein) prmtop. This is the prmtop *tleap* creates using only the unsolvated protein. The last flag `-lp` tells MMGBSA to load in your ligand prmtop. This is the prmtop *tleap* creates using only the unsolvated ligand. It is very important that all of these prmtop files are loaded in this order, and that the correct prmtop files are loaded in for each step.

The next flag `-y` tells MMGBSA which trajectory files to load in. In this example, every 25ns chunk NETCDF (.nc) file outputted from the MD step is loaded in (`*md0*nc` tells it to load in all files that contain the characters `md0` at some point in the middle of the name and the characters `nc` at the end of the name). This is also a step where errors can commonly occur.

The last flag `-eo MMGBSA_DBind.csv` is pretty simple. This is simply the per frame breakdown of the average delta binding energy found in `MMGBSA_Dbind.dat`. Graphing this allows us to have a better understanding of how the energy changes throughout the course of the simulation.

Note that if you ran MD on an apo system without a ligand, you will not have a receptor or a ligand. In this case, you'll only be loading in your solvated complex prmtop using the `-sp` flag and your unsolvated complex prmtop using the `-cp` flag. Note that the complex prmtop in this case is just the unsolvated protein prmtop. In this case, your command line will look like the following example.

```
mpirun -np 20 python $AMBERHOME/bin/MMPBSA.py.MPI -O -i mmgbsa.in -o
MMGBSA_DBind.dat -do MMGBSA-Decomp.dat -sp Protein_Ligand_solv.prmtop -cp
Protein_Ligand.prmtop -y *md0*nc -eo MMGBSA_DBind.csv
```

The last sections of the script simply prepare `MMGBSA_Dbind.csv` and `MMGBSA-Decomp.dat` for graphing. First, copies of both files are created so that the original isn't overwritten. This prevents us from losing our data in case something goes wrong.

The next section first focuses on modifying `MMGBSA-Decomp.dat`. First, we need to delete the first 8 title lines as these are simply non-graphable descriptor lines. Next, we delete all rows following the last amino acid in your system. This number will vary. In this example, I have 759 amino acids, and as such I want to delete lines 760 and onwards (note that the number 800000 is just an arbitrarily large number to ensure that all lines afterwards are deleted). Lastly, data columns 1, 2, and 18 are copied from the data file and written to a new file. This is because the final data file includes a lot of information that we generally aren't interested in. Column 1 is the residue number, 2 is the residue 3-letter code and number in the protein, and 18 is the average energy contribution of each amino acid to the delta binding energy. From here, this new file is ready for graphing.

The last section modifies `MMGBSA_Dbind.csv`. First, we delete the first two lines as these are just title lines. Next, we change all of the commas in the file (csv stands for comma separated values) to spaces. Lastly, we write columns 1 (multiplied by 0.1 as we are converting from frames to nanoseconds), 2, 3, 4, 5, 6, 7, and 8 into the new .csv file. This file is almost ready for

graphing if you have a ligand. If you don't have a ligand, you're good to start graphing. From here, open up the file and navigate through it until you the number in the first column is equal to the nanoseconds you've ran your simulation for. After this point, you'll notice another section of data. This is because `MMGBSA_Dbind.csv` is split into 4 different sections (if you ran without a ligand, there is only one section). The first section is the delta binding energy (also called the ΔG energy). The second is the contribution of the solvated complex to the delta binding energy. The third is the contribution of the unsolvated receptor to the delta binding energy. The fourth is the contribution of the unsolvated ligand to the delta binding energy. We're only interested in the delta binding energy. As such, just delete these columns, and then you are good to graph.

On Unusual Solvent and Ions

An important thing to note is that MMGBSA by default will strip all water molecules and the following ions from the solvated complex when attempting to read the unsolvated complex: Cl^- , ClO^- , Cs^+ , IB^- , K^+ , Li^+ , Mg^{2+} , Na^+ , Rb^+ , and F^- . This process is what allows MMGBSA to understand what solvent (usually water) and ions are present in the system. These ions must be named in your parameter files according to the following list for them to be interpreted properly: `WAT`, `Cl*`, `CIO`, `Cs+`, `IB`, `K*`, `Li+`, `MG*`, `Na+`, `Rb+`, `CS`, `RB`, `NA`, `F`, `CL`.

There are some cases where the solvent or ions present in your system will not be one of those that is by default interpreted by MMGBSA. For example, your system may contain Zn^{2+} as an ion. In that event, you will need to add an additional line to your input file so that it looks like the following.

```
Input file for running GB
&general
  verbose=1, interval=1, startframe=1
  strip_mask=:WAT:Cl-:Na+:ZN,
/
&gb
  igb=5, saltcon=0.150
/
&decomp
idecomp=1, print_res="1-790"
dec_verbose=1,
/
```

The flag `strip_mask` will override the default stripping used by MMGBSA. As such, the program will only be capable of understanding the solvent / ions listed in this command. In the above example, only water (`WAT`), Cl^- (`Cl-`), Na^+ (`Na+`), and Zn^{2+} (`ZN`) will be understood. The Zn^{2+} ion is not understood by default, but this line will allow MMGBSA to understand it. However, to reiterate, the program will only understand the solvent and ions listed in this line. If you want it to understand another solvent or ion, you'll need to add the residue name that this solvent / ion is given to this line.

It should be noted that special care needs to be given to parameter topology creation in *tleap* when dealing with atypical solvent and ions. The solvated `prmtop` loaded by MMGBSA will contain all solvent and ions, whereas the complex `prmtop` should not contain any solvent or ions. If you intend on running MMGBSA on a system with unique solvent / ions, your `prmtop`

files need to be built carefully. The following *tleap* input file example correctly builds parameters to account for unique ions.

```
source leaprc.protein.ff14SB
source leaprc.water.tip3p
loadamberparams frcmod.ionsjc_tip3p
COV=loadpdb COVID_Spike_minimized.pdb
ACE=loadpdb hACE2_minimized_No_Zn.pdb
ZN=loadpdb Zn_Only.pdb
com=combine { COV ACE }
coz=combine { com ZN }
saveamberparm ACE hACE2_minimized.prmtop hACE2_minimized.inpcrd
saveamberparm COV 6vw1_COVID_Spike_minimized.prmtop
6vw1_COVID_Spike_minimized.inpcrd
saveamberparm ZN Zn.prmtop Zn.inpcrd
saveamberparm com 6vw1_COVID_Spike_hACE2.prmtop 6vw1_COVID_Spike_hACE2.inpcrd
saveamberparm coz 6vw1_COVID_Spike_hACE2_Zn.prmtop
6vw1_COVID_Spike_hACE2_Zn.inpcrd
addIons coz Na+ 0
addIons coz Cl- 0
saveamberparm coz 6vw1_COVID_Spike_hACE2_Zn_ions.prmtop
6vw1_COVID_Spike_hACE2_Zn_ions.inpcrd
solvateOct coz TIP3PBOX 12.0
saveamberparm coz 6vw1_COVID_Spike_hACE2_Zn_solv.prmtop
6vw1_COVID_Spike_hACE2_Zn_solv.inpcrd
quit
```

Note that the receptor and the ligand (COV and ACE) are combined first to form the receptor-ligand complex *com*. A prmtop file is then saved containing only the receptor and ligand (named 6vw1_COVID_Spike_hACE2.prmtop). Next, the receptor-ligand complex *com* is combined with the atypical ion (ZN) to form the combination *coz*. A prmtop file of the ligand-receptor complex containing the unique ion is then saved (named 6vw1_COVID_Spike_hACE2_Zn.prmtop). From here, this combination *coz* then has all other ions and solvent molecules added and the final prmtop file is created (named 6vw1_COVID_Spike_hACE2_Zn_solv.prmtop).

From here, your command line to run MMGBSA will look like the following example.

```
mpirun -np 20 python $AMBERHOME/bin/MMPBSA.py.MPI -O -i mmgbsa_entropy.in -o
MMGBSA_Entropy_6vw1_COVID_Spike_hACE2_Zn_Seed1_100ns.dat -sp
6vw1_COVID_Spike_hACE2_Zn_solv.prmtop -cp 6vw1_COVID_Spike_hACE2.prmtop -rp
6vw1_COVID_Spike_minimized.prmtop -lp hACE2_minimized.prmtop -y *md*nc -eo
MMGBSA_Entropy_6vw1_COVID_Spike_hACE2_Zn_Seed1_100ns.csv
```

Like usual, the solvated complex prmtop loaded into MMGBSA is the final prmtop file. The complex prmtop file you'll load in next will be the prmtop detailing only the receptor-ligand complex (named 6vw1_COVID_Spike_hACE2.prmtop), and not the prmtop detailing the receptor-ligand complex with the atypical ion added (named 6vw1_COVID_Spike_hACE2_Zn.prmtop).

As long as this example is followed, you should have no issues running MMGBSA on your system containing a unique ion. In the event that you have a unique solvent, no special care will need to be taken when using *tleap*, and your final solvated prmtop will be loaded into MMGBSA the same way as usual. However, you will still need to be sure that your input file

properly tells the program to strip out this atypical solvent by adding the residue name of this solvent to the `strip_mask` flag.

Umbrella MD Simulations

Example mdin files for Umbrella MD

Umbrella MD is typically used to generate a free energy profile of a system and can be very useful at identifying transition states, intermediates and the relative stabilities of various states or conformations. Umbrella MD is often used for simulations in which the energy barrier of interest is so high that the likelihood of an unrestrained MD simulation sampling the conformation is rare. This method works by breaking a reaction coordinate into a series of windows and then by applying a restraint, which in this case is a harmonic potential, that forces the reaction coordinate to remain close to the center of that window.

This is a very brief and concise explanation of Umbrella MD. For an example Umbrella MD simulation, please refer to the AMBER Umbrella MD tutorial at <http://ambermd.org/tutorials/advanced/tutorial17/>.

The following website highlights the math and methods behind Umbrella MD and its analysis tool Weighted Histogram Analysis Method (WHAM)

http://biomol.bme.utexas.edu/wiki/index.php/ABS:Umbrella_sampling_and_WHAM

Umbrella MD simulations involve the same four main steps as unrestrained MD: minimization, heating, equilibration, and MD. The same generic input files of unrestrained MD can be used for Umbrella MD simulations. There is just one slight addition that needs to be made to each input file.

Minimization

This is an example of an input `min1.mdin` file for an unrestrained MD simulation.

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncy=1000,
  maxcyc=5000,
  ntp=50,
  cut=8,
  iwrap=1,
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H=',
/
```

Now, for Umbrella MD, the input `min1.mdin` file would look like this.

```
Minimization to relax initial bad contacts, explicit solvent
&cntrl
  imin=1,
  ncy=1000,
  maxcyc=5000,
  ntp=50,
```

```

    cut=8,
    iwrap=1,
    ntr=1,
    restraint_wt=10.0,
    restraintmask='!@H=',
    nmropt=1,
&end
&wt
    type='END',
&end
DISANG=dist.RST
/

```

Notice that the main changes to the input file (outlined below) are located at the bottom.

`nmropt=1` lets the program know that a restraint will be defined and `DISANG=` points to the file that defines the restraint which, in this case, is `dist.RST`. **You will need to add those changes (outlined below) to the end of all of your minimization files.**

```

nmropt = 1,
&end
&wt
    type='END',
&end
DISANG=dist.RST

```

Heating

An example input `heat.mdin` file for an unrestrained MD simulation is outlined below.

```

Explicit solvent initial heating mdin
&cntrl
    imin=0, irest=0, ntx=1,
    ntp=50000, ntwx=50000, ntwr=50000, nstlim=1000000,
    dt=0.002, ntt=3, gamma_ln=5.0, ig=-1,
    ntc=2, ntf=2, cut=8, ntb=2, ntp=1,
    iwrap=1, ioutfm=1, nmropt=1,
    ntr=1, restraint_wt=10, restraintmask=':1-566'
/
&wt
    TYPE='TEMP0', ISTEP1=0, ISTEP2=100000,
    VALUE1=10.0, VALUE2=100.0,
/
&wt
    TYPE='TEMP0', ISTEP1=100001, ISTEP2=500000,
    VALUE1=100.0, VALUE2=300.0,
/
&wt TYPE='END' /

```

For Umbrella MD, the input `heat.mdin` file would look like this.

```

Explicit solvent initial heating mdin
&cntrl
    imin=0, irest=0, ntx=1,
    ntp=50000, ntwx=50000, ntwr=50000, nstlim=1000000,

```

```

    dt=0.002, ntt=3, gamma_ln=5.0, ig=-1,
    ntc=2, ntf=2, cut=8, ntb=2, ntp=1,
    iwrap=1, ioutfm=1, nmropt=1,
    ntr=1, restraint_wt=10, restraintmask=':1-566'
/
&wt
    TYPE='TEMP0', ISTEP1=0, ISTEP2=100000,
    VALUE1=10.0, VALUE2=100.0,
/
&wt
    TYPE='TEMP0', ISTEP1=100001, ISTEP2=500000,
    VALUE1=100.0, VALUE2=300.0,
/
&wt TYPE='END' /
&end
&wt
    type='END',
&end
DISANG=dist.RST
/

```

The heating file already contains `nmropt=1` so you only simply need to add

```

&end
&wt
    type='END',
&end
DISANG=dist.RST

```

To the end of the `heat.mdin`

Equilibration

The same changes made to the heating and minimization files need to be applied to all of the equilibration files.

First, we will look at an example input `eq1.mdin` file for an unrestrained MD simulation.

```

Explicit solvent molecular dynamics constant pressure MD
&cntrl
    imin=0, irest=0, ntx=1,
    ntpr=10000, ntwx=10000, ntwr=10000, nstlim=250000,
    dt=0.002, ntt=3, tempi=300,
    temp0=300, gamma_ln=1.0, ig=-1,
    ntp=1, ntc=2, ntf=2, cut=9,
    ntb=2, iwrap=1, ioutfm=1,
    ntr=1, restraint_wt=10, restraintmask=':1-566'
/

```

Now, for Umbrella MD, the input `eq1.mdin` file would look like this.

```

Explicit solvent molecular dynamics constant pressure MD
&cntrl
    imin=0, irest=0, ntx=1,
    ntpr=10000, ntwx=10000, ntwr=10000, nstlim=250000,

```

```

    dt=0.002, ntt=3, tempi=300,
    temp0=300, gamma_ln=1.0, ig=-1,
    ntp=1, ntc=2, ntf=2, cut=9,
    ntb=2, iwrap=1, ioutfm=1,
    ntr=1, restraint_wt=10, nmropt=1, restraintmask=':1-566'
&end
&wt
    type='END',
&end
DISANG=dist.RST
/

```

Notice that the variable that tells Amber that a restraint will be defined (`nmropt=1`) is located between `restraint_wt=10` and `restraintmask=':1-566'`. This is followed by

```

&end
&wt
    type='END',
&end
DISANG=dist.RST

```

You will continue this process for the rest of the equilibration files up to `eq7.mdin`. Equilibration 7 has no restraints on any atoms in the system. Thus, the line referring to the restraint has been removed (i.e. there is no `ntr=1, restraint_wt=0.1, restraintmask=':1-566'`) in `eq7.mdin`. This file will look slightly different from the rest of the equilibration files.

An example Umbrella MD `eq7.mdin` file would look as follows:

```

Explicit solvent molecular dynamics constant pressure MD
&cntrl
    imin=0, irest=0, ntx=1,
    ntp=10000, ntwx=10000, ntwr=10000, nstlim=250000,
    dt=0.002, ntt=3, tempi=300,
    temp0=300, gamma_ln=1.0, ig=-1,
    ntp=1, ntc=2, ntf=2, cut=9,
    ntb=2, iwrap=1, ioutfm=1, nmropt=1,
&end
&wt
    type='END',
&end
DISANG=dist.RST
/

```

Restrained MD

Here is a sample `md.mdin` file for your typical unrestrained MD simulation

```

Explicit solvent molecular dynamics constant pressure 25 ns MD
&cntrl
    imin=0, irest=1, ntx=5,
    ntp=50000, ntwx=50000, ntwr=-50000, nstlim=12500000,
    dt=0.002, ntt=3, tempi=300,

```

```
temp0=300, gamma_ln=1.0, ig=-1,
ntp=1, ntc=2, ntf=2, cut=9,
ntb=2, iwrap=1, ioutfm=1,
/
```

Now, here is a `md.mdin` for an Umbrella MD simulation.

```
Explicit solvent molecular dynamics constant pressure 25 ns MD
&cntrl
  imin=0, irest=1, ntx=5,
  ntp=50000, ntwx=50000, ntwr=-50000, nstlim=12500000,
  dt=0.002, ntt=3, tempi=300,
  temp0=300, gamma_ln=1.0, ig=-1,
  ntp=1, ntc=2, ntf=2, cut=9,
  ntb=2, iwrap=1, ioutfm=1, nmropt=1
&end
&wt
  type='DUMPFREQ', istep1=50,
&end
&wt
  type='END',
&end
DISANG=dist.RST
DUMPAVE=dist_4.5.dat
/
```

Outside of the main changes made to the input file (shown in bold below), the `md.mdin` has some additional variables.

```
&end
&wt
  type='DUMPFREQ', istep1=50,
&end
&wt
  type='END',
&end
DISANG=dist.RST
DUMPAVE=dist_4.5.dat
```

The `DUMPFREQ` command is used to request that a separate file be created to hold your torsion or distance values. `istep1=50` tells the system to write the simulated distance or torsion every 50 steps to the file indicated by `DUMPAVE=`.

Restraint File

The restraint itself is given in a `.RST` file.

For this example, it's a distance restraint file (`dist.RST`).

```
&rst iat=-1, -1, r1=0.0, r2=4.5, r3=4.5, r4=30.0, rk2=500.0, rk3=500.0,
igr1=385,386,387,388,389,390,391,392,393,394,395,396,397,398,
igr2=6796,6797,6798,6799,6800,6801,6803,6804,6802,6805,6806,/
```

In this example, I am restraining two residues to stay 4.5 angstroms apart from each other. The `iat=-1, -1` variable indicates that the restraints should be placed on the center of mass of the atoms indicated by variables `igr1` and `igr2`. The numbers following `igr1` are the atom numbers

for each atom on one residue. The numbers following `igr2` are the atom numbers for each atom on the other residue. `r1`, `r2`, `r3`, and `r4` define the shape of the potential. `r1` and `r4` are the lower and upper bounds for the distance restraint. You want these values to be as far away from your `r2` and `r3` values as possible so that the potential is essentially harmonic everywhere. `r2` and `r3` indicate the distance in which the residues are restrained to. `rk2` and `rk3` are the force constants. These force constants need to be raised or lowered depending on how well it does or does not restrain the residues to the specified distance. In order to have a perfectly harmonic potential, `rk2` needs to equal `rk3` and `r2` needs to equal `r3`. A perfectly harmonic potential is ideal for using the WHAM code, but it is not required.

Here is an example of a dihedral restraint file (`chi.RST`). It has a very similar setup to that of the `dist.RST`.

```
&rst iat=39,40,42,43, r1=0.0, r2=170., r3=170., r4=360., rk2=30.0, rk3=30.0,
/
```

It is a lot shorter than the `dist.RST` because the `iat` variable is pointing to individual atoms as opposed to a center of mass. In this file, `r1`, `r2`, `r3`, and `r4` still define the shape of the potential. `r1` and `r4` are the lower and upper bounds for the torsion restraint. You want these values to be as far away from your `r2` and `r3` values as possible so that the potential is essentially harmonic everywhere. `r2` and `r3` indicate the torsion in which the atoms are restrained to. `rk2` and `rk3` are the force constants. These force constants need to be raised or lowered depending on how well it does or does not restrain the residues to the specified torsion. In order to have a perfectly harmonic potential, `rk2` needs to equal `rk3` and `r2` needs to equal `r3`. A perfectly harmonic potential is ideal for using the WHAM code, but it is not required.

Analysis of Umbrella MD Simulations

Using gnuplot to generate histograms

The output text data file for an Umbrella MD simulation (in this case `dist_4.5.dat`) looks like this

```
0 4.682
50 4.519
100 4.549
150 4.480
200 4.557
... ..
```

where the first column is the step number and the second column is the distance value. There is no immediate need to convert the first column into time because we only care about the frequency of each distance value. To graph the frequency of the distance values for a simulation, the *gnuplot* input file (`gnuplot.in`) would look like this

```
set term png font '/usr/share/fonts/liberation/LiberationSans-Regular.ttf, 10'
set output "Histogram_Distances.png"
set title "Umbrella MD Histograms"
binwidth=0.2
bin(x,width)=width*floor(x/width)
```

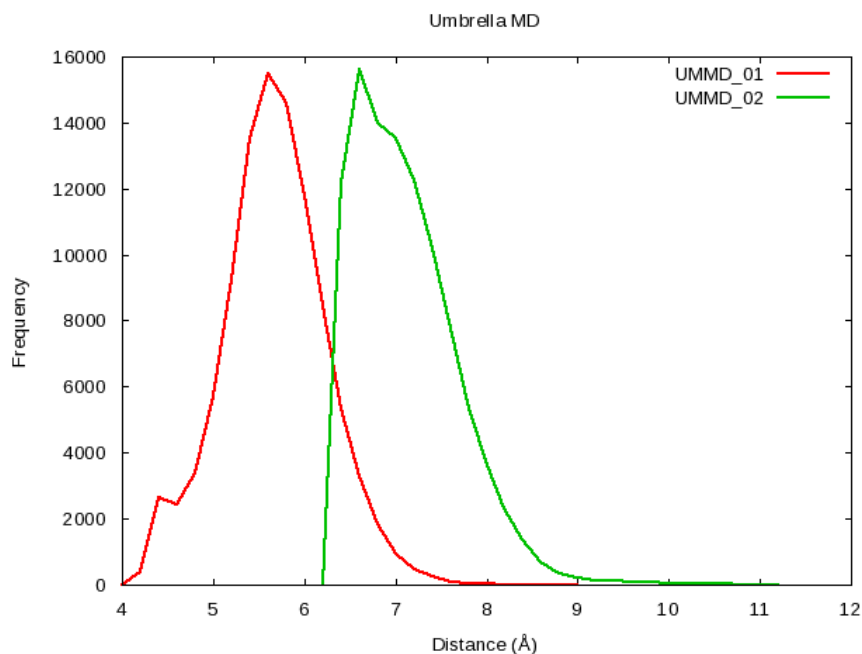
```

set xlabel "Distance (Å)"
set ylabel "Frequency"
set key top right
plot 'dist_4.5.dat' using (bin($2,binwidth)):(1.0) smooth frequency with lines lw 2 ti
"UMMD 01", 'dist_5.5.dat' using (bin($2,binwidth)):(1.0) smooth frequency with lines
lw 2 ti "UMMD 02"

```

The first line (`set term png`) sets the stage for *gnuplot* to save the image of the plot as a .png file. The end of the first line sets the font to *LiberationSans-Regular* in size 10. The second line defines the saved name of the image file as `Histogram_Distances.png`. The third line defines the title placed at the top of the graph as `Umbrella MD Histograms`. The fourth and fifth lines define the manner in which the histogram bins are drawn. For example, `binwidth=0.2` defines the width of the bins of the histogram as 0.2. The next two lines define the x- and y-labels as 'Distance (Å)' and 'Frequency' respectively. The `set key` command tells *gnuplot* where to set your legend. In this case, it will place the legend in the top right of the image. Finally, the `plot` command tells *gnuplot* to plot the second column (`$2`) of two data files (`dist_4.5.dat` and `dist_5.5.dat`) into bins (using `(bin($2,binwidth))`) in which data of the same x-values are replaced by single points having the summed y-values (`smooth frequency`). Those points are then smoothly connected by line segments (`with lines`) that have a line width of 2 (`lw 2`). Lastly, the `plot` command defines the resulting lined histograms as `UMMD 01` and `UMMD 02` in the legend (`ti "UMMD 01"` and `ti "UMMD 02"`).

The resulting image looks like this



Notice how the two histograms overlap. All of the histograms must overlap in order to get accurate results from WHAM later on.

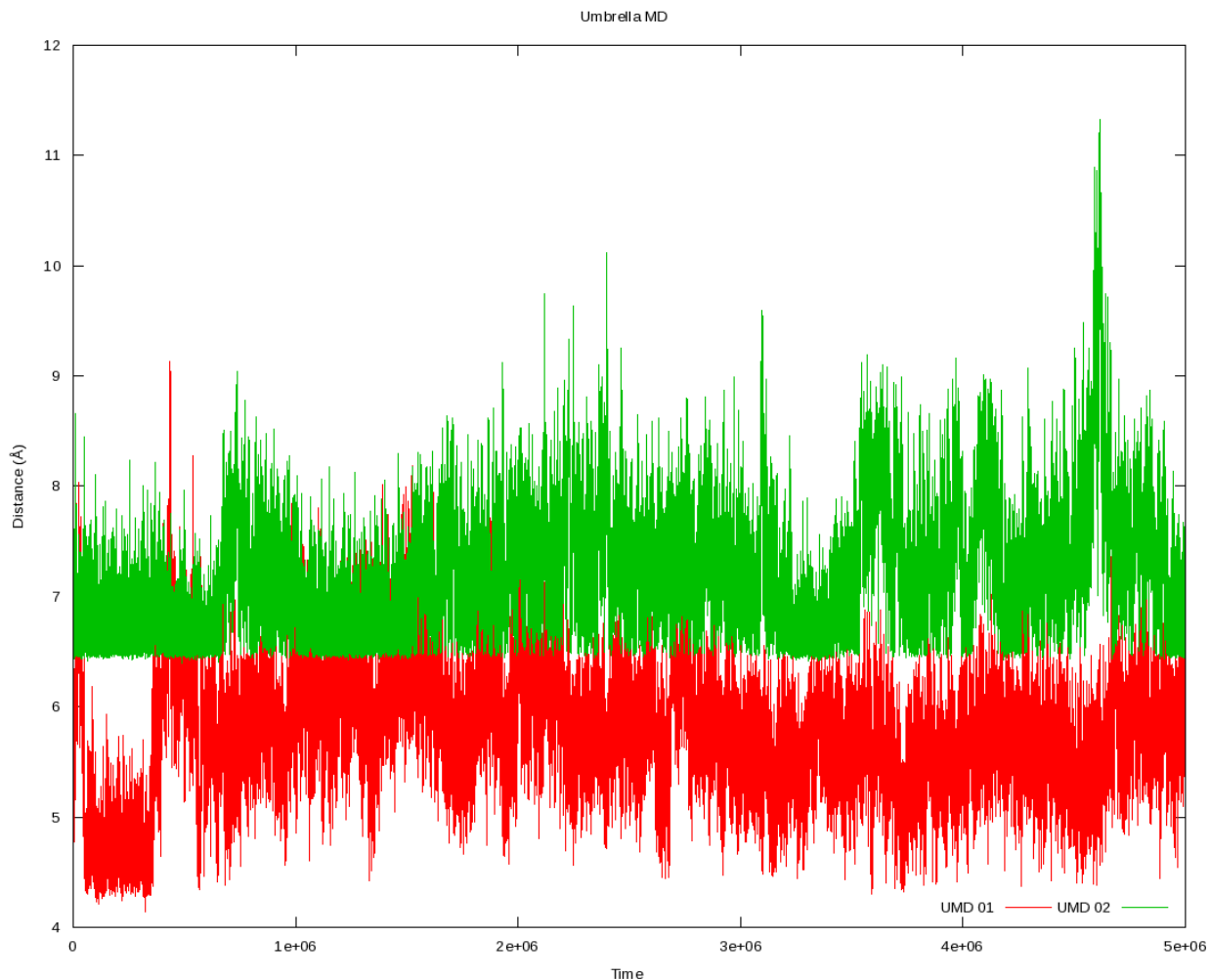
Now to ensure that there are no gaps in your data. You can also plot a graph that graphs your distance values over time (`Distances.png`). To do this, you should include the following in your `gnuplot.in` file.

```

set term png size 110, 900 font '/usr/share/fonts/liberation/LiberationSans-
Regular.ttf, 10'
set output "Distances.png"
set title "Umbrella MD"
set xlabel "Time"
set ylabel "Distance (Å)"
set key bottom right horizontal
plot 'dist_4.5.dat' w l lw .5 ti "UMD 01", 'dist_5.5.dat' w l lw .5 ti "UMD 02"

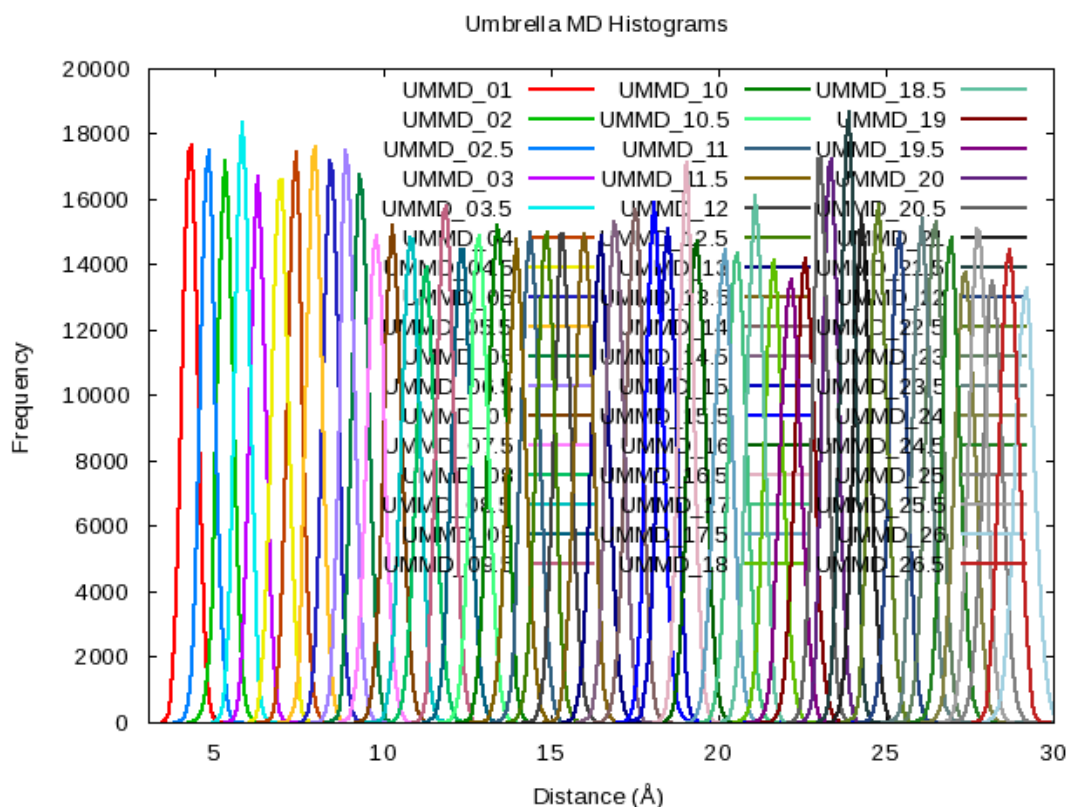
```

The setup of this input *gnuplot* file is fairly identical to the *gnuplot* setup for a *rmsd.dat* file. The resulting plot would look a little like this



An ideal graph of this would have as little white space as possible between each simulation. This is seen in the above example.

Once you have a complete sampling distribution for your Umbrella MD simulations (examples of complete sampling distributions shown below), you are ready to begin your WHAM calculations.



Weighted Histogram Analysis Method (WHAM)

WHAM is used to generate the graph of the PMF from the data obtained from Umbrella MD simulations.

Downloading and Compiling Wham

Downloading WHAM

You can download the WHAM software from the following website:

<http://membrane.urmc.rochester.edu/content/wham>

Compiling WHAM

1. Untar the `wham-release-2.0.0.tgz` file using the command `tar -xzf wham-release-2.0.0.tgz`. This will create a directory `wham/`.
2. Cd into your wham directory (`cd /home/netid/Downloads/wham`) and then type in the following commands
 - a. `make clean`
 - b. `make`

These commands will delete all object files and executables and build the wham executable.

3. Copy the `wham*` file from the `/home/netid/Downloads/wham/wham/` directory over to your `/home/netid/bin` directory. If you do not currently have a `bin` directory in your `home` folder, make one using `mkdir bin`.

Making WHAM Executable

Now, we must add wham to PATH so that we can execute the `wham*` script by simply typing in the command `wham`

1. First, view your `.cshrc` text file within your home folder
 - a. Switch over to your home directory (`cd /home/netid`)
 - b. List your hidden text files (`ls -a`)
 - c. View your `.cshrc` file using the command `vi .cshrc`
2. Insert the following command into your `.cshrc` file if you don't have it already


```
set path=($path ~/bin)
```
3. You are now ready to run WHAM! From now on, any script that you place in your `/home/netid/bin` directory will be automatically added to PATH.

Executing WHAM

To run WHAM, you simply need to use the following command within the directory in which you would like to store your data files.

```
wham {P|Ppi|Pval} hist_min hist_max num_bins tol temperature numpad
      metadatafile freefile [num_MC_trials randSeed]
```

A more in depth explanation of this command can be found in the WHAM manual at: <http://membrane.urmc.rochester.edu/sites/default/files/wham/doc.pdf>

The Amber Umbrella MD tutorial recommends the following command line for dihedral restraints:

```
wham -P hist_min hist_max num_bins tol temperature num_pad meta.dat
      result.dat
```

where

- `P` means that the reaction coordinate is periodic (ie $0^\circ = 360^\circ$)
- `hist_min hist_max` mean that it will generate a PMF within the range defined by `hist_min` and `hist_max`
- `Num_bins` are the number of bins that will be used to generate the PMF. This number is relative to the number of data points within your data files so you will need to experiment with this number
- `Tol` is the accepted tolerance for the iterations. It is suggested that `tol` should be set to 0.01
- `Temperature` should be set to the temperature in which the simulations were ran if they were all ran at the same temperature
- `Num_pad` is typically set to 0 for wham 1-d

- `Meta.dat` is the input file for WHAM
- `Result.dat` is the output file from WHAM

The WHAM command for distance restraints is very similar to that of the dihedral. It looks like this

```
wham hist_min hist_max num_bins tol temperature num_pad meta.dat result.dat
```

Notice that the only difference between the command for the distance restraint and the dihedral restraint is that the distance command lacks the `-p`.

Before executing WHAM, you must first create a `meta.dat` file that includes the following

```
<filename> <harmonic potential minimum in degrees or angstroms> <force  
constant in the correct units>
```

The input `meta.dat` file tells WHAM the filename of each file, what the minimum harmonic potential was, and what the force constant used was.

Here is an example of a `meta.dat` file

```
dist_4.5.dat 4.5 0.0304617  
dist_5.5.dat 5.5 0.330815  
dist_6.5.dat 6.5 609.235  
dist_7.5.dat 7.5 60.9235  
... ..
```

A note on units. Amber and WHAM use two different force constant units. Thus, you must multiply your original force constants by $2(\pi/180)^2$ or 0.0006092. For example, if your original force constant was 200 then your force constant in WHAM's units is 0.12184.

Analysis of Wham Calculations

WHAM does the following while running

1. Makes histograms
2. Modifies the histograms depending on the potential used
3. Converts each histogram to a free energy curve snippet
4. Iteratively aligns all curve snippets using a least squares fit
5. Writes the complete PMF curve to the output file

In this case, the output file is `result.dat` and it looks like this.

```
#Coor    Free  +/-      Prob      +/-  
4.500625 24.510591 -nan 0.000000 -nan  
4.501875 24.608179 -nan 0.000000 -nan  
4.503125 24.721963 -nan 0.000000 -nan  
4.504375 24.067302 -nan 0.000000 -nan  
... ..
```

To plot your PMF, you would need to extract the first two columns of the `result.dat` file. To extract the first two columns, you can use the following command:

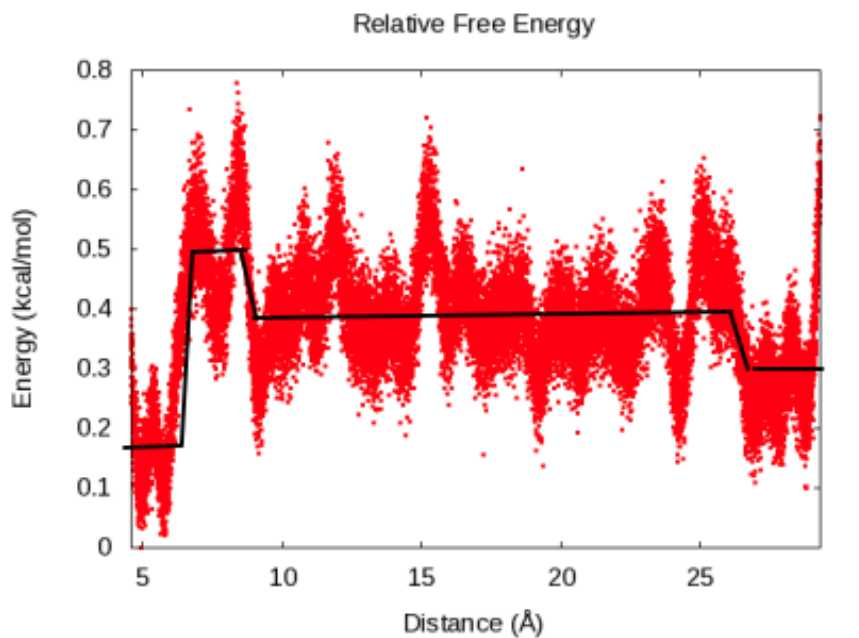
```
cat result.dat | awk '{print$1,$2}' > pmf.dat
```

You can now plot the `pmf.dat` file using a similar *gnuplot* command as the `rmsd.dat` file

```
set term png font '/usr/share/fonts/liberation/LiberationSans-Regular.ttf, 14'
```

```
set output "PMF.png"
set title "Free Energy Surface"
set xlabel "Distance (Å)"
set ylabel "Energy (kcal/mol)"
set nokey
plot 'pmf.dat' w l lw 3
```

The resulting graph looks like this



From this graph, you can see that the energy barrier for pulling the two residues apart is really high.

Amber Troubleshooting

My simulation keeps dying at minimization and heating

Problem:

Initially, the minimization and heating files continuously failed when running AMBER. We decided to run the initial steps using Sander on CPUs start from heating onwards on GPUs. There are no error files being printed, for example, no amber.out file. There are also no error messages being printed on the command lines. However, the heating restart file is completely messed up with spacing, blank lines and asterisks. It looks like this:

[illegible]

Also, in the minimization '.mdout' files, this message is repeated:

... RESTARTED DUE TO LINMIN FAILURE ...

Possible Solution:

Have you visualized your restart files in VMD? Minimization 1 looks fine, but all the restart files starting from min2 look strange, with some atoms being really far away from where they are supposed to be. This explains why the BOND energy in your mdout files is really high (thus all the *********). I wonder if this is because the system is too large, making it difficult to keep track of all the values. Try setting `ntxo=2` in your minimization `mdin` files. This will print your `.rst7` files as binary instead of ASCII so you don't have to worry about the actual spacing for the numerical coordinate values. Once you change `ntxo=2` re-run your minimizations and then check the final minimized structure in VMD to see if it looks normal.

Problem:

I made the changes to the minimization .mdin files and also tried to look at the last .rst7 file in VMD but many errors showed up and it did not open. The .rst7 files for minimization look really unusual. The error in the minimization .mdout files now looks like this:

NSTEP	ENERGY	RMS	GMAX	NAME	NUMBER
1	-6.6838E+09	6.5318E+05	8.3885E+06	HD21	1168
BOND =	2107796.1346	ANGLE =	55967.9215	DIHED =	4087.5122
VDWAALS =	34706399.0791	EEL =	-5156.8663	EGB =	-4672.9014
1-4 VDW =	*****	1-4 EEL =	1839.1536	RESTRAINT =	0.0000
....	RESTARTED DUE TO LINMIN FAILURE	...			
....	RESTARTED DUE TO LINMIN FAILURE	...			
....	RESTARTED DUE TO LINMIN FAILURE	...			
....	RESTARTED DUE TO LINMIN FAILURE	...			

Solution:

I am currently running minimization for your system in parallel using sander.MPI on the CPUs and I am not receiving any LINMIN FAILURE messages. And there are no ***** energy

values being printed in the mdout files. Everything appears to be running smoothly. Once the minimizations finished, I started running heating using pmemd.cuda on a GPU and so far have seen no problems. I have simulated now into the unrestrained MD calculation and have had no issues running your system after minimizing using the Amber CPU code (sander.MPI).

Additional Notes:

Most errors in simulations will occur in minimization, and many times it is possible to use the sander.MPI code to solve these issues. Issues can emerge from a wide range of reasons, so you need to be careful. If you're running MD on an apo protein, as long as you've properly prepped your structure the issue will almost always be due to sizing. In this case, it is sufficient to run minimization using the sander.MPI code. You can invoke minimization using sander.MPI using a script like the following:

```
#!/bin/csh -f
module load amber/18

# Run minimization using local CPUs -- Only use if an error with GPU minimization occurs i.e.
# Illegal Memory Access

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min1.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c 3bcz_chainA_10mer_reorient2_solv.inpcrd -ref
3bcz_chainA_10mer_reorient2_solv.inpcrd -o min1.mdout -r min1.rst7 -x min1.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min2.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min1.rst7 -ref min1.rst7 -o min2.mdout -r min2.rst7 -x
min2.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min3.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min2.rst7 -ref min2.rst7 -o min3.mdout -r min3.rst7 -x
min3.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min4.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min3.rst7 -ref min3.rst7 -o min4.mdout -r min4.rst7 -x
min4.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min5.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min4.rst7 -ref min4.rst7 -o min5.mdout -r min5.rst7 -x
min5.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min6.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min5.rst7 -ref min5.rst7 -o min6.mdout -r min6.rst7 -x
min6.nc

mpirun -np 10 $AMBERHOME/bin/sander.MPI -O -i min7.mdin -p
3bcz_chainA_10mer_reorient2_solv.prmtop -c min6.rst7 -ref min6.rst7 -o min7.mdout -r
min7.rst7 -x min7.nc
```

The flag `-np` in this instance is simply the number of processors you intend on running your job on. Running on more processors will always increase your job speed. Refer to the table [here](#) for detailed information on what processors are available where.

I can't get MMGBSA/MMGBSA to run properly

MMGPBSA / MMGBSA problems are quite common and can occur for a variety of different reasons.

Problem:

I tried to start an MMGBSA calculation, but it didn't start due to an issue with the number of atoms in the solvated prmtop.

Solution:

First and foremost, check that you've loaded in the proper prmtop files. It sounds dumb, but it's actually a lot more common than you might think. I've run hundreds of MMGBSA calculations in my time, and I still make mistakes like this. This is the most common cause of this problem. Check the names of the files and ensure that the correct prmtop files are loaded in after the correct flags. If you've ensured that all of your files are correct, then there are a couple of options.

First, we need to determine how many atoms are missing. If you notice a large number of atoms are missing (meaning in the thousands) the issue is probably with the solvated prmtop. Once again, make sure you're loading in the proper prmtop. To be absolutely sure it's the correct one, make sure that this prmtop name matches the one that was used in your MD scripts. If it is indeed the correct one, I would first recommend trying to make a new solvated prmtop by just running *tleap* again. Then move this over and try to run MMGBSA again. This is to just make sure that the file wasn't corrupted when rsyncing over from another workstation. If MMGBSA still will not run, then there unfortunately are not a lot of options. With thousands of atoms missing, you'll need to rerun the simulation. It's rare, but sometimes there are glitches in the program and atoms are randomly deleted.

If there are only a small number of atoms, there is an option (assuming that this too is an issue with your solvated prmtop file). Normally when we run MMGBSA we use the flag `-y *md0*nc` to load in the 25ns chunks produced by Amber. Sometimes, one of these files may be missing a few water molecules. This is not a big deal, as a couple of water molecules missing from the box don't have too large of an effect on the data. In order to fix this problem, what you can do is use *cpptraj* to create an `imaged_solv.nc` file. This isn't difficult, and instructions on how to do so can be found [here](#). Next, load in `imaged_solv.nc` after the flag `-y` instead of the 25ns chunks. From here, you're good to run MMGBSA. It should run without issue. This is because this `imaged_solv.nc` was created in *cpptraj* using your solvated prmtop. As such, the `imaged_solv.nc` will be corrected according to the solvated prmtop. You will not see any atom number disagreement errors again (if you still do though, you're going to have to rerun the simulation).

If there are any atom number disagreements with the complex prmtop, receptor prmtop, or ligand prmtop, you have no choice but to rerun the simulations. This is because every atom in these prmtops is critical to the system. A few water molecules missing from the solvent box in your solvated prmtop is not a big deal, as there are hundreds if not thousands of water molecules in the box. However, a single atom missing from (for example) your ligand prmtop means that there is a fundamentally important atom missing from your ligand in the simulation. Therefore, as I stated earlier, you'll have to rerun the simulation.

Problem:

My MMGBSA calculation started correctly, but when I open up `MMGBSA_Dbind.csv` it looks like it did not calculate the energy of all of the frames in my simulation. Example: a 500ns simulation is loaded into MMGBSA, but for some reason it appears that only 4782 frames were calculated as opposed to 5000 frames.

Solution:

First, go ahead and make sure that your simulation actually did run to term. Use `cat mdinfo` in the directory where you ran MD, and check the number at the top of the file in the `TIME (PS)` section. In this example, a 500ns simulation that ran to completion would tell you it ran for *at least* 500500.00 picoseconds. Converted to nanoseconds, this is 500.5 ns. Note that the 0.5ns represents the half-ns of data generated in step 7 of equilibration. If you see 500000.00 ps for example, this means the simulation only ran for 4995 of 5000 frames.

If the simulation did run to completion, then the most likely reason for this error is a power outage that resulted in your simulation stopping, forcing you to restart it. Any abrupt termination of MD results in an `md0#.nc` file that is missing a proper termination line. As such, while the data in this file is correct, MMGBSA will not read it due to the fact that it is missing this termination line. You can correct this issue very easily. What you can do is use `cpptraj` to create an `imaged_solv.nc` file. This isn't difficult, and instructions on how to do so can be found [here](#). Next, load in `imaged_solv.nc` after the flag `-y` instead of the 25ns chunks. From here, you're good to run MMGBSA. It should run without issue. This is because `cpptraj` essentially takes all of the frames from your `md0#.nc` files and compiles them into a single NETCDF. Therefore, a missing termination line from one of your individual `md0#.nc` files doesn't matter.